CODE DOCUMENTATION · BUILT 2026-10-20 · 30 FILES · 3,928 LINES

XRay*Lens* AI: source code

This document contains every file written for XRayLens AI. The model weights (Purino/resnet50-chest-xray-pneumonia) and the evaluation datasets are third-party and downloaded at runtime by the scripts below. Run order: `pip install -r requirements.txt` → `streamlit run app.py`. To reproduce the evaluation: `scripts/fetch_eval_data.py` → `scripts/run_evaluation.py` → `scripts/build_gallery.py` → `pytest`.

§ Architecture

```
upload / gallery ─► inputs.py ─► model.py ──────────► scores, label
 (PNG, JPEG, DICOM)  validate,    letterbox 320 px,
                     warn         ResNet-50
                                      │
                                      ├─► explainability.py ─► Grad-CAM (logit-difference), area-normalised
                                      │                        attribution, randomisation sanity check
                                      └─► perturbations.py ──► 8 edits, re-scored
results/scores.csv ─► evaluation.py ─► live metrics at any threshold (Reliability page)
```

§ Contents

#	File	Lines
01	app.py	19
02	views/examine.py	241
03	views/reliability.py	271
04	views/provenance.py	75
05	xraylens/model.py	114
06	xraylens/explainability.py	149
07	xraylens/inputs.py	123
08	xraylens/perturbations.py	82
09	xraylens/evaluation.py	103
10	xraylens/app_state.py	97
11	xraylens/ui.py	247
12	scripts/fetch_eval_data.py	174
13	scripts/run_evaluation.py	157
14	scripts/build_gallery.py	121
15	scripts/export_onnx.py	145
16	scripts/export_web.py	136
17	scripts/check_web_parity.py	155
18	scripts/serve_web.py	24
19	web/js/model.js	91
20	web/js/imaging.js	458

21	web/js/examine.js	280
22	web/js/reliability.js	212
23	web/js/common.js	59
24	web/index.html	98
25	web/vercel.json	21
26	tests/conftest.py	25
27	tests/test_preprocessing.py	109
28	tests/test_inference.py	109
29	requirements.txt	15
30	.streamlit/config.toml	18

01 app.py

```python
"""XRayLens AI: an explainable-AI laboratory for chest radiographs.

Run with:  streamlit run app.py
"""
import streamlit as st

st.set_page_config(page_title="XRayLens AI", page_icon="◐", layout="wide",
                   initial_sidebar_state="collapsed")

from xraylens.ui import inject_theme  # noqa: E402  (set_page_config must come first)

inject_theme()
page = st.navigation([
    st.Page("views/examine.py", title="Examine", default=True),
    st.Page("views/reliability.py", title="Reliability"),
    st.Page("views/provenance.py", title="Provenance"),
], position="top")
page.run()

```

02 views/examine.py

```python
"""Steps 01-04: choose an image, read the output, inspect Grad-CAM, stress-test it."""
from __future__ import annotations

import altair as alt
import numpy as np
import pandas as pd
import streamlit as st
from PIL import Image

from xraylens import ui
from xraylens.app_state import (GALLERY, analyse, get_gallery, get_metrics, get_model, get_scores,
                                image_key, render_overlay, stress)
from xraylens.explainability import randomization_check
from xraylens.inputs import InputError, LoadedImage, assess, load_upload

ui.altair_theme()
gallery = get_gallery()
metrics = get_metrics()
scores_df = get_scores()

ui.html_block("""
<header class="xl-mast">
  <div class="xl-kicker">An explainable-AI laboratory for chest radiographs</div>
  <div class="xl-title">XRay<em>Lens</em></div>
  <p class="xl-dek">A pneumonia classifier gives you an answer. This lab shows how it got there,
  how far you can trust it, and where it breaks.</p>
  <div class="xl-stamp">Research &amp;amp; education only<br>Not for diagnosis</div>
</header>
""")

# ── 01 ──────────────────────────────────────────────────────────────────
ui.section("01", "Choose a radiograph",
           "Start with the curated contact sheet. Each case was picked from the evaluation to teach "
           "one thing. You can also upload a public, de-identified research image.")

if "selected" not in st.session_state:
    st.session_state.selected = ("gallery", gallery[0]["id"]) if gallery else None

sheet, upload_col = st.columns([8, 3], gap="large")
with sheet:
    per_row = 4
    for start in range(0, len(gallery), per_row):
        cols = st.columns(per_row, gap="small")
        for col, item in zip(cols, gallery[start:start + per_row]):
            with col:
                st.image(str(GALLERY / item["thumb"]), use_container_width=True)
                ui.html_block(f'<div class="xl-sheet-cap"><b>{ui.esc(item["title"])}</b>'
                              f'{ui.esc(item["subtitle"])}</div>')
                active = st.session_state.selected == ("gallery", item["id"])
                if st.button("● Loaded" if active else "Load", key=f"g_{item['id']}",
                             use_container_width=True, type="primary" if active else "secondary"):
                    st.session_state.selected = ("gallery", item["id"])
                    st.rerun()

with upload_col:
    up = st.file_uploader("Upload an image", type=["png", "jpg", "jpeg", "bmp", "tif", "tiff", "webp", "dcm"],
                          help="PNG, JPEG or DICOM, up to 20 MB. Images are processed in memory and not stored.")
    if up is not None and st.session_state.get("last_upload") != up.file_id:
        st.session_state.last_upload = up.file_id
        st.session_state.selected = ("upload", up.file_id)
    ui.html_block('<div class="xl-fig">Use only public, de-identified research images. '
                  'Do not upload real patient records. DICOM tags other than modality and '
                  'view are ignored.</div>')

sel = st.session_state.selected
if sel is None:
    st.stop()
```

```

item = None
try:
    if sel[0] == "gallery":
        item = next(g for g in gallery if g["id"] == sel[1])
        img = Image.open(GALLERY / item["file"])
        img.load()
        loaded = LoadedImage(image=img, source=item["title"], warnings=assess(img))
    else:
        if up is None:
            st.session_state.selected = ("gallery", gallery[0]["id"]) if gallery else None
            st.rerun()
        loaded = load_upload(up.getvalue(), up.name)
except InputError as exc:
    ui.note(f"<b>Can’t analyse this file.</b> {ui.esc(exc)}", "warn")
    st.stop()

model, cfg = get_model()
key = image_key(loaded.image)
with st.spinner("Running ResNet-50 and Grad-CAM…"):
    res = analyse(loaded.image, key, item.get("mask") if item else None)
pred, thr = res["pred"], cfg["threshold"]
p = pred.pneumonia_score

# ── 02 ──────────────────────────────────────────────────────────────
ui.section("02", "Read the model’s output",
           "The model returns two softmax scores that sum to 1. A fixed threshold of 0.5 turns them into a
label.")

left, right = st.columns([5, 6], gap="large")
with left:
    st.image(res["boxed"], use_container_width=True)
    w, h = loaded.image.size
    ui.html_block(f'<div class="xl-fig"><b>MODEL INPUT</b> · {w}×{h} source → letterboxed to '
                  f'{cfg["image_size"]}×{cfg["image_size"]}, black bars added · '
                  f'{ui.esc(loaded.source)}</div>')
with right:
    saturated = p < 1e-3 or p > 1 - 1e-3
    ui.score_readout(pred.label, p, thr, {
        "PNEUMONIA score": f"{p:.6f}",
        "NORMAL score": f"{pred.scores['NORMAL']:.6f}",
        "Logit margin (z₁ - z₀)": f"{res['logit_margin']:+.2f}",
        "Saturated (<0.001 or >0.999)": "yes" if saturated else "no",
    })

    rs = scores_df[scores_df.cohort == "rsna"]
    share_norm = (rs[rs.finding == "Normal"].score ≥ p).mean()
    share_opac = (rs[rs.finding == "Lung Opacity"].score ≥ p).mean()
    ui.note(f"<b>What the score is, and what it isn’t.</b> {p:.4f} is the network’s output for this image, "
            "not the probability that a patient has pneumonia. For context, from our own external test: "
            f"<b>{ui.pct(share_norm, 0)}</b> of 200 adult radiographs read as <i>normal</i> (RSNA) scored "
            f"at least this high, as did <b>{ui.pct(share_opac, 0)}</b> of 200 with lung opacity.")

if item:
    tags = "".join(f'<span class="xl-tag" style="color:{c}">{ui.esc(t)}</span>' for t, c in item["tags"])
    ui.html_block(f'<div style="margin-top:.6rem">{tags}</div>')
    ui.note(item["story"], "warn" if item.get("seen_in_training") else "cool")
for wmsg in loaded.warnings:
    ui.note(f"<b>Input check.</b> {ui.esc(wmsg)}", "warn")
if loaded.meta:
    ui.html_block('<div class="xl-fig">DICOM · ' + " · ".join(f"{k} {ui.esc(v)}" for k, v in loaded.meta.items())
+ "</div>")

# ── 03 ──────────────────────────────────────────────────────────────
ui.section("03", "See where the model looked",
           "Grad-CAM weights the last convolutional block’s feature maps by how strongly they push "
           "the chosen class’s logit up. Bright regions influenced the output. That does not make them correct.")

c1, c2 = st.columns([3, 2])
with c1:
```

```
    target = st.radio("Explain which class", ["Predicted: " + pred.label, "NORMAL", "PNEUMONIA"],
                      horizontal=True, label_visibility="visible")
with c2:
    alpha = st.slider("Heatmap opacity", 0.0, 0.9, 0.5, 0.05)
cls = pred.label if target.startswith("Predicted") else target
cam = res["cams"][cls]
stats = res["stats"][cls]

a, b, c = st.columns([4, 4, 3], gap="medium")
with a:
    st.image(res["boxed"], use_container_width=True)
    ui.html_block('<div class="xl-fig"><b>A</b> · input as seen by the model</div>')
with b:
    st.image(render_overlay(res["boxed"], cam, alpha, res["lung"]), use_container_width=True)
    lung_txt = " · teal outline = radiologist lung mask" if res["lung"] is not None else ""
    ui.html_block(f'<div class="xl-fig"><b>B</b> · Grad-CAM for {cls}, layer4 (10×10 upsampled){lung_txt}</div>')
with c:
    def bar(label, mass, area):
        ratio = mass / area if area > 0 else float("nan")
        flag = ' style="color:var(--pneumonia)"' if (ratio == ratio and ratio > 1.3 and label ≠ "Inside lungs")
else ""
        return (f'<div><div class="k">{label}</div><div class="v"{flag}>{ui.pct(mass, 0)}</div>'
                f'<div class="ci">of heat · {ui.pct(area, 0)} of pixels · ×{ratio:.2f}</div></div>')
    rows = [bar("Black padding", stats["padding"], stats["padding_area"]),
            bar("Outer 12% frame", stats["border"], stats["border_area"])]
    if "lungs" in stats:
        rows.append(bar("Inside lungs", stats["lungs"], stats["lungs_area"]))
    ui.html_block(f'<div class="xl-stats" style="grid-template-columns:1fr">{"".join(rows)}</div>')
    ui.html_block('<div class="xl-fig">×1.00 is what a uniform, uninformative map would score. '
                  'Heat on padding or the frame (text markers, tubes, collimator edges) is a shortcut warning.
</div>')

if stats["padding"] + stats["border"] > 0.5:
    ui.note("<b>Most of this map sits outside the anatomy</b>, on the black bars or the image frame. "
            "Either there is little evidence for this class, or the model is using acquisition "
            "artefacts. Neither supports a clinical reading.", "warn")

with st.expander("Sanity check: does this map depend on what the model learned?"):
    st.markdown("Adebayo et al. (2018) showed that some saliency maps look the same even when a "
                "network's weights are randomised. Those maps act as edge detectors, not explanations. "
                "Here we destroy the learned weights from the top down and recompute Grad-CAM. "
                "A faithful map should **lose its resemblance** (rank correlation near 0).")
    if st.button("Run randomisation test", key="rand"):
        with st.spinner("Re-initialising fc → layer4 → layer3…"):
            r = randomization_check(model, res["x"], {"NORMAL": 0, "PNEUMONIA": 1}[cls], cam)
        cols = st.columns(4)
        cols[0].image(render_overlay(res["boxed"], cam, .6, None), caption="Trained weights",
use_container_width=True)
        for col, stage, m in zip(cols[1:], r["stages"], r["maps"]):
            col.image(render_overlay(res["boxed"], m, .6, None),
                      caption=f"↳ random {stage['randomised']} · ρ = {stage['spearman']:+.2f}",
use_container_width=True)
        worst = max(abs(s["spearman"]) for s in r["stages"])
        if worst < 0.4:
            ui.note(f"<b>Passes.</b> Every randomised map is weakly correlated with the original (|ρ| ≤
{worst:.2f}), "
                    "so this heatmap depends on the learned weights. That is necessary but not sufficient: "
                    "it can still faithfully show a shortcut.", "cool")
        else:
            ui.note(f"<b>Caution.</b> A randomised model reproduces this map (|ρ| up to {worst:.2f}). "
                    "Some of what you see is image structure, not model reasoning.", "warn")

# ── 04 ──────────────────────────────────────────────────────────────
ui.section("04", "Stress-test this image",
           "Eight edits that do not change the anatomy a radiologist would read. A trustworthy model's "
           "score should hardly move.")
with st.spinner("Scoring 8 perturbed copies…"):
    st_df = stress(loaded.image, key)
base = float(st_df.score.iloc[0])
st_df["flipped"] = (st_df.score ⩾ thr) ≠ (base ⩾ thr)
```

```
st_df["label"] = np.where(st_df.score ≥ thr, "PNEUMONIA", "NORMAL")

chart_df = st_df.drop(columns=["image"]).iloc[1:].copy()
chart_df["baseline"] = base
order = chart_df.edit.tolist()
rule = alt.Chart(chart_df).mark_rule(color=ui.MUTED, strokeWidth=2).encode(
    y=alt.Y("edit:N", sort=order, title=None, axis=alt.Axis(labelLimit=260)), x="baseline:Q", x2="score:Q")
base_pt = alt.Chart(chart_df).mark_point(shape="stroke", size=160, color=ui.BONE, strokeWidth=2,
angle=90).encode(
    y=alt.Y("edit:N", sort=order), x="baseline:Q")
dots = alt.Chart(chart_df).mark_circle(size=140, stroke=ui.INK, strokeWidth=2, opacity=1).encode(
    y=alt.Y("edit:N", sort=order),
    x=alt.X("score:Q", scale=alt.Scale(domain=[0, 1]), title="PNEUMONIA score  (| = original image)"),
    color=alt.Color("label:N", scale=alt.Scale(domain=["NORMAL", "PNEUMONIA"], range=[ui.NORMAL, ui.PNEUMONIA]),
                    legend=alt.Legend(title="Label after edit")),
    tooltip=[alt.Tooltip("edit:N", title="Edit"), alt.Tooltip("score:Q", format=".5f", title="Score"),
             alt.Tooltip("baseline:Q", format=".5f", title="Original"), alt.Tooltip("flipped:N", title="Label
flipped")])
thr_rule = alt.Chart(pd.DataFrame({"t": [thr]})).mark_rule(strokeDash=[4, 4], color=ui.BONE,
opacity=.5).encode(x="t:Q")
st.altair_chart((thr_rule + rule + base_pt + dots).properties(height=300), use_container_width=True)

flips = st_df[st_df.flipped]
thumbs = st.columns(len(st_df))
for col, r in zip(thumbs, st_df.itertuples()):
    col.image(r.image, use_container_width=True)
    mark = " ⟲ flipped" if r.flipped else ""
    col.markdown(f'<div class="xl-fig"><b>{ui.esc(r.edit)}</b><br>{r.score:.4f}{mark}</div>',
unsafe_allow_html=True)

rob = metrics["robustness"]
mk, inv = rob["marker"], rob["invert"]
summary = (f"<b>{len(flips)} of 8 edits flip this image's label.</b> " if len(flips) else
           "<b>No edit flips this image's label.</b> ")
ui.note(summary + f"Across {mk['n']} external adult radiographs, burned-in text markers flipped "
        f"{ui.pct(mk['flip_rate'])} of labels, {mk['to_pneumonia']} toward PNEUMONIA and {mk['to_normal']} "
        f"toward NORMAL. Inverting greyscale flipped {ui.pct(inv['flip_rate'])}. Edits that only add "
        "artefacts push the model one way, toward PNEUMONIA.")

ui.html_block('<div style="height:1.2rem"></div>')
st.page_link("views/reliability.py", label="Next: how reliable is this model across 1,462 images? →")
ui.html_block('<div class="xl-foot">XRayLens AI · Model: Purino/resnet50-chest-xray-pneumonia (research only) · '
              'Gallery images: Kermany et al. 2018 (CC BY 4.0) and NLM Montgomery/Shenzhen sets · '
              'Outputs are not medical advice.</div>')

```

03 views/reliability.py

```python
"""Step 05: how reliable is the model? Everything here is recomputed from results/scores.csv."""
from __future__ import annotations

import altair as alt
import numpy as np
import pandas as pd
import streamlit as st

from xraylens import evaluation as ev
from xraylens import ui
from xraylens.app_state import get_metrics, get_scores

ui.altair_theme()
M = get_metrics()
S = get_scores()

COHORT_NAMES = {"kermany_test": "Kermany test (pediatric, Guangzhou)", "rsna": "RSNA (adult, NIH)",
                "montgomery": "Montgomery (adult, USA)", "shenzhen": "Shenzhen (adult, China)"}

k = M["cohorts"]["kermany_test"]
r = M["cohorts"]["rsna_normal_vs_opacity"]

ui.html_block(f"""
<header class="xl-mast">
  <div class="xl-kicker">Reliability report · {len(S):,} images · 4 cohorts · every number reproducible</div>
  <div class="xl-title" style="font-size:clamp(3rem,7.5vw,6.2rem)">Two numbers,<br><em>one model</em></div>
  <p class="xl-dek">The same weights look perfect or badly miscalibrated, depending on which images you test them on.</p>
</header>
<div class="xl-hero">
  <div>
    <div class="xl-kicker" style="color:var(--muted)">Kermany “test” folder · n = {k['n']}</div>
    <div class="n"><span class="xl-strike">{k['auc']:.3f}</span> <small>AUC</small></div>
    <div class="cap">Sensitivity {ui.pct(k['sensitivity'])} · specificity {ui.pct(k['specificity'])}.
    Not a valid test: the published weights were refit on these exact images.</div>
  </div>
  <div>
    <div class="xl-kicker">RSNA adults · external · n = {r['n']}</div>
    <div class="n" style="color:var(--pneumonia)">{ui.pct(r['specificity'])}<small> specificity</small></div>
    <div class="cap">{ui.ci(r['specificity_ci'])}. Normal adult radiographs that the model correctly calls
    NORMAL at the shipped 0.5 threshold. AUC {r['auc']:.3f} ({r['auc_ci'][0]:.3f}–{r['auc_ci'][1]:.3f}).</div>
  </div>
</div>
""")

# ── 01 ──────────────────────────────────────────────────────────────────
ui.section("01", "The leakage trap",
           "The easiest way to make a medical model look good is to test it on data it has already seen.")
a, b = st.columns([6, 5], gap="large")
with a:
    st.markdown(
        "The model card says the final weights were *“retrained on all cleaned images from the original "
        "train, test, and validation folders”* and that *“it has no independent test evaluation.”* "
        "So the 624-image Kermany test folder, the benchmark most tutorials report, is **training data** "
        "for this model. We scored it anyway, to show how misleading that number would be.")
    st.markdown(
        "Kermany images are all from children aged one to five at a single hospital. "
        "We could not find a second pediatric pneumonia set with a compatible licence in time, so our external "
        "tests use **adult** radiographs. That measures generalisation and population shift together, "
        "and we report it that way.")
with b:
    sat = M["saturation"]
    ui.stats_row([
        ("Kermany saturated", ui.pct(sat["kermany_test"], 0), "score < 0.001 or > 0.999"),
        ("RSNA saturated", ui.pct(sat["rsna"], 0), "same definition"),
        ("RSNA calibration error", f"{M['calibration']['rsna_normal_vs_opacity']['ece']:.2f}", "ECE, 10 bins"),
    ])
```

```
    ui.html_block('<div class="xl-fig">Near-0 or near-1 outputs on unfamiliar adult images show overconfidence, '
                  'not certainty. The model card itself warns that softmax outputs are not calibrated probabilities.</div>')

# ── 02 ──────────────────────────────────────────────────────────────────
ui.section("02", "Move the threshold",
           "The shipped threshold is 0.5. Because outputs pile up near 1, useful thresholds sit extremely "
           "close to 1. Every number below is recomputed live from per-image scores.")

ops = M.get("operating_points", {})
choices = {"0.5 (shipped)": 0.5}
for t in (0.9, 0.99, 0.999, 0.9999):
    choices[str(t)] = t
for name, op in ops.items():
    choices[f"spec ≥ {name.split('_')[-1]}% on RSNA ({op['threshold']:.7f})"] = op["threshold"]
choice = st.select_slider("Decision threshold on the PNEUMONIA score", options=list(choices), value="0.5 (shipped)")
thr = choices[choice]

rs = S[(S.cohort == "rsna") & S.finding.isin(["Normal", "Lung Opacity"])]
m = ev.binary_metrics(rs.label.values, rs.score.values, thr)
c = m["confusion"]
left, right = st.columns([5, 6], gap="large")
with left:
    cm = pd.DataFrame([
        {"true": "Lung opacity", "pred": "PNEUMONIA", "n": c["tp"], "kind": "correct"},
        {"true": "Lung opacity", "pred": "NORMAL", "n": c["fn"], "kind": "missed"},
        {"true": "Normal", "pred": "PNEUMONIA", "n": c["fp"], "kind": "false alarm"},
        {"true": "Normal", "pred": "NORMAL", "n": c["tn"], "kind": "correct"},
    ])
    base = alt.Chart(cm).encode(
        x=alt.X("pred:N", sort=["NORMAL", "PNEUMONIA"], title="Model says", axis=alt.Axis(orient="top", labelAngle=0)),
        y=alt.Y("true:N", sort=["Normal", "Lung opacity"], title="RSNA label"))
    heat = base.mark_rect(stroke=ui.INK, strokeWidth=3).encode(
        color=alt.Color("n:Q", scale=alt.Scale(range=["#141c21", "#5c6b70"]), legend=None),
        tooltip=["true", "pred", "n", "kind"])
    txt = base.mark_text(fontSize=26, font="Instrument Serif").encode(
        text="n:Q", color=alt.value(ui.BONE))
    sub = base.mark_text(dy=22, fontSize=10).encode(text="kind:N", color=alt.value(ui.MUTED))
    st.altair_chart((heat + txt + sub).properties(height=260), use_container_width=True)
    ui.html_block(f'<div class="xl-fig"><b>RSNA</b> · 200 normal vs 200 lung-opacity adults · threshold {thr:g}</div>')
with right:
    ui.stats_row([
        ("Sensitivity", ui.pct(m["sensitivity"]), ui.ci(m["sensitivity_ci"])),
        ("Specificity", ui.pct(m["specificity"]), ui.ci(m["specificity_ci"])),
        ("Precision", ui.pct(m["precision"]), ui.ci(m["precision_ci"])),
        ("F1", f"{m['f1']:.3f}", "harmonic mean of PPV and sensitivity"),
    ])
    rows = []
    for (cohort, finding), g in S.groupby(["cohort", "finding"]):
        if finding in ("Pneumonia", "Lung Opacity"):
            continue
        fr = ev.flag_rate(g.score.values, thr)
        rows.append({"group": f"{COHORT_NAMES[cohort].split(' (')[0]} · {finding.replace('No Lung Opacity / Not Normal', 'other abnormality')}",
                     "rate": fr["rate"], "lo": fr["rate_ci"][0], "hi": fr["rate_ci"][1], "n": fr["n"],
                     "kind": "Normal" if finding == "Normal" else "Out of scope"})
    fr_df = pd.DataFrame(rows).sort_values("rate", ascending=False)
    order = fr_df.group.tolist()
    bars = alt.Chart(fr_df).mark_bar(height=14, cornerRadiusEnd=4).encode(
        y=alt.Y("group:N", sort=order, title=None, axis=alt.Axis(labelLimit=260)),
        x=alt.X("rate:Q", scale=alt.Scale(domain=[0, 1]), axis=alt.Axis(format="%"), title="share called PNEUMONIA (no pneumonia label)"),
        color=alt.Color("kind:N", scale=alt.Scale(domain=["Normal", "Out of scope"], range=[ui.NORMAL, ui.OTHER]),
                        legend=alt.Legend(title=None)),
        tooltip=["group", alt.Tooltip("rate:Q", format=".1%"), alt.Tooltip("lo:Q", format=".1%", title="95% CI low"),
```

```
                  alt.Tooltip("hi:Q", format=".1%", title="95% CI high"), "n"])
    whisk = alt.Chart(fr_df).mark_rule(color=ui.BONE, opacity=.6).encode(y=alt.Y("group:N", sort=order),
x="lo:Q", x2="hi:Q")
    st.altair_chart((bars + whisk).properties(height=alt.Step(30), title="False-alarm rate where no pneumonia is
labelled"),
                    use_container_width=True)

roc = M["roc"]["rsna_normal_vs_opacity"]
roc_df = pd.DataFrame({"fpr": roc["fpr"], "tpr": roc["tpr"]})
op_df = pd.DataFrame({"fpr": [1 - m["specificity"]], "tpr": [m["sensitivity"]], "label": [f"threshold {thr:g}"]})
roc_c, note_c = st.columns([5, 6], gap="large")
with roc_c:
    diag = alt.Chart(pd.DataFrame({"x": [0, 1], "y": [0, 1]})).mark_line(strokeDash=[4, 4], color=ui.MUTED,
strokeWidth=1).encode(x="x", y="y")
    curve = alt.Chart(roc_df).mark_line(color=ui.BONE, strokeWidth=2, interpolate="step-after").encode(
        x=alt.X("fpr:Q", title="1 - specificity"), y=alt.Y("tpr:Q", title="sensitivity"))
    pt = alt.Chart(op_df).mark_point(size=180, filled=True, color=ui.AMBER, stroke=ui.INK, strokeWidth=2).encode(
        x="fpr:Q", y="tpr:Q", tooltip=["label", alt.Tooltip("fpr:Q", format=".3f"), alt.Tooltip("tpr:Q",
format=".3f")])
    st.altair_chart((diag + curve + pt).properties(height=300, title=f"ROC, RSNA · AUC {r['auc']:.3f}"),
use_container_width=True)
with note_c:
    ui.html_block('<div style="height:2.6rem"></div>')
    ui.note(f"<b>Good ranking, bad threshold.</b> An AUC of {r['auc']:.3f} means the model usually ranks an
opacity image "
            "above a normal one. The shipped 0.5 cut-off just sits in the wrong place for adults. "
            + "".join(f"For {op['specificity'] * 100:.0f}% specificity you need a threshold of
{op['threshold']:.7f}, "
                      f"which keeps {op['sensitivity'] * 100:.0f}% sensitivity. " for op in ops.values())
            + "Choosing that threshold on the same 400 images we report on is optimistic. A real deployment would
fix it on separate data.")

# ── 02b ──────────────────────────────────────────────────────────────────
d = S.copy()
d["group"] = d.cohort.map(lambda x: COHORT_NAMES[x].split(" (")[0]) + " · " + d.finding.replace(
    {"No Lung Opacity / Not Normal": "Other abnormality"})
d["expected"] = np.select([d.finding.isin(["Pneumonia", "Lung Opacity"]), d.finding.eq("Normal")],
                          ["Pneumonia / opacity", "Normal"], "Out of scope")
# Logit scale spreads out the saturated tails, so 0.999 and 0.999999 can be told apart.
d["logit"] = np.log(np.clip(d.score, 1e-7, 1 - 1e-7) / (1 - np.clip(d.score, 1e-7, 1 - 1e-7)))
groups = d.groupby("group").expected.first().sort_values().index.tolist()
d["j"] = np.random.default_rng(0).uniform(size=len(d))  # seeded so dots don't move on rerun
jitter = alt.Chart(d).mark_circle(size=14, opacity=.6).encode(
    x=alt.X("logit:Q", title="model logit of PNEUMONIA score (0 = score 0.5; ±16 ≈ fully saturated)",
            scale=alt.Scale(domain=[-17, 17])),
    y=alt.Y("group:N", sort=groups, title=None, axis=alt.Axis(labelLimit=240)),
    yOffset=alt.YOffset("j:Q"),
    color=alt.Color("expected:N", scale=alt.Scale(domain=["Normal", "Pneumonia / opacity", "Out of scope"],
                                                  range=[ui.NORMAL, ui.PNEUMONIA, ui.OTHER]),
                    legend=alt.Legend(title=None)),
    tooltip=["group", alt.Tooltip("score:Q", format=".6f"), "source_id"])
tl = float(np.log(min(max(thr, 1e-7), 1 - 1e-7) / (1 - min(max(thr, 1e-7), 1 - 1e-7))))
trule = alt.Chart(pd.DataFrame({"t": [tl]})).mark_rule(color=ui.AMBER, strokeDash=[4, 3]).encode(x="t:Q")
st.altair_chart((jitter + trule).properties(height=520, title="Every image's score, by cohort (amber =
threshold)"),
                use_container_width=True)

ui.note("<b>Read the strip chart left to right.</b> Normal adult radiographs (teal) stretch across the whole
range "
        "and many sit far right, fully saturated as PNEUMONIA. No threshold separates them cleanly from lung "
        "opacity, and the 0.5 threshold falls inside the normal cluster.")

# ── 03 ──────────────────────────────────────────────────────────────────
ui.section("03", "Is a score of 0.9 really 90%?",
           "A calibrated model's scores of 0.9 come true about 90% of the time. Bins below show how often RSNA "
           "images in each score band actually had lung opacity.")
cal = pd.DataFrame(M["calibration"]["rsna_normal_vs_opacity"]["bins"])
cal["mid"] = (cal.lo + cal.hi) / 2
cc, ct = st.columns([6, 4], gap="large")
with cc:
```

```
    ideal = alt.Chart(pd.DataFrame({"x": [0, 1], "y": [0, 1]})).mark_line(strokeDash=[4, 4], color=ui.MUTED).encode(
        x=alt.X("x:Q", title="mean model score in bin"), y=alt.Y("y:Q", title="observed share with lung opacity"))
    pts = alt.Chart(cal).mark_circle(color=ui.AMBER, opacity=1, stroke=ui.INK, strokeWidth=2).encode(
        x="mean_score:Q", y="observed_rate:Q", size=alt.Size("n:Q", scale=alt.Scale(range=[60, 900]), legend=None),
        tooltip=[alt.Tooltip("lo:Q", format=".1f"), alt.Tooltip("hi:Q", format=".1f"), "n",
                 alt.Tooltip("mean_score:Q", format=".3f"), alt.Tooltip("observed_rate:Q", format=".3f")])
    st.altair_chart((ideal + pts).properties(height=300), use_container_width=True)
with ct:
    top = cal.iloc[-1]
    ui.stats_row([("ECE", f"{M['calibration']['rsna_normal_vs_opacity']['ece']:.3f}", "0 = perfectly calibrated"),
                  ("Top bin", f"{top['observed_rate'] * 100:.0f}%", f"observed, at mean score {top['mean_score']:.3f} (n={int(top['n'])})")])
    ui.html_block('<div class="xl-fig">Most images land in the top bin, where the model claims near certainty '
                  'and is right far less often. Treat the score as a ranking, not a probability.</div>')

# ── 04 ──────────────────────────────────────────────────────────────────
ui.section("04", "Do heatmaps land on the lungs?",
           "Montgomery County radiographs come with radiologist-drawn lung masks, so we can measure "
           "how much Grad-CAM mass falls inside the lungs compared with the lungs' share of the image.")
att = M["attribution"]["montgomery"]
mg = S[S.cohort == "montgomery"].copy()
mg["enrichment"] = mg.cam_lungs / mg.cam_lungs_area
la, lb = st.columns([4, 6], gap="large")
with la:
    ui.stats_row([("Heat inside lungs", ui.pct(att["lungs_median"], 0), f"median · IQR {ui.pct(att['lungs_iqr'][0], 0)}–{ui.pct(att['lungs_iqr'][1], 0)}"),
                  ("Lungs' share of pixels", ui.pct(att["lungs_area_median"], 0), "median"),
                  ("Enrichment", f"×{att['lungs_enrichment_median']:.2f}", f"{ui.pct(att['lungs_enrichment_gt1'], 0)} of images above ×1")])
with lb:
    hist = alt.Chart(mg).mark_bar(color=ui.AMBER, cornerRadiusEnd=4, binSpacing=2).encode(
        x=alt.X("enrichment:Q", bin=alt.Bin(step=0.25), title="lung enrichment (heat share ÷ area share)"),
        y=alt.Y("count():Q", title="images"), tooltip=["count()"])
    one = alt.Chart(pd.DataFrame({"x": [1]})).mark_rule(color=ui.BONE, strokeDash=[4, 4]).encode(x="x:Q")
    st.altair_chart((hist + one).properties(height=230, title="×1 = no more attention to lungs than a uniform map"),
                    use_container_width=True)
ui.note("<b>What this does and doesn't show.</b> Enrichment above ×1 means the explanation favours lung tissue "
        "beyond chance. Values at or below ×1 mean the map spreads onto the mediastinum, the abdomen, bones or the frame. "
        "Neither proves the model reasons clinically. Grad-CAM is a coarse 10×10 summary of one layer.")

# ── 05 ──────────────────────────────────────────────────────────────────
ui.section("05", "Robustness to edits that shouldn't matter",
           f"Each of {M['robustness']['marker']['n']} external adult radiographs was re-scored after eight edits that do not change the anatomy.")
rb = pd.DataFrame([{"edit": v["label"], "to PNEUMONIA": v["to_pneumonia"], "to NORMAL": v["to_normal"],
                    "flip": v["flip_rate"]} for v in M["robustness"].values()]).sort_values("flip", ascending=False)
long = rb.melt(id_vars=["edit", "flip"], value_vars=["to PNEUMONIA", "to NORMAL"], var_name="direction", value_name="images")
chart = alt.Chart(long).mark_bar(height=16, cornerRadiusEnd=3, stroke=ui.INK, strokeWidth=2).encode(
    y=alt.Y("edit:N", sort=rb.edit.tolist(), title=None, axis=alt.Axis(labelLimit=260)),
    x=alt.X("images:Q", stack="zero", title="labels flipped"),
    color=alt.Color("direction:N", scale=alt.Scale(domain=["to NORMAL", "to PNEUMONIA"], range=[ui.NORMAL, ui.PNEUMONIA]),
                    legend=alt.Legend(title="Flipped")),
    order=alt.Order("direction:N", sort="descending"),
    tooltip=["edit", "direction", "images", alt.Tooltip("flip:Q", format=".1%", title="total flip rate")])
st.altair_chart(chart.properties(height=alt.Step(30)), use_container_width=True)
mk = M["robustness"]["marker"]
ui.note(f"<b>The asymmetry is the finding.</b> Adding a text marker, noise or inversion moves labels almost only toward "
        f"PNEUMONIA (markers: {mk['to_pneumonia']} toward PNEUMONIA, {mk['to_normal']} toward NORMAL). The model treats "
        "“looks unusual” as evidence of disease. One plausible cause, which we have not verified, is that
```

```
pneumonia images in "
 244          "the training set differ systematically in acquisition (portable films, more annotations, lower
contrast). "
 245          "Either way it is a shortcut, not radiology.", "warn")
 246
 247  # ── 06 ──────────────────────────────────────────────────────────────────
 248  ui.section("06", "What this evaluation cannot tell you")
 249  st.markdown("""
 250  - **No in-distribution test exists.** Every Kermany image was used to train the final weights. We can't measure
pediatric performance honestly with this checkpoint.
 251  - **External cohorts are adults.** Poor specificity on RSNA, Montgomery and Shenzhen mixes *population shift*
(children → adults) with genuine generalisation error. We can't separate them.
 252  - **Labels differ.** RSNA “Lung Opacity” is a bounding-box finding suggestive of pneumonia, not a confirmed
diagnosis. Montgomery and Shenzhen labels are for tuberculosis, so their normals are the cleanest negatives we have.
 253  - **Samples are modest.** 200 per RSNA class, 138 Montgomery and 100 Shenzhen images. Confidence intervals are
shown and are wide.
 254  - **Grad-CAM is one view.** It summarises one layer at 10×10 resolution. Passing the randomisation test shows the
maps depend on the weights, not that they capture the model’s full reasoning.
 255  - **Nothing here is clinical validation.** This is a teaching instrument for reading model behaviour critically.
 256  """)
 257
 258  with st.expander("Table view · all headline metrics"):
 259      rows = []
 260      for name, v in M["cohorts"].items():
 261          rows.append({"cohort": name, "n": v["n"], "valid test": v["valid_test"], "sensitivity": v["sensitivity"],
 262                       "specificity": v["specificity"], "precision": v["precision"], "F1": v["f1"], "AUC":
v["auc"]})
 263      st.dataframe(pd.DataFrame(rows), hide_index=True, use_container_width=True)
 264      st.dataframe(pd.DataFrame([{"group": g, **{kk: vv for kk, vv in v.items() if kk ≠ "rate_ci"}} for g, v in
M["flag_rates"].items()]),
 265                   hide_index=True, use_container_width=True)
 266      st.dataframe(rb, hide_index=True, use_container_width=True)
 267      st.download_button("Download per-image scores (CSV)", S.to_csv(index=False), "xraylens_scores.csv",
"text/csv")
 268
 269  ui.html_block(f'<div class="xl-foot">Model revision {M["model"]["revision"][:12]} · evaluated with torch
{M["environment"]["torch"]} on '
 270                f'{M["environment"]["device"]} · reproduce with <code>python scripts/fetch_eval_data.py && python
scripts/run_evaluation.py</code></div>')
 271
```

04 views/provenance.py

```python
"""Model card, data lineage, licences and what this project contributes."""
from __future__ import annotations

import streamlit as st

from xraylens import ui
from xraylens.app_state import get_metrics

M = get_metrics()

ui.html_block("""
<header class="xl-mast">
  <div class="xl-kicker">Provenance · where every pixel and weight came from</div>
  <div class="xl-title" style="font-size:clamp(3rem,7.5vw,6.2rem)">Read the <em>label</em><br>before the result</div>
</header>
""")

ui.section("01", "The model", "We did not train this network. We audited it.")
a, b = st.columns([7, 5], gap="large")
with a:
    st.markdown(f"""
**[Purino/resnet50-chest-xray-pneumonia](https://huggingface.co/Purino/resnet50-chest-xray-pneumonia)**, pinned at revision
`{M['model']['revision'][:12]}`. It is a torchvision ResNet-50 (ImageNet V2 initialisation) fine-tuned for two classes,
NORMAL and PNEUMONIA, at 320 × 320 px with aspect-preserving black padding.

Notes from the model card, quoted or paraphrased:
- Final weights were **refit on all 5,824 cleaned Kermany images**, including the original test folder. *“It has no independent test evaluation.”*
- Its reported metrics (ROC-AUC 0.998) come from a **selection checkpoint** on an internal split, and *“do not evaluate `model.safetensors`.”*
- Source images are from **young children at one centre**. Performance on adults, other hospitals and other diseases is unknown.
- Patient grouping was inferred from filenames. *“Near-duplicate or undocumented patient overlap may remain.”*
- *“Softmax outputs are not clinically calibrated probabilities.”*
- Research only. Not a clinical diagnostic device.
""")
with b:
    ui.note("<b>Why this model?</b> Its card is unusually candid about training scope, leakage and limits. That made it "
            "a good subject for an audit. Each warning above is something XRayLens measures rather than repeats.", "cool")
    ui.stats_row([("Parameters", "23.5 M", "ResNet-50 + 2-way head"), ("Input", "320 px", "RGB, letterboxed"),
                  ("Threshold", f"{M['model']['threshold']}", "fixed, from config.json")])

ui.section("02", "The data", "Four cohorts, used for different questions. Images stay out of the repository.")
st.markdown("""
| Cohort | Who | Used here for | Licence / terms | In the repo? |
|---|---|---|---|---|
| **Kermany et al. 2018**, test folder (624) | Children aged 1–5, Guangzhou | Showing the leakage gap. **Seen in training.** | CC BY 4.0 | Gallery examples only |
| **RSNA Pneumonia Detection Challenge 2018**, sample of 600 | Adults, NIH ChestX-ray14 | External sensitivity and specificity, robustness | RSNA/Kaggle challenge terms, non-commercial research | **No.** IDs and scores only |
| **Montgomery County CXR set** (138) | Adults, USA, TB programme | Adult specificity, Grad-CAM vs lung masks | NLM public release, citation required | Gallery examples only |
| **Shenzhen CXR set**, sample of 100 | Adults, China, TB programme | Out-of-scope disease probe | NLM public release, citation required | Gallery examples only |

Samples are drawn with a fixed seed (`20261020`), and every source is pinned to a Hugging Face commit in
`scripts/fetch_eval_data.py`. One dataset we **rejected**: a Hugging Face set labelled as African patients turned out to
be Kermany images under a new name (identical `IM-xxxx-xxxx` filenames). Testing on it would have repeated the leakage trap.
""")

```

```
ui.section("03", "What XRayLens adds", "Our contribution is the instrument, not the classifier.")
st.markdown("""
1. **Grad-CAM from logits, not softmax.** This model's softmax saturates, so softmax gradients vanish exactly where the model is most confident.
2. **Area-normalised attribution.** Heat on lungs, the frame and padding is always reported against each region's pixel share, so “40% of heat on the lungs” can't hide a 45% area.
3. **Randomisation sanity check** (Adebayo et al. 2018), live for any image.
4. **Perturbation stress lab.** Eight anatomy-preserving edits per image, plus population flip rates.
5. **An evaluation that refuses the easy number.** Leakage made visible, external adult cohorts, Wilson and bootstrap 95% intervals, calibration, and a live threshold explorer.
""")

ui.section("04", "Citations")
st.markdown("""
- Kermany D, Goldbaum M, Cai W, et al. Identifying medical diagnoses and treatable diseases by image-based deep learning. *Cell* 2018;172(5):1122–1131. Data: Mendeley Data, doi:10.17632/rscbjbr9sj.2 (CC BY 4.0).
- Jaeger S, Candemir S, Antani S, et al. Two public chest X-ray datasets for computer-aided screening of pulmonary diseases. *Quant Imaging Med Surg* 2014;4(6):475–477.
- Shih G, Wu CC, Halabi SS, et al. Augmenting the National Institutes of Health chest radiograph dataset with expert annotations of possible pneumonia. *Radiol Artif Intell* 2019;1(1):e180041.
- Selvaraju RR, et al. Grad-CAM: visual explanations from deep networks via gradient-based localization. *ICCV* 2017.
- Adebayo J, et al. Sanity checks for saliency maps. *NeurIPS* 2018.
- Zech JR, et al. Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs. *PLOS Medicine* 2018;15(11):e1002683.
""")

ui.html_block('<div class="xl-foot">XRayLens AI is an educational research prototype. It is not a medical device, '
              'and nothing it displays is medical advice.</div>')

```

05 xraylens/model.py

```python
"""Model loading, preprocessing and inference for the Purino ResNet-50 classifier.

Preprocessing mirrors the model card's ``inference.py`` exactly (EXIF transpose,
RGB, aspect-preserving letterbox pad to 320x320 with black, ImageNet
normalisation). Any deviation here would silently change the model's outputs, so
``tests/test_preprocessing.py`` pins this behaviour.
"""
from __future__ import annotations

import json
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import torch
from PIL import Image, ImageOps
from safetensors.torch import load_file
from torchvision import models
from torchvision import transforms as T

REPO_ID = "Purino/resnet50-chest-xray-pneumonia"
# Pin the exact Hub commit so results are reproducible even if the repo changes.
REVISION = "da2a7724bfa53ca6693fd4e1783b211963ed2a55"
MODEL_FILES = ("config.json", "model.safetensors")
DEFAULT_DIR = Path(__file__).resolve().parent.parent / "models" / "resnet50-chest-xray-pneumonia"


@dataclass(frozen=True)
class Prediction:
    label: str
    scores: dict[str, float]
    threshold: float

    @property
    def pneumonia_score(self) -> float:
        return self.scores["PNEUMONIA"]

    @property
    def margin(self) -> float:
        """Distance of the pneumonia score from the decision threshold (0-0.5)."""
        return abs(self.pneumonia_score - self.threshold)


def download_model(folder: Path = DEFAULT_DIR) -> Path:
    """Fetch config and weights from the Hugging Face Hub if they are not cached."""
    folder = Path(folder)
    if all((folder / f).exists() for f in MODEL_FILES):
        return folder
    from huggingface_hub import hf_hub_download

    folder.mkdir(parents=True, exist_ok=True)
    for name in MODEL_FILES:
        hf_hub_download(REPO_ID, name, revision=REVISION, local_dir=folder)
    return folder


def pick_device() -> torch.device:
    if torch.cuda.is_available():
        return torch.device("cuda")
    # MPS gives ~5x on Apple silicon; Grad-CAM backward works there too.
    if torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")


def load_model(folder: Path = DEFAULT_DIR, device: torch.device | None = None):
    folder = download_model(folder)
```

```
    config = json.loads((folder / "config.json").read_text())
    model = models.resnet50(weights=None)
    model.fc = torch.nn.Linear(model.fc.in_features, config["num_classes"])
    model.load_state_dict(load_file(str(folder / "model.safetensors")))
    model.to(device or pick_device()).eval()
    return model, config


def letterbox(image: Image.Image, size: int) -> Image.Image:
    """EXIF-transpose, convert to RGB and pad to a centred square, as in training."""
    image = ImageOps.exif_transpose(image).convert("RGB")
    return ImageOps.pad(image, (size, size), method=Image.Resampling.BILINEAR,
                        color=(0, 0, 0), centering=(0.5, 0.5))


def to_tensor(image: Image.Image, config: dict) -> torch.Tensor:
    """Letterboxed PIL image -> normalised (1, 3, H, W) tensor."""
    tf = T.Compose([T.ToTensor(), T.Normalize(config["mean"], config["std"])])
    return tf(image).unsqueeze(0)


def preprocess(image: Image.Image, config: dict) -> tuple[Image.Image, torch.Tensor]:
    boxed = letterbox(image, config["image_size"])
    return boxed, to_tensor(boxed, config)


def scores_from_logits(logits: torch.Tensor, config: dict) -> Prediction:
    probs = logits.softmax(1)[0].detach().float().cpu().tolist()
    scores = dict(zip(config["classes"], probs))
    label = config["classes"][int(probs[1] >= config["threshold"])]
    return Prediction(label=label, scores=scores, threshold=config["threshold"])


@torch.inference_mode()
def predict(image: Image.Image, model, config: dict) -> Prediction:
    _, x = preprocess(image, config)
    device = next(model.parameters()).device
    return scores_from_logits(model(x.to(device)), config)


@torch.inference_mode()
def predict_batch(images: list[Image.Image], model, config: dict) -> np.ndarray:
    """Return the PNEUMONIA softmax score for each image (used by evaluation)."""
    device = next(model.parameters()).device
    x = torch.cat([preprocess(im, config)[1] for im in images]).to(device)
    return model(x).softmax(1)[:, 1].float().cpu().numpy()

```

06 xraylens/explainability.py

```python
"""Grad-CAM for the ResNet-50 classifier, plus checks on whether a heatmap is trustworthy.

Implementation notes
--------------------
* Target layer is ``model.layer4[-1]`` (the last bottleneck block, 2048 x 10 x 10
  at 320 px input). That is the deepest spatial feature map, which is standard for
  ResNet Grad-CAM (Selvaraju et al., 2017).
* We back-propagate the *logit difference* z_target - z_other rather than the
  softmax score. This model's softmax is saturated (scores of 0.0000/1.0000 are
  common), and the softmax gradient vanishes at saturation, so a softmax-based
  CAM would be numerically empty exactly on the most confident images.
* ``randomization_check`` implements the model-parameter randomisation test of
  Adebayo et al., "Sanity Checks for Saliency Maps" (NeurIPS 2018): if the map
  barely changes when learned weights are destroyed, it is not explaining the
  model.
"""
from __future__ import annotations

import copy

import numpy as np
import torch
import torch.nn.functional as F
from matplotlib import colormaps
from PIL import Image
from scipy.stats import spearmanr


def gradcam(model, x: torch.Tensor, target: int, layer=None) -> np.ndarray:
    """Return a Grad-CAM map in [0, 1] at the input resolution (H, W)."""
    layer = layer if layer is not None else model.layer4[-1]
    store: dict[str, torch.Tensor] = {}

    def fwd_hook(_m, _i, out):
        store["act"] = out
        out.register_hook(lambda g: store.__setitem__("grad", g))

    handle = layer.register_forward_hook(fwd_hook)
    try:
        device = next(model.parameters()).device
        x = x.to(device).requires_grad_(False)
        with torch.enable_grad():
            model.zero_grad(set_to_none=True)
            logits = model(x)
            other = 1 - target
            (logits[0, target] - logits[0, other]).backward()
    finally:
        handle.remove()

    act, grad = store["act"].detach(), store["grad"].detach()
    weights = grad.mean(dim=(2, 3), keepdim=True)          # global-average-pooled grads
    cam = F.relu((weights * act).sum(dim=1, keepdim=True))  # (1, 1, h, w)
    cam = F.interpolate(cam, size=x.shape[-2:], mode="bilinear", align_corners=False)
    cam = cam[0, 0].float().cpu().numpy()
    peak = cam.max()
    return cam / peak if peak > 1e-12 else np.zeros_like(cam)


def overlay(image: Image.Image, cam: np.ndarray, alpha: float = 0.45,
            cmap: str = "inferno") -> Image.Image:
    """Blend a CAM onto a (letterboxed) image. Weak activations stay transparent."""
    base = np.asarray(image.convert("RGB"), dtype=np.float32) / 255.0
    heat = colormaps[cmap](cam)[..., :3].astype(np.float32)
    a = (alpha * np.clip(cam * 1.4, 0, 1))[..., None]
    out = base * (1 - a) + heat * a
    return Image.fromarray((out * 255).clip(0, 255).astype(np.uint8))

```

```python

def content_mask(image: Image.Image) -> np.ndarray:
    """True where the letterboxed image has real pixels (False on black padding).

    Padding is exact black from ``ImageOps.pad``; a tiny tolerance absorbs bilinear
    edges. Genuine pure-black pixels inside an X-ray are rare and sparse.
    """
    arr = np.asarray(image.convert("L"))
    rows = arr.max(axis=1) > 2
    cols = arr.max(axis=0) > 2
    mask = np.zeros_like(arr, dtype=bool)
    if rows.any() and cols.any():
        r0, r1 = np.argmax(rows), len(rows) - np.argmax(rows[::-1])
        c0, c1 = np.argmax(cols), len(cols) - np.argmax(cols[::-1])
        mask[r0:r1, c0:c1] = True
    return mask


def attribution_stats(cam: np.ndarray, image: Image.Image,
                      lung_mask: np.ndarray | None = None) -> dict[str, float]:
    """Where does the heatmap's mass go?

    * ``padding``  - share on the black letterbox bars (never informative).
    * ``border``   - share in the outer 12% frame of the real image, where text
      markers, rulers, tubes and collimator edges live (classic shortcut zones).
    * ``lungs``    - share inside a radiologist lung mask, when one is supplied.

    Each share comes with a matching ``*_area`` baseline.
    """
    total = float(cam.sum()) or 1.0
    content = content_mask(image)
    # ``*_area`` is the share of pixels in each region: the value a uniform,
    # uninformative heatmap would score. Compare mass against area, never alone.
    stats = {"padding": float(cam[~content].sum()) / total,
             "padding_area": float((~content).mean())}

    ys, xs = np.where(content)
    if len(ys):
        r0, r1, c0, c1 = ys.min(), ys.max() + 1, xs.min(), xs.max() + 1
        mh, mw = int((r1 - r0) * 0.12), int((c1 - c0) * 0.12)
        inner = np.zeros_like(content)
        inner[r0 + mh:r1 - mh, c0 + mw:c1 - mw] = True
        stats["border"] = float(cam[content & ~inner].sum()) / total
        stats["border_area"] = float((content & ~inner).mean())
    else:
        stats["border"] = stats["border_area"] = 0.0
    if lung_mask is not None:
        lung_mask = lung_mask.astype(bool)
        stats["lungs"] = float(cam[lung_mask].sum()) / total
        stats["lungs_area"] = float(lung_mask.mean())
    return stats


def peak_location(cam: np.ndarray) -> tuple[int, int]:
    y, x = np.unravel_index(int(np.argmax(cam)), cam.shape)
    return int(y), int(x)


def randomization_check(model, x: torch.Tensor, target: int, cam: np.ndarray,
                        seed: int = 0) -> dict:
    """Adebayo et al. cascading randomisation, top-down.

    Re-initialises fc, then layer4, then layer3 (cumulatively) and recomputes
    Grad-CAM each time. Returns Spearman rank correlation with the original map
    for each stage. A trustworthy explanation should decorrelate quickly.
    """
    torch.manual_seed(seed)
    probe = copy.deepcopy(model).eval()
    stages = [("fc", [probe.fc]), ("layer4", [probe.layer4]), ("layer3", [probe.layer3])]
    results, maps = [], []
```

```
    for name, modules in stages:
        for module in modules:
            for m in module.modules():
                if hasattr(m, "reset_parameters"):
                    m.reset_parameters()
        rand_cam = gradcam(probe, x, target, layer=probe.layer4[-1])
        rho = spearmanr(cam.ravel()[::7], rand_cam.ravel()[::7]).statistic
        results.append({"randomised": name, "spearman": float(np.nan_to_num(rho))})
        maps.append(rand_cam)
    del probe
    return {"stages": results, "maps": maps}

```

07 xraylens/inputs.py

```python
"""Turn untrusted uploads into a PIL image, or a clear reason why not.

The model will return a confident score for *any* image, a cat photo included, so
validation cannot rely on the model. These checks catch broken or obviously
unsuitable files and attach warnings. They cannot prove an image is a chest
radiograph.
"""
from __future__ import annotations

import io
from dataclasses import dataclass, field

import numpy as np
from PIL import Image, ImageOps, UnidentifiedImageError

MAX_BYTES = 20 * 1024 * 1024
MAX_PIXELS = 50_000_000          # guards against decompression bombs
MIN_SIDE = 64
ACCEPTED = {"png", "jpg", "jpeg", "bmp", "tif", "tiff", "webp", "dcm", "dicom"}


class InputError(ValueError):
    """The file cannot be analysed. The message is safe to show to users."""


@dataclass
class LoadedImage:
    image: Image.Image
    source: str
    warnings: list[str] = field(default_factory=list)
    meta: dict = field(default_factory=dict)


def _from_dicom(data: bytes) -> tuple[Image.Image, dict]:
    import pydicom
    try:
        ds = pydicom.dcmread(io.BytesIO(data), force=True)
        arr = ds.pixel_array.astype(np.float32)
    except Exception as exc:  # pydicom raises many types
        raise InputError("This DICOM file could not be decoded. Compressed transfer "
                         "syntaxes may need extra codecs.") from exc
    if arr.ndim == 3 and arr.shape[-1] not in (3, 4):
        arr = arr[arr.shape[0] // 2]  # multi-frame: take the middle frame
    if getattr(ds, "PhotometricInterpretation", "") == "MONOCHROME1":
        arr = arr.max() - arr
    lo, hi = np.percentile(arr, [0.5, 99.5])
    arr = np.clip((arr - lo) / max(hi - lo, 1e-6), 0, 1)
    # Report only technical tags. Never surface patient-identifying fields.
    meta = {k: str(getattr(ds, k)) for k in ("Modality", "ViewPosition", "BodyPartExamined")
            if getattr(ds, k, None)}
    return Image.fromarray((arr * 255).astype(np.uint8)), meta


def load_upload(data: bytes, filename: str) -> LoadedImage:
    if not data:
        raise InputError("The file is empty.")
    if len(data) > MAX_BYTES:
        raise InputError(f"The file is {len(data) / 1e6:.1f} MB. The limit is {MAX_BYTES // 2**20} MB.")
    ext = filename.rsplit(".", 1)[-1].lower() if "." in filename else ""
    if ext and ext not in ACCEPTED:
        raise InputError(f"'.{ext}' files are not supported. Use PNG, JPEG or DICOM.")

    meta: dict = {}
    is_dicom = ext in {"dcm", "dicom"} or data[128:132] == b"DICM"
    if is_dicom:
        image, meta = _from_dicom(data)
    else:
```

```
        Image.MAX_IMAGE_PIXELS = MAX_PIXELS
        try:
            with Image.open(io.BytesIO(data)) as probe:
                probe.verify()
            image = Image.open(io.BytesIO(data))
            image = ImageOps.exif_transpose(image)
            image.load()
        except Image.DecompressionBombError as exc:
            raise InputError("The image has too many pixels to process safely.") from exc
        except (UnidentifiedImageError, OSError, SyntaxError) as exc:
            raise InputError("This file is not a readable image, or it is truncated.") from exc
        if getattr(image, "n_frames", 1) > 1:
            image.seek(0)
    return LoadedImage(image=image, source=filename, warnings=assess(image), meta=meta)


def looks_inverted(image: Image.Image) -> bool:
    """Two anatomical cues, either of which signals white-is-air polarity.

    1. Centre much darker than the frame (works when the body is surrounded by air).
    2. Midline mediastinum/spine darker than both lung fields (works on tight crops).
    On the 1,462 evaluation radiographs this warns on 5 genuine images (0.3%) and
    catches 99.6% of their inverted copies.
    """
    g = np.asarray(image.convert("L").resize((128, 128)), dtype=np.float32)
    border = np.concatenate([g[:8].ravel(), g[-8:].ravel(), g[:, :8].ravel(), g[:, -8:].ravel()])
    dark_centre = g[40:88, 40:88].mean() < border.mean() - 40
    lungs = np.concatenate([g[32:77, 26:45].ravel(), g[32:77, 83:102].ravel()]).mean()
    dark_midline = g[26:100, 58:70].mean() < lungs - 5
    return bool(dark_centre or dark_midline)


def assess(image: Image.Image) -> list[str]:
    """Heuristic warnings. Raises InputError only for unusable images."""
    w, h = image.size
    if min(w, h) < MIN_SIDE:
        raise InputError(f"The image is {w}×{h} px. It needs to be at least {MIN_SIDE} px on each side.")
    warnings = []
    rgb = np.asarray(image.convert("RGB").resize((128, 128)), dtype=np.float32)
    if rgb.std() < 2:
        raise InputError("The image is blank (a single flat colour).")
    chroma = (rgb.max(axis=2) - rgb.min(axis=2)).mean()
    if chroma > 12:
        warnings.append("This image contains colour. Radiographs are greyscale, so it may not "
                        "be an X-ray. The model will still produce a confident-looking score.")
    ratio = max(w, h) / min(w, h)
    if ratio > 1.8:
        warnings.append(f"Unusual aspect ratio ({ratio:.1f}:1). The image is letterboxed with "
                        "black bars, and the model saw few images shaped like this.")
    if min(w, h) < 256:
        warnings.append("Low resolution. The model works at 320 px, so fine detail is lost.")
    if looks_inverted(image):
        warnings.append("The image looks inverted (bone dark, air bright). The model was trained "
                        "on bone-is-white radiographs.")
    return warnings

```

08 xraylens/perturbations.py

```python
"""Controlled image edits for stress-testing the classifier.

None of these edits change the anatomy a radiologist would read. Large shifts in
the model's score therefore point to sensitivity to acquisition artefacts, not
clinically meaningful findings.
"""
from __future__ import annotations

from typing import Callable

import numpy as np
from PIL import Image, ImageDraw, ImageEnhance, ImageFilter, ImageFont, ImageOps


def _gray(im: Image.Image) -> Image.Image:
    return ImageOps.exif_transpose(im).convert("L")


def hflip(im):
    return ImageOps.mirror(_gray(im))


def invert(im):
    return ImageOps.invert(_gray(im))


def rotate10(im):
    return _gray(im).rotate(10, resample=Image.Resampling.BILINEAR, fillcolor=0)


def low_contrast(im):
    return ImageEnhance.Contrast(_gray(im)).enhance(0.55)


def blur(im):
    g = _gray(im)
    return g.filter(ImageFilter.GaussianBlur(radius=max(g.size) / 300))


def noise(im, sigma: float = 18.0, seed: int = 0):
    arr = np.asarray(_gray(im), dtype=np.float32)
    arr += np.random.default_rng(seed).normal(0, sigma, arr.shape)
    return Image.fromarray(arr.clip(0, 255).astype(np.uint8))


def marker(im):
    """Burn a laterality-style 'L' marker and a tag into the top corner.

    Hospital-specific markers are a well-documented shortcut in chest X-ray models
    (Zech et al., PLOS Medicine 2018).
    """
    g = _gray(im).copy()
    d = ImageDraw.Draw(g)
    size = max(12, g.width // 14)
    try:
        font = ImageFont.load_default(size=size)
    except TypeError:  # Pillow < 10.1
        font = ImageFont.load_default()
    x, y = int(g.width * 0.78), int(g.height * 0.04)
    d.text((x, y), "L", fill=255, font=font)
    d.text((int(g.width * 0.04), y), "PORTABLE", fill=235, font=font)
    return g


def crop_center(im):
    """Tight crop to the central 70%, as in a differently collimated exposure."""
    g = _gray(im)
```

```
    w, h = g.size
    return g.crop((int(w * .15), int(h * .15), int(w * .85), int(h * .85)))


PERTURBATIONS: dict[str, tuple[str, Callable[[Image.Image], Image.Image]]] = {
    "hflip": ("Mirror left-right", hflip),
    "rotate10": ("Rotate 10°", rotate10),
    "crop": ("Tighter collimation (70% crop)", crop_center),
    "contrast": ("Low contrast (×0.55)", low_contrast),
    "blur": ("Mild blur", blur),
    "noise": ("Sensor noise (σ = 18)", noise),
    "marker": ("Burned-in text markers", marker),
    "invert": ("Inverted greyscale", invert),
}

```

09 xraylens/evaluation.py

```python
"""Metrics used by both the offline evaluation script and the live dashboard.

Everything is computed from per-image scores, so the dashboard can recompute any
metric at any threshold without touching the model.
"""
from __future__ import annotations

import math

import numpy as np
from sklearn.metrics import roc_auc_score, roc_curve


def wilson(k: int, n: int, z: float = 1.96) -> tuple[float, float]:
    """Wilson score interval for a binomial proportion (well-behaved near 0 and 1)."""
    if n == 0:
        return (math.nan, math.nan)
    p = k / n
    denom = 1 + z * z / n
    centre = (p + z * z / (2 * n)) / denom
    half = z * math.sqrt(p * (1 - p) / n + z * z / (4 * n * n)) / denom
    return (max(0.0, centre - half), min(1.0, centre + half))


def confusion(y: np.ndarray, score: np.ndarray, threshold: float) -> dict[str, int]:
    pred = score >= threshold
    y = y.astype(bool)
    return {"tp": int((pred & y).sum()), "fn": int((~pred & y).sum()),
            "fp": int((pred & ~y).sum()), "tn": int((~pred & ~y).sum())}


def binary_metrics(y: np.ndarray, score: np.ndarray, threshold: float = 0.5) -> dict:
    """Sensitivity, specificity, PPV, F1, balanced accuracy and AUC with 95% CIs."""
    y, score = np.asarray(y).astype(int), np.asarray(score, dtype=float)
    c = confusion(y, score, threshold)
    tp, fn, fp, tn = c["tp"], c["fn"], c["fp"], c["tn"]
    out = {"n": int(len(y)), "positives": int(y.sum()), "negatives": int((1 - y).sum()),
           "threshold": threshold, "confusion": c}

    def prop(k, n, name):
        out[name] = k / n if n else math.nan
        out[name + "_ci"] = wilson(k, n)

    prop(tp, tp + fn, "sensitivity")
    prop(tn, tn + fp, "specificity")
    prop(tp, tp + fp, "precision")
    prop(tp + tn, len(y), "accuracy")
    s, p = out["sensitivity"], out["precision"]
    out["f1"] = 2 * p * s / (p + s) if p + s and not math.isnan(p) else math.nan
    out["balanced_accuracy"] = (out["sensitivity"] + out["specificity"]) / 2
    if 0 < y.sum() < len(y):
        out["auc"] = float(roc_auc_score(y, score))
        out["auc_ci"] = bootstrap_auc(y, score)
    else:
        out["auc"], out["auc_ci"] = math.nan, (math.nan, math.nan)
    return out


def bootstrap_auc(y: np.ndarray, score: np.ndarray, reps: int = 1000, seed: int = 0):
    """Percentile bootstrap, stratified so every resample has both classes."""
    rng = np.random.default_rng(seed)
    pos, neg = np.where(y == 1)[0], np.where(y == 0)[0]
    aucs = []
    for _ in range(reps):
        idx = np.concatenate([rng.choice(pos, len(pos)), rng.choice(neg, len(neg))])
        aucs.append(roc_auc_score(y[idx], score[idx]))
    return (float(np.percentile(aucs, 2.5)), float(np.percentile(aucs, 97.5)))
```

```
def flag_rate(score: np.ndarray, threshold: float = 0.5) -> dict:
    """Share of images called PNEUMONIA, for cohorts where none should be."""
    score = np.asarray(score)
    k, n = int((score >= threshold).sum()), len(score)
    return {"n": n, "flagged": k, "rate": k / n if n else math.nan, "rate_ci": wilson(k, n)}


def roc_points(y, score):
    fpr, tpr, thr = roc_curve(y, score)
    return {"fpr": fpr.tolist(), "tpr": tpr.tolist(), "thresholds": np.clip(thr, 0, 1).tolist()}


def calibration(y, score, bins: int = 10) -> dict:
    """Reliability-diagram bins and expected calibration error (ECE)."""
    y, score = np.asarray(y), np.asarray(score)
    edges = np.linspace(0, 1, bins + 1)
    idx = np.clip(np.digitize(score, edges[1:-1]), 0, bins - 1)
    rows, ece = [], 0.0
    for b in range(bins):
        m = idx == b
        if not m.any():
            continue
        conf, acc = float(score[m].mean()), float(y[m].mean())
        rows.append({"bin": b, "lo": float(edges[b]), "hi": float(edges[b + 1]),
                     "n": int(m.sum()), "mean_score": conf, "observed_rate": acc})
        ece += m.mean() * abs(conf - acc)
    return {"bins": rows, "ece": float(ece)}


def saturation_share(score, eps: float = 1e-3) -> float:
    """Fraction of outputs pinned within eps of 0 or 1."""
    score = np.asarray(score)
    return float(((score < eps) | (score > 1 - eps)).mean())

```

10 xraylens/app_state.py

```python
"""Cached loaders shared by the Streamlit pages."""
from __future__ import annotations

import hashlib
import io
import json
from pathlib import Path

import numpy as np
import pandas as pd
import streamlit as st
from PIL import Image, ImageFilter

from .explainability import attribution_stats, gradcam, overlay, peak_location
from .model import load_model, preprocess, scores_from_logits
from .perturbations import PERTURBATIONS

ROOT = Path(__file__).resolve().parent.parent
GALLERY = ROOT / "assets" / "gallery"
RESULTS = ROOT / "results"


@st.cache_resource(show_spinner="Loading ResNet-50 weights…")
def get_model():
    return load_model()


@st.cache_data
def get_gallery() -> list[dict]:
    path = GALLERY / "gallery.json"
    return json.loads(path.read_text()) if path.exists() else []


@st.cache_data
def get_metrics() -> dict:
    return json.loads((RESULTS / "metrics.json").read_text())


@st.cache_data
def get_scores() -> pd.DataFrame:
    return pd.read_csv(RESULTS / "scores.csv", keep_default_na=False, na_values=[""])


def image_key(image: Image.Image) -> str:
    buf = io.BytesIO()
    image.save(buf, format="PNG")
    return hashlib.sha1(buf.getvalue()).hexdigest()


@st.cache_data(show_spinner=False, max_entries=32)
def analyse(_image: Image.Image, key: str, lung_mask_path: str | None = None) -> dict:
    """Prediction plus Grad-CAM for both classes. ``key`` stands in for the image in the cache."""
    model, cfg = get_model()
    boxed, x = preprocess(_image, cfg)
    device = next(model.parameters()).device
    import torch
    with torch.inference_mode():
        logits = model(x.to(device))
    pred = scores_from_logits(logits, cfg)
    lung = None
    if lung_mask_path:
        mask_boxed, _ = preprocess(Image.open(ROOT / lung_mask_path), cfg)
        lung = np.asarray(mask_boxed.convert("L")) > 127
    cams, stats = {}, {}
    for idx, name in enumerate(cfg["classes"]):
        cams[name] = gradcam(model, x, idx)
        stats[name] = attribution_stats(cams[name], boxed, lung)
```

```
    lg = logits[0].float().cpu().numpy()
    return {"boxed": boxed, "x": x, "pred": pred, "cams": cams, "stats": stats, "lung": lung,
            "logit_margin": float(lg[1] - lg[0]), "peak": {k: peak_location(v) for k, v in cams.items()}}


def lung_outline(mask: np.ndarray) -> np.ndarray:
    m = Image.fromarray((mask * 255).astype(np.uint8))
    inner = np.asarray(m.filter(ImageFilter.MinFilter(5))) > 127
    return mask & ~inner


def render_overlay(boxed: Image.Image, cam: np.ndarray, alpha: float, lung: np.ndarray | None) -> Image.Image:
    out = overlay(boxed, cam, alpha=alpha)
    if lung is not None:
        arr = np.asarray(out).copy()
        arr[lung_outline(lung)] = (56, 165, 150)
        out = Image.fromarray(arr)
    return out


@st.cache_data(show_spinner=False, max_entries=16)
def stress(_image: Image.Image, key: str) -> pd.DataFrame:
    model, cfg = get_model()
    from .model import predict_batch
    names = ["original"] + list(PERTURBATIONS)
    ims = [_image] + [fn(_image) for _, fn in PERTURBATIONS.values()]
    scores = predict_batch(ims, model, cfg)
    labels = ["Original"] + [lab for lab, _ in PERTURBATIONS.values()]
    return pd.DataFrame({"key": names, "edit": labels, "score": scores, "image": ims})

```

xraylens/ui.py

```
"""Visual system for the Streamlit app: theme CSS and small HTML components.

Theme: a radiology lightbox. Film-black ground, bone-white type, a faint
graticule, amber for attention. Instrument Serif for display, Newsreader for
reading and JetBrains Mono for every number the model produces.
"""
from __future__ import annotations

import html

import streamlit as st

# Chart palette: validated with the dataviz CVD checker on #10161a (dark).
NORMAL, PNEUMONIA, OTHER = "#38a596", "#e05c35", "#8b7ff0"
INK, BONE, MUTED, AMBER, RULE = "#0b0f12", "#ece6d6", "#93a1a6", "#ffb547", "rgba(236,230,214,.16)"

CSS = """
@import url('https://fonts.googleapis.com/css2?
family=Instrument+Serif:ital@0;1&family=Newsreader:ital,opsz,wght@0,6..72,300..700;1,6..72,300..700&family=JetBrains+Mo
no:wght@400;600&display=swap');

:root {
  --ink: #0b0f12;
  --film: #10161a;
  --film-2: #141c21;
  --bone: #ece6d6;
  --bone-dim: #c9c2b2;
  --muted: #93a1a6;
  --amber: #ffb547;
  --normal: #38a596;
  --pneumonia: #e05c35;
  --other: #8b7ff0;
  --rule: rgba(236,230,214,.16);
  --rule-strong: rgba(236,230,214,.38);
  --display: 'Instrument Serif', 'Newsreader', Georgia, serif;
  --serif: 'Newsreader', Georgia, serif;
  --mono: 'JetBrains Mono', ui-monospace, monospace;
}

html, body, .stApp { background: var(--ink); color: var(--bone); }

/* Layered ground: lightbox glow, warm under-light, then a 48px graticule. */
.stApp {
  background:
    radial-gradient(ellipse 80% 55% at 78% -8%, rgba(127,209,199,.11), transparent 60%),
    radial-gradient(ellipse 60% 50% at -5% 105%, rgba(255,181,71,.07), transparent 60%),
    repeating-linear-gradient(0deg, rgba(236,230,214,.028) 0 1px, transparent 1px 48px),
    repeating-linear-gradient(90deg, rgba(236,230,214,.028) 0 1px, transparent 1px 48px),
    var(--ink) !important;
  background-attachment: fixed !important;
}
[data-testid="stHeader"] { background: transparent; }
[data-testid="stMainBlockContainer"], .block-container {
  max-width: 1240px; padding-top: 2.2rem; padding-left: clamp(16px, 4vw, 56px); padding-right: clamp(16px, 4vw,
56px);
}

.stApp, .stMarkdown, p, li, label, .stRadio, .stSelectbox, input, textarea, button {
  font-family: var(--serif) !important;
}
.stMarkdown p, .stMarkdown li { font-size: 1.08rem; line-height: 1.6; color: var(--bone-dim); }
h1, h2, h3, h4 { font-family: var(--display) !important; font-weight: 400 !important; color: var(--bone)
!important; letter-spacing: -.01em; }
code, .mono, [data-testid="stMetricValue"], [data-testid="stDataFrame"] * { font-family: var(--mono) !important;
}
code { color: var(--amber) !important; background: rgba(255,181,71,.08) !important; }
a { color: var(--amber) !important; text-decoration-thickness: 1px; text-underline-offset: 3px; }
```

```
  63
  64  /* Sidebar: a dark film strip edged with sprocket holes. */
  65  [data-testid="stSidebar"] {
  66    background:
  67      radial-gradient(circle at 10px 14px, var(--ink) 3.5px, transparent 4px) 0 0 / 100% 28px repeat-y,
  68      linear-gradient(180deg, #0e1417, #0b0f12) !important;
  69    border-right: 1px solid var(--rule);
  70  }
  71  [data-testid="stSidebarNav"] a span { font-family: var(--mono) !important; font-size: .82rem; letter-spacing:
.06em; text-transform: uppercase; }
  72
  73  /* Rotated running head on the right edge. Purely decorative. */
  74  .xl-runhead {
  75    position: fixed; right: -8px; top: 50%; transform: rotate(90deg) translateX(50%); transform-origin: right top;
  76    font: 600 10px/1 var(--mono); letter-spacing: .32em; color: rgba(236,230,214,.28); z-index: 0; pointer-events:
none; white-space: nowrap;
  77  }
  78  @media (max-width: 900px) { .xl-runhead { display: none; } }
  79
  80  /* Masthead */
  81  .xl-mast { position: relative; margin: 0 0 2.4rem; padding-bottom: 1.4rem; border-bottom: 1px solid var(--rule-
strong); }
  82  .xl-kicker { font: 600 .72rem/1 var(--mono); letter-spacing: .26em; text-transform: uppercase; color: var(--
amber); }
  83  .xl-title { font-family: var(--display); font-size: clamp(3.4rem, 9vw, 7.6rem); line-height: .86; margin: .5rem 0
0; color: var(--bone); letter-spacing: -.03em; }
  84  .xl-title em { color: var(--amber); font-style: italic; }
  85  .xl-dek { font-family: var(--serif); font-size: clamp(1.1rem, 1.6vw, 1.35rem); line-height: 1.45; color: var(--
bone-dim); max-width: 40ch; margin: 1.9rem 0 0 clamp(0px, 14vw, 200px); font-style: italic; }
  86  .xl-stamp {
  87    position: absolute; right: 0; top: .2rem; transform: rotate(-4deg);
  88    border: 1.5px solid var(--pneumonia); color: var(--pneumonia); padding: .45rem .7rem;
  89    font: 600 .7rem/1.25 var(--mono); letter-spacing: .14em; text-transform: uppercase; text-align: center;
  90  }
  91  @media (max-width: 700px) { .xl-stamp { position: static; display: inline-block; margin-top: 1rem; transform:
rotate(-2deg); } .xl-dek { margin-left: 0; } }
  92
  93  /* Section heads: an oversized numeral hanging into the left gutter. */
  94  .xl-sec { display: grid; grid-template-columns: minmax(64px, 120px) 1fr; align-items: end; gap: 0 18px; margin:
3.2rem 0 1.1rem; }
  95  .xl-sec .num { font-family: var(--display); font-style: italic; font-size: clamp(3.6rem, 7vw, 6rem); line-height:
.78; color: transparent; -webkit-text-stroke: 1px var(--amber); }
  96  .xl-sec h2 { margin: 0 !important; padding: 0 !important; font-size: clamp(1.9rem, 3.2vw, 2.6rem) !important;
line-height: 1.05; border-bottom: 1px solid var(--rule); padding-bottom: .5rem !important; }
  97  .xl-sec .sub { grid-column: 2; color: var(--muted); font-size: 1.02rem; margin-top: .5rem; font-family: var(--
serif); max-width: 62ch; }
  98
  99  /* Score readout */
 100  .xl-readout { border-top: 3px solid var(--bone); padding-top: .9rem; }
 101  .xl-readout .lab { font: 600 .7rem/1 var(--mono); letter-spacing: .22em; color: var(--muted); text-transform:
uppercase; }
 102  .xl-readout .call { font-family: var(--display); font-size: clamp(2.8rem, 6vw, 4.6rem); line-height: .95; margin:
.35rem 0 .15rem; }
 103  .xl-readout .call.normal { color: var(--normal); }
 104  .xl-readout .call.pneumonia { color: var(--pneumonia); }
 105  .xl-track { position: relative; height: 46px; margin: 1.3rem 0 .4rem; }
 106  .xl-track .bar { position: absolute; left: 0; right: 0; top: 18px; height: 10px;
 107    background: linear-gradient(90deg, var(--normal), #3c4a50 50%, var(--pneumonia)); border-radius: 5px; opacity:
.85; }
 108  .xl-track .thr { position: absolute; top: 6px; width: 0; height: 34px; border-left: 1.5px dashed var(--bone); }
 109  .xl-track .thr span { position: absolute; top: -14px; left: 4px; font: .64rem var(--mono); color: var(--bone);
white-space: nowrap; }
 110  .xl-track .pin { position: absolute; top: 10px; width: 16px; height: 26px; margin-left: -8px; background: var(--
bone); border: 2px solid var(--ink); border-radius: 3px; box-shadow: 0 0 0 1px var(--bone); }
 111  .xl-axis { display: flex; justify-content: space-between; font: .68rem var(--mono); color: var(--muted); }
 112  .xl-kv { display: grid; grid-template-columns: auto 1fr; gap: .3rem 1.2rem; margin-top: 1rem; font: .84rem/1.5
var(--mono); }
 113  .xl-kv dt { color: var(--muted); } .xl-kv dd { margin: 0; color: var(--bone); text-align: right; }
 114
 115  /* Notes and warnings */
```

```
 116  .xl-note { border-left: 3px solid var(--amber); padding: .7rem 0 .7rem 1rem; margin: 1rem 0; background: linear-gradient(90deg, rgba(255,181,71,.07), transparent 70%); }
 117  .xl-note.warn { border-color: var(--pneumonia); background: linear-gradient(90deg, rgba(224,92,53,.09), transparent 70%); }
 118  .xl-note.cool { border-color: var(--normal); background: linear-gradient(90deg, rgba(56,165,150,.08), transparent 70%); }
 119  .xl-note b { color: var(--bone); font-weight: 600; }
 120  .xl-note p { margin: 0; color: var(--bone-dim); font-size: 1rem; line-height: 1.55; }
 121  .xl-tag { display: inline-block; font: 600 .66rem/1 var(--mono); letter-spacing: .14em; text-transform: uppercase; padding: .32rem .5rem; border: 1px solid currentColor; margin: 0 .35rem .35rem 0; }
 122
 123  /* Big comparison numbers */
 124  .xl-hero { display: grid; grid-template-columns: 1.15fr .85fr; gap: 0; border-top: 1px solid var(--rule-strong); border-bottom: 1px solid var(--rule-strong); }
 125  .xl-hero > div { padding: 1.4rem 1.4rem 1.2rem 0; }
 126  .xl-hero > div + div { padding-left: 1.6rem; border-left: 1px solid var(--rule-strong); }
 127  .xl-hero .n { font-family: var(--display); font-size: clamp(3.4rem, 8vw, 6.4rem); line-height: .9; }
 128  .xl-hero .n small { font-size: .38em; color: var(--muted); }
 129  .xl-hero .cap { font: .74rem/1.45 var(--mono); color: var(--muted); letter-spacing: .04em; margin-top: .5rem; max-width: 42ch; }
 130  .xl-strike { text-decoration: line-through; text-decoration-color: var(--pneumonia); text-decoration-thickness: 3px; color: var(--bone-dim); }
 131  @media (max-width: 700px) { .xl-hero { grid-template-columns: 1fr; } .xl-hero > div + div { border-left: 0; border-top: 1px solid var(--rule-strong); padding-left: 0; } }
 132
 133  /* Stat rows (inline numbers with CIs) */
 134  .xl-stats { display: grid; grid-template-columns: repeat(auto-fit, minmax(150px, 1fr)); border-top: 1px solid var(--rule); }
 135  .xl-stats > div { padding: .8rem .9rem .8rem 0; border-bottom: 1px solid var(--rule); }
 136  .xl-stats .k { font: 600 .66rem/1 var(--mono); letter-spacing: .18em; text-transform: uppercase; color: var(--muted); }
 137  .xl-stats .v { font: 600 1.7rem/1.1 var(--mono); color: var(--bone); margin-top: .35rem; }
 138  .xl-stats .ci { font: .7rem var(--mono); color: var(--muted); }
 139
 140  /* Figures */
 141  .xl-fig { font: .72rem/1.4 var(--mono); color: var(--muted); letter-spacing: .03em; margin-top: .35rem; }
 142  .xl-fig b { color: var(--amber); font-weight: 600; }
 143  [data-testid="stImage"] img { border: 1px solid var(--rule); }
 144  [data-testid="stImageCaption"], [data-testid="caption"] { font-family: var(--mono) !important; font-size: .72rem !important; color: var(--muted) !important; }
 145
 146  /* Widgets */
 147  .stButton > button, [data-testid="stBaseButton-secondary"], [data-testid="stBaseButton-primary"] {
 148    border-radius: 0 !important; font-family: var(--mono) !important; font-size: .74rem !important;
 149    letter-spacing: .12em; text-transform: uppercase; border: 1px solid var(--rule-strong) !important; background: transparent !important; color: var(--bone) !important;
 150  }
 151  .stButton > button:hover { border-color: var(--amber) !important; color: var(--amber) !important; }
 152  .stButton > button[data-testid="stBaseButton-primary"], .stButton > button[kind="primary"] { background: var(--amber) !important; color: var(--ink) !important; border-color: var(--amber) !important; }
 153  [data-testid="stFileUploaderDropzone"] { background: repeating-linear-gradient(-45deg, rgba(236,230,214,.03) 0 8px, transparent 8px 16px), var(--film) !important; border: 1px dashed var(--rule-strong) !important; border-radius: 0 !important; }
 154  [data-testid="stExpander"] details { border-radius: 0 !important; border: 1px solid var(--rule) !important; background: rgba(16,22,26,.6); }
 155  [data-testid="stExpander"] summary p { font-family: var(--mono) !important; font-size: .8rem !important; letter-spacing: .08em; text-transform: uppercase; }
 156  [data-baseweb="tab-list"] { gap: 1.6rem; border-bottom: 1px solid var(--rule); }
 157  [data-baseweb="tab"] p { font-family: var(--mono) !important; font-size: .78rem !important; letter-spacing: .1em; text-transform: uppercase; }
 158  [data-testid="stSlider"] label p, [data-testid="stWidgetLabel"] p { font-family: var(--mono) !important; font-size: .72rem !important; letter-spacing: .14em; text-transform: uppercase; color: var(--muted) !important; }
 159  hr { border-color: var(--rule) !important; }
 160
 161  /* Contact sheet captions */
 162  .xl-sheet-cap { font: .7rem/1.35 var(--mono); color: var(--muted); margin: .25rem 0 .4rem; min-height: 2.8em; }
 163  .xl-sheet-cap b { color: var(--bone); font-weight: 600; display: block; letter-spacing: .04em; }
 164
 165  .xl-foot { margin-top: 4rem; padding-top: 1rem; border-top: 1px solid var(--rule); font: .72rem/1.6 var(--mono); color: var(--muted); }
```

```
"""


def inject_theme() -> None:
    st.markdown(f"<style>{CSS}</style>", unsafe_allow_html=True)
    st.markdown('<div class="xl-runhead">XRAYLENS · RESEARCH PROTOTYPE · NOT A DIAGNOSTIC DEVICE</div>',
                unsafe_allow_html=True)


def html_block(markup: str) -> None:
    st.markdown(markup, unsafe_allow_html=True)


def esc(text) -> str:
    return html.escape(str(text))


def section(num: str, title: str, sub: str = "") -> None:
    sub_html = f'<div class="sub">{sub}</div>' if sub else ""
    html_block(f'<div class="xl-sec"><div class="num">{num}</div><h2>{title}</h2>{sub_html}</div>')


def note(body: str, kind: str = "") -> None:
    html_block(f'<div class="xl-note {kind}"><p>{body}</p></div>')


def pct(x: float, digits: int = 1) -> str:
    return "—" if x ≠ x else f"{x * 100:.{digits}f}%"


def ci(lo_hi, digits: int = 1) -> str:
    lo, hi = lo_hi
    return "" if lo ≠ lo else f"95% CI {lo * 100:.{digits}f}–{hi * 100:.{digits}f}%"


def stats_row(items: list[tuple[str, str, str]]) -> None:
    cells = "".join(f'<div><div class="k">{k}</div><div class="v">{v}</div><div class="ci">{c}</div></div>'
                    for k, v, c in items)
    html_block(f'<div class="xl-stats">{cells}</div>')


def score_readout(label: str, score: float, threshold: float, extra: dict[str, str]) -> None:
    pos = min(max(score, 0.0), 1.0) * 100
    kv = "".join(f"<dt>{esc(k)}</dt><dd>{esc(v)}</dd>" for k, v in extra.items())
    html_block(f"""
<div class="xl-readout">
  <div class="lab">Model output · class with score ≥ threshold</div>
  <div class="call {label.lower()}">{label.title()}</div>
  <div class="xl-track" role="img" aria-label="Pneumonia score {score:.4f} against threshold {threshold}">
    <div class="bar"></div>
    <div class="thr" style="left:{threshold * 100}%"><span>threshold {threshold}</span></div>
    <div class="pin" style="left:{pos}%"></div>
  </div>
  <div class="xl-axis"><span>0 · NORMAL</span><span>PNEUMONIA · 1</span></div>
  <dl class="xl-kv">{kv}</dl>
</div>""")


def altair_theme():
    """Register a matching Altair theme (transparent, mono axes, recessive grid)."""
    import altair as alt

    @alt.theme.register("xraylens", enable=True)
    def _theme():
        return alt.theme.ThemeConfig({
            "config": {
                "background": "transparent",
                "font": "JetBrains Mono",
                "view": {"stroke": None},
                "axis": {"labelColor": MUTED, "titleColor": MUTED, "gridColor": "rgba(236,230,214,.08)",
```

```
                          "domainColor": "rgba(236,230,214,.3)", "tickColor": "rgba(236,230,214,.3)",
                          "labelFont": "JetBrains Mono", "titleFont": "JetBrains Mono",
                          "titleFontWeight": 400, "labelFontSize": 10, "titleFontSize": 10},
                  "legend": {"labelColor": BONE, "titleColor": MUTED, "labelFont": "JetBrains Mono",
                             "titleFont": "JetBrains Mono", "titleFontWeight": 400, "orient": "top"},
                  "header": {"labelColor": BONE, "titleColor": MUTED, "labelFont": "JetBrains Mono",
                             "titleFont": "JetBrains Mono", "labelFontSize": 11},
                  "title": {"color": BONE, "font": "Newsreader", "fontWeight": 400, "fontSize": 15, "anchor":
"start"},
                  "text": {"color": BONE, "font": "JetBrains Mono"},
              }
          })

```

12 scripts/fetch_eval_data.py

```python
"""Download and downsample the evaluation cohorts into data/eval/ (git-ignored).

Cohorts
-------
kermany_test   624 pediatric images, the original Kermany "test" folder.
               WARNING: the final model was refit on ALL Kermany images, test folder
               included. Scores here measure memorisation, not generalisation. We
               keep it only to show how large that gap is.
rsna           Adult frontal radiographs (RSNA Pneumonia Detection Challenge 2018,
               derived from NIH ChestX-ray14). Fixed-seed sample of N per class:
               "Normal", "Lung Opacity" and "No Lung Opacity / Not Normal".
montgomery     138 adult radiographs (NLM, Montgomery County TB programme) with
               radiologist-drawn lung masks. Normal vs TB, used for (a) adult
               specificity and (b) measuring how much Grad-CAM mass lands in lungs.
shenzhen       Fixed-seed sample of adult radiographs (NLM / Shenzhen No.3 Hospital),
               normal vs TB. Out-of-scope disease probe.

Images are resized so the long side is 512 px. That is above the model's 320 px
input, so the letterboxed tensor is unchanged in practice, and it keeps disk use
small. Nothing here is committed: only manifests of IDs and per-image scores.
"""
from __future__ import annotations

import argparse
import io
import random
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path

import numpy as np
import pandas as pd
from huggingface_hub import HfApi, hf_hub_download
from PIL import Image

ROOT = Path(__file__).resolve().parent.parent
RAW, OUT = ROOT / "data" / "raw", ROOT / "data" / "eval"
LONG_SIDE = 512

RSNA = ("Baldezo313/rsna-pneumonia-dataset", "a7f1dad69625cb4d02b9ef8ee849d5c0e6fc24b4")
KERMANY = ("hf-vision/chest-xray-pneumonia", "c1a67c18df1a52ec332ffc00418f4eaa61a9f2bc")
NLM = "Famatsu123/montgomery-shenzhen-tuberculosis-cxr"


def shrink(im: Image.Image) -> Image.Image:
    im = im.convert("L")
    s = LONG_SIDE / max(im.size)
    return im.resize((round(im.width * s), round(im.height * s)), Image.Resampling.LANCZOS) if s < 1 else im


def dicom_to_image(path: str) -> Image.Image:
    import pydicom
    ds = pydicom.dcmread(path)
    arr = ds.pixel_array.astype(np.float32)
    if getattr(ds, "PhotometricInterpretation", "") == "MONOCHROME1":
        arr = arr.max() - arr
    lo, hi = np.percentile(arr, [0.5, 99.5])
    arr = np.clip((arr - lo) / max(hi - lo, 1e-6), 0, 1)
    return Image.fromarray((arr * 255).astype(np.uint8))


def fetch_kermany() -> pd.DataFrame:
    p = hf_hub_download(KERMANY[0], "data/test-00000-of-00001.parquet", repo_type="dataset",
                        revision=KERMANY[1], local_dir=RAW / "kermany")
    df = pd.read_parquet(p)
    out = OUT / "kermany_test"
    out.mkdir(parents=True, exist_ok=True)
    rows = []
```

```
    for i, r in df.iterrows():
        name = f"kermany_{i:04d}.png"
        shrink(Image.open(io.BytesIO(r.image["bytes"]))).save(out / name)
        rows.append({"cohort": "kermany_test", "file": f"kermany_test/{name}",
                     "source_id": f"test[{i}]", "label": int(r.label),
                     "finding": ["Normal", "Pneumonia"][int(r.label)]})
    return pd.DataFrame(rows)


def fetch_rsna(per_class: int, seed: int) -> pd.DataFrame:
    api = HfApi()
    info = pd.read_csv(hf_hub_download(RSNA[0], "stage_2_detailed_class_info.csv", repo_type="dataset",
                                       revision=RSNA[1], local_dir=RAW / "rsna"))
    info = info.drop_duplicates("patientId")  # one row per bounding box upstream
    where = {}
    for d in ("stage_2_train_images_0", "stage_2_train_images_1", "stage_2_train_images_2"):
        for f in api.list_repo_tree(RSNA[0], path_in_repo=d, repo_type="dataset", revision=RSNA[1]):
            where[Path(f.path).stem] = f.path
    info = info[info.patientId.isin(where)]
    picks = pd.concat([g.sample(min(per_class, len(g)), random_state=seed)
                       for _, g in info.groupby("class", sort=True)])
    out = OUT / "rsna"
    out.mkdir(parents=True, exist_ok=True)

    def one(row):
        name = f"{row.patientId}.png"
        if not (out / name).exists():
            dcm = hf_hub_download(RSNA[0], where[row.patientId], repo_type="dataset",
                                  revision=RSNA[1], local_dir=RAW / "rsna")
            shrink(dicom_to_image(dcm)).save(out / name)
            Path(dcm).unlink()  # keep disk use bounded
        finding = row["class"]
        # Binary task: Lung Opacity → positive, Normal → negative,
        # "Not Normal" (other abnormality) → negative for pneumonia but tracked separately.
        return {"cohort": "rsna", "file": f"rsna/{name}", "source_id": row.patientId,
                "label": int(finding == "Lung Opacity"), "finding": finding}

    with ThreadPoolExecutor(8) as pool:
        return pd.DataFrame(pool.map(one, [r for _, r in picks.iterrows()]))


def fetch_nlm(shenzhen_n: int, seed: int) -> pd.DataFrame:
    api = HfApi()
    sha = api.dataset_info(NLM).sha
    mont = sorted(f.path for f in api.list_repo_tree(NLM, path_in_repo="MontgomerySet/CXR_png",
                                                     repo_type="dataset", revision=sha)
                  if f.path.endswith(".png"))
    shen = sorted(f.path for f in api.list_repo_tree(NLM, path_in_repo="ChinaSet_AllFiles/CXR_png",
                                                     repo_type="dataset", revision=sha)
                  if f.path.endswith(".png"))
    rng = random.Random(seed)
    shen_pos = [p for p in shen if p.endswith("_1.png")]
    shen_neg = [p for p in shen if p.endswith("_0.png")]
    shen = rng.sample(shen_neg, shenzhen_n // 2) + rng.sample(shen_pos, shenzhen_n // 2)
    (OUT / "montgomery").mkdir(parents=True, exist_ok=True)
    (OUT / "shenzhen").mkdir(parents=True, exist_ok=True)

    def get(path):
        local = hf_hub_download(NLM, path, repo_type="dataset", revision=sha, local_dir=RAW / "nlm")
        im = Image.open(local)
        im.load()
        Path(local).unlink()
        return im

    def one(path):
        cohort = "montgomery" if path.startswith("Montgomery") else "shenzhen"
        stem = Path(path).stem
        tb = int(stem.endswith("_1"))
        im = get(path)
        small = shrink(im)
```

```
        small.save(OUT / cohort / f"{stem}.png")
        row = {"cohort": cohort, "file": f"{cohort}/{stem}.png", "source_id": stem,
               "label": 0,  # neither TB nor normal is "pneumonia" in this binary task
               "finding": "Tuberculosis" if tb else "Normal", "mask": ""}
        if cohort == "montgomery":
            masks = [np.asarray(get(f"MontgomerySet/ManualMask/{side}/{stem}.png").convert("L")
                                .resize(small.size, Image.Resampling.NEAREST)) > 127
                     for side in ("leftMask", "rightMask")]
            name = f"{stem}_lungs.png"
            Image.fromarray(((masks[0] | masks[1]) * 255).astype(np.uint8)).save(OUT / cohort / name)
            row["mask"] = f"{cohort}/{name}"
        return row

    with ThreadPoolExecutor(6) as pool:
        return pd.DataFrame(pool.map(one, mont + shen)), sha


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--rsna-per-class", type=int, default=200)
    ap.add_argument("--shenzhen", type=int, default=100)
    ap.add_argument("--seed", type=int, default=20261020)
    a = ap.parse_args()
    OUT.mkdir(parents=True, exist_ok=True)
    parts = [fetch_kermany(), fetch_rsna(a.rsna_per_class, a.seed)]
    nlm, sha = fetch_nlm(a.shenzhen, a.seed)
    parts.append(nlm)
    manifest = pd.concat(parts, ignore_index=True)
    manifest["mask"] = manifest.get("mask", "").fillna("")
    manifest.to_csv(OUT / "manifest.csv", index=False)
    print(manifest.groupby(["cohort", "finding"]).size().to_string())
    print("NLM revision", sha)


if __name__ == "__main__":
    main()

```

13 scripts/run_evaluation.py

```python
"""Score every evaluation image and write results/ (committed).

Outputs
-------
results/scores.csv     one row per image: cohort, source id, label, finding, model
                       score, Grad-CAM attribution shares, perturbed scores.
results/metrics.json   headline metrics per cohort, calibration, robustness.

Run ``python scripts/fetch_eval_data.py`` first. Deterministic for a given device.
"""
from __future__ import annotations

import json
import platform
import sys
import time
from pathlib import Path

import numpy as np
import pandas as pd
import torch
from PIL import Image

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))

from xraylens import evaluation as ev  # noqa: E402
from xraylens.explainability import attribution_stats, gradcam  # noqa: E402
from xraylens.model import REPO_ID, REVISION, load_model, predict_batch, preprocess  # noqa: E402
from xraylens.perturbations import PERTURBATIONS  # noqa: E402

EVAL, OUT = ROOT / "data" / "eval", ROOT / "results"
BATCH = 16


def score_files(paths, model, cfg, transform=None):
    scores = []
    for i in range(0, len(paths), BATCH):
        ims = [Image.open(EVAL / p) for p in paths[i:i + BATCH]]
        if transform:
            ims = [transform(im) for im in ims]
        scores.append(predict_batch(ims, model, cfg))
    return np.concatenate(scores)


def main():
    torch.manual_seed(0)
    t0 = time.time()
    model, cfg = load_model()
    df = pd.read_csv(EVAL / "manifest.csv", keep_default_na=False)
    print(f"{len(df)} images on {next(model.parameters()).device}")

    df["score"] = score_files(df.file.tolist(), model, cfg)

    # Grad-CAM attribution for the predicted class on every external image.
    cams = []
    for r in df.itertuples():
        im = Image.open(EVAL / r.file)
        boxed, x = preprocess(im, cfg)
        target = int(r.score ⩾ cfg["threshold"])
        cam = gradcam(model, x, target)
        lung = None
        if r.mask:
            mask_boxed, _ = preprocess(Image.open(EVAL / r.mask), cfg)
            lung = np.asarray(mask_boxed.convert("L")) > 127
        cams.append(attribution_stats(cam, boxed, lung))
    cam_df = pd.DataFrame(cams).add_prefix("cam_")
```

```
    df = pd.concat([df, cam_df], axis=1)
    print(f"grad-cam done {time.time() - t0:.0f}s")

    # Robustness: rescore external adult cohorts under each perturbation.
    ext = df.cohort.isin(["rsna", "montgomery", "shenzhen"])
    for key, (_, fn) in PERTURBATIONS.items():
        df.loc[ext, f"pert_{key}"] = score_files(df.loc[ext, "file"].tolist(), model, cfg, fn)
    print(f"perturbations done {time.time() - t0:.0f}s")

    thr = cfg["threshold"]
    k = df[df.cohort == "kermany_test"]
    rsna = df[df.cohort == "rsna"]
    rsna_main = rsna[rsna.finding.isin(["Normal", "Lung Opacity"])]
    metrics = {
        "model": {"repo": REPO_ID, "revision": REVISION, "threshold": thr},
        "environment": {"python": platform.python_version(), "torch": torch.__version__,
                        "device": str(next(model.parameters()).device)},
        "cohorts": {
            "kermany_test": {**ev.binary_metrics(k.label.values, k.score.values, thr),
                             "valid_test": False,
                             "note": "Model was refit on these images. Memorisation, not generalisation."},
            "rsna_normal_vs_opacity": {**ev.binary_metrics(rsna_main.label.values, rsna_main.score.values, thr),
                                       "valid_test": True},
        },
        "flag_rates": {},
        "calibration": {"rsna_normal_vs_opacity": ev.calibration(rsna_main.label.values,
rsna_main.score.values)},
        "saturation": {c: ev.saturation_share(g.score.values) for c, g in df.groupby("cohort")},
        "attribution": {},
        "robustness": {},
    }
    for (cohort, finding), g in df.groupby(["cohort", "finding"]):
        if finding == "Pneumonia" or finding == "Lung Opacity":
            continue
        metrics["flag_rates"][f"{cohort}:{finding}"] = ev.flag_rate(g.score.values, thr)
    for cohort, g in df.groupby("cohort"):
        a = {"padding_median": float(g.cam_padding.median()),
             "border_median": float(g.cam_border.median()),
             "border_area_median": float(g.cam_border_area.median()),
             "padding_or_border_gt_50pct": float(((g.cam_padding + g.cam_border) > .5).mean())}
        if g.cam_lungs.notna().any():
            a["lungs_median"] = float(g.cam_lungs.median())
            a["lungs_iqr"] = [float(g.cam_lungs.quantile(.25)), float(g.cam_lungs.quantile(.75))]
            a["lungs_area_median"] = float(g.cam_lungs_area.median())
            # Enrichment > 1 means the map concentrates on lungs more than chance.
            enrich = g.cam_lungs / g.cam_lungs_area
            a["lungs_enrichment_median"] = float(enrich.median())
            a["lungs_enrichment_gt1"] = float((enrich > 1).mean())
        metrics["attribution"][cohort] = a
    e = df[ext]
    base_pred = e.score ≥ thr
    for key, (label, _) in PERTURBATIONS.items():
        p = e[f"pert_{key}"]
        metrics["robustness"][key] = {
            "label": label, "n": int(len(e)),
            "flip_rate": float(((p ≥ thr) ≠ base_pred).mean()),
            "flip_rate_ci": ev.wilson(int(((p ≥ thr) ≠ base_pred).sum()), len(e)),
            "median_abs_shift": float((p - e.score).abs().median()),
            "to_pneumonia": int(((p ≥ thr) & ~base_pred).sum()),
            "to_normal": int(((p < thr) & base_pred).sum()),
        }
    rs = rsna_main
    metrics["operating_points"] = {}
    for target in (0.8, 0.9):
        # Lowest threshold reaching the target specificity (scores are saturated, so
        # thresholds can sit extremely close to 1).
        for t in sorted(set(rs.score.values)):
            c = ev.confusion(rs.label.values, rs.score.values, float(t))
            spec, sens = c["tn"] / (c["tn"] + c["fp"]), c["tp"] / (c["tp"] + c["fn"])
            if spec ≥ target:
```

```
                metrics["operating_points"][f"rsna_spec_{int(target * 100)}"] = {
                    "threshold": float(t), "sensitivity": sens, "specificity": spec}
                break
    metrics["roc"] = {"rsna_normal_vs_opacity": ev.roc_points(rsna_main.label.values, rsna_main.score.values)}
    metrics["runtime_seconds"] = round(time.time() - t0, 1)

    OUT.mkdir(exist_ok=True)
    df.drop(columns=["file", "mask"]).to_csv(OUT / "scores.csv", index=False, float_format="%.9g")
    (OUT / "metrics.json").write_text(json.dumps(metrics, indent=2, default=float))
    print(json.dumps({k2: {m: v[m] for m in ("n", "sensitivity", "specificity", "auc")}
                      for k2, v in metrics["cohorts"].items()}, indent=1))
    print(json.dumps(metrics["flag_rates"], indent=1))
    print(json.dumps(metrics["attribution"], indent=1))
    print(json.dumps({k2: (v["flip_rate"], v["to_pneumonia"], v["to_normal"]) for k2, v in
metrics["robustness"].items()}, indent=1))
    print(json.dumps(metrics["operating_points"], indent=1))
    print(json.dumps(metrics["saturation"], indent=1), "ECE", metrics["calibration"]["rsna_normal_vs_opacity"]
["ece"])


if __name__ == "__main__":
    main()

```

14 scripts/build_gallery.py

```python
"""Pick teaching cases from the evaluation by rule and copy them into assets/gallery/.

Only redistributable sources are used: Kermany (CC BY 4.0) and the NLM Montgomery
and Shenzhen sets. RSNA images are never copied. Every pick is a deterministic
rule over results/scores.csv, so the gallery can be regenerated.
"""
from __future__ import annotations

import json
import shutil
import sys
from pathlib import Path

import numpy as np
import pandas as pd
from PIL import Image, ImageDraw, ImageFilter, ImageOps

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
from xraylens.model import load_model, predict  # noqa: E402

EVAL, OUT = ROOT / "data" / "eval", ROOT / "assets" / "gallery"
AMBER, PNEU, NORM, OTHER, MUTED = "#ffb547", "#e05c35", "#38a596", "#8b7ff0", "#93a1a6"


def thumb(im: Image.Image) -> Image.Image:
    """Uniform 4:5 contact-sheet frame so every card in the sheet lines up."""
    return ImageOps.pad(im.convert("L"), (240, 300), color=11)


def main():
    s = pd.read_csv(ROOT / "results" / "scores.csv", keep_default_na=False, na_values=[""])
    man = pd.read_csv(EVAL / "manifest.csv", keep_default_na=False)
    s = s.merge(man[["cohort", "source_id", "file", "mask"]], on=["cohort", "source_id"])
    s["offbody"] = s.cam_padding + s.cam_border
    k, mg, sz = (s[s.cohort == c] for c in ("kermany_test", "montgomery", "shenzhen"))

    picks = [
        ("textbook-normal", k[(k.finding == "Normal") & (k.score < 1e-3)].sort_values("offbody").iloc[0],
         "Textbook normal", "Child · Kermany", True,
         "This child’s radiograph is part of the model’s <b>training set</b>. A near-zero score here shows
memory, "
         "not skill. Compare it with the adult normals in this sheet."),
        ("textbook-pneumonia", k[(k.finding == "Pneumonia") & (k.score > .999)].sort_values("offbody").iloc[0],
         "Textbook pneumonia", "Child · Kermany", True,
         "Labelled pneumonia, <b>seen in training</b>, scored with saturated confidence. The heatmap sits on the
"
         "anatomy, so this is the explanation everyone hopes for. Now look at the next case."),
        ("right-answer-wrong-place", k[(k.finding == "Pneumonia") & (k.score >
.99)].sort_values("offbody").iloc[-1],
         "Right answer, wrong place?", "Child · Kermany", True,
         "Correct label, but most Grad-CAM mass falls on the image frame and padding rather than the lungs. "
         "A right answer from the wrong evidence is how shortcut learning hides inside high accuracy."),
        ("adult-false-alarm", mg[(mg.finding == "Normal") & (mg.score > .99)].sort_values("cam_lungs").iloc[0],
         "Healthy adult, flagged", "Adult · Montgomery", False,
         "An adult radiograph read as <b>normal</b> by radiologists, scored as pneumonia with near-certainty. "
         "The teal outline is the radiologist’s lung mask. Note where the heat peaks: on the burned-in "
         "<b>“PA ERECT”</b> marker, not the lungs. In step 04, the tighter crop that removes the marker "
         "turns the verdict to NORMAL. This is shortcut learning caught in the act."),
        ("adult-true-normal", mg[(mg.finding == "Normal") & (mg.score < .05)].sort_values("cam_lungs").iloc[-1],
         "Healthy adult, cleared", "Adult · Montgomery", False,
         "A normal adult the model gets right. Use the stress test below to see how easily that verdict flips."),
        ("tb-flagged", mg[(mg.finding == "Tuberculosis") & (mg.score > .99)].sort_values("cam_lungs").iloc[-1],
         "Tuberculosis, called pneumonia", "Adult · Montgomery", False,
         "Tuberculosis is a disease this model was never taught. It can only answer NORMAL or PNEUMONIA, so "
         "abnormal lungs of any cause get pushed into one of two boxes."),
        ("tb-missed", sz[(sz.finding == "Tuberculosis") & (sz.score < .01)].sort_values("score").iloc[0],
```

```
          "Tuberculosis, called normal", "Adult · Shenzhen", False,
          "The same disease from another hospital, now scored <b>normal</b> with confidence. A binary classifier "
          "has no way to say “something else is wrong”."),
    ]

    if OUT.exists():
        shutil.rmtree(OUT)
    OUT.mkdir(parents=True)
    items = []
    for gid, row, title, subtitle, seen, story in picks:
        src = EVAL / row.file
        dst = f"{gid}.png"
        shutil.copy(src, OUT / dst)
        thumb(Image.open(src)).save(OUT / f"{gid}_thumb.jpg", quality=88)
        mask = ""
        if row["mask"]:
            mask = f"assets/gallery/{gid}_lungs.png"
            shutil.copy(EVAL / row["mask"], ROOT / mask)
        tags = [("seen in training", PNEU)] if seen else [("external · never seen", NORM)]
        tags.append((f"label: {row.finding.lower()}", MUTED))
        if row.cohort in ("montgomery",):
            tags.append(("radiologist lung mask", NORM))
        licence = "CC BY 4.0, Kermany et al. 2018" if row.cohort == "kermany_test" else "NLM public release,
Jaeger et al. 2014"
        items.append({"id": gid, "file": dst, "thumb": f"{gid}_thumb.jpg", "mask": mask, "title": title,
                      "subtitle": subtitle, "seen_in_training": seen, "story": story, "tags": tags,
                      "source": f"{row.cohort}:{row.source_id}", "licence": licence, "label": row.finding})
        print(f"{gid:26s} {row.cohort}:{row.source_id:18s} score={row.score:.6f} offbody={row.offbody:.2f}")

    # A deliberately out-of-domain input. Generated here, so there are no licence questions.
    rng = np.random.default_rng(7)
    im = Image.new("L", (512, 420), 30)
    d = ImageDraw.Draw(im)
    for i in range(14):
        r = 20 + i * 16
        d.ellipse((256 - r, 210 - r, 256 + r, 210 + r), outline=int(80 + 10 * i), width=6)
    for _ in range(40):
        x, y = rng.integers(0, 512), rng.integers(0, 420)
        d.rectangle((x, y, x + rng.integers(8, 40), y + rng.integers(8, 40)), fill=int(rng.integers(90, 230)))
    im = im.filter(ImageFilter.GaussianBlur(1.2))
    im.save(OUT / "not-an-xray.png")
    thumb(im).save(OUT / "not-an-xray_thumb.jpg", quality=88)
    model, cfg = load_model()
    p = predict(im, model, cfg)
    items.append({"id": "not-an-xray", "file": "not-an-xray.png", "thumb": "not-an-xray_thumb.jpg", "mask": "",
                  "title": "Not an X-ray", "subtitle": "Synthetic pattern", "seen_in_training": False,
                  "story": f"Rings and rectangles, no anatomy at all. The model still answers "
                           f"<b>{p.label}</b> ({p.pneumonia_score:.4f}). A classifier has no “I don’t know”
output. "
                           "Recognising an unsuitable input is the job of the system around it.",
                  "tags": [("out of domain", OTHER), ("generated", MUTED)], "source": "generated",
                  "licence": "Generated by scripts/build_gallery.py", "label": "n/a"})
    print(f"{'not-an-xray':26s} generated score={p.pneumonia_score:.6f}")
    (OUT / "gallery.json").write_text(json.dumps(items, indent=2))


if __name__ == "__main__":
    main()

```

15 scripts/export_onnx.py

```python
"""Export the classifier to an ONNX graph for in-browser inference (web/).

Two properties matter:

1. The graph outputs the logits AND raw class-activation maps. For this
   architecture (layer4 → global average pool → fc), Grad-CAM at layer4 equals
   CAM with fc-weight differences, exactly:
       d(z_t - z_o)/dA_k(i,j) = (W[t,k] - W[o,k]) / (H*W)      for every (i, j)
   so Grad-CAM's channel weights are (W[t]-W[o])/(H*W), and the map is
   ReLU(sum_k (W[t,k]-W[o,k]) * A_k) up to a positive scale that max-normalisation
   removes. The browser therefore needs no gradients. verify() checks this
   against xraylens.explainability.gradcam.

2. The weights are NOT redistributed. Every initializer is rewritten as ONNX
   external data pointing at a byte range inside the model author's original
   model.safetensors, which the browser downloads directly from Hugging Face.
   The .onnx file we ship holds only the graph (a few hundred KB).
"""
from __future__ import annotations

import json
import os
import struct
import sys
from pathlib import Path

import numpy as np
import onnx
import torch
from onnx import numpy_helper
from safetensors.numpy import load_file

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
from xraylens.explainability import gradcam  # noqa: E402
from xraylens.model import DEFAULT_DIR, load_model, preprocess  # noqa: E402

OUT = ROOT / "web" / "model"


class WithCAM(torch.nn.Module):
    def __init__(self, net):
        super().__init__()
        self.net = net

    def forward(self, x):
        n = self.net
        x = n.maxpool(n.relu(n.bn1(n.conv1(x))))
        a = n.layer4(n.layer3(n.layer2(n.layer1(x))))            # (B, 2048, 10, 10)
        logits = n.fc(torch.flatten(n.avgpool(a), 1))
        w = n.fc.weight                                          # (2, 2048)
        diff = torch.stack([w[0] - w[1], w[1] - w[0]])           # class c vs the other
        cams = torch.relu(torch.einsum("ck,bkhw->bchw", diff, a))  # un-normalised CAMs
        return logits, cams


def safetensors_offsets(path: Path) -> tuple[dict, int]:
    with open(path, "rb") as f:
        n = struct.unpack("<Q", f.read(8))[0]
        header = json.loads(f.read(n))
    header.pop("__metadata__", None)
    return header, 8 + n


def main():
    model, cfg = load_model(device=torch.device("cpu"))
    wrapped = WithCAM(model).eval()
```

```
    x = torch.zeros(1, 3, 320, 320)
    OUT.mkdir(parents=True, exist_ok=True)
    tmp = OUT / "_tmp.onnx"
    torch.onnx.export(wrapped, (x,), str(tmp), input_names=["input"], output_names=["logits", "cams"],
                      dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}, "cams": {0: "batch"}},
                      opset_version=17, do_constant_folding=False, dynamo=False)
    m = onnx.load(str(tmp), load_external_data=True)
    tmp.unlink()
    for extra in OUT.glob("_tmp.onnx*"):
        extra.unlink()

    st_path = DEFAULT_DIR / "model.safetensors"
    header, base = safetensors_offsets(st_path)
    tensors = load_file(str(st_path))
    by_bytes = {}
    for name, arr in tensors.items():
        by_bytes.setdefault(arr.tobytes(), []).append(name)

    kept, mapped = [], 0
    for init in m.graph.initializer:
        arr = numpy_helper.to_array(init)
        names = by_bytes.get(np.ascontiguousarray(arr).tobytes())
        if not names or arr.dtype != np.float32:
            kept.append(init.name)  # small constants created by the exporter stay inline
            continue
        meta = header[names[0]]
        start, end = meta["data_offsets"]
        assert meta["dtype"] == "F32" and end - start == arr.nbytes
        init.ClearField("raw_data")
        del init.float_data[:]
        init.data_location = onnx.TensorProto.EXTERNAL
        del init.external_data[:]
        for k, v in (("location", "model.safetensors"), ("offset", str(base + start)), ("length",
str(arr.nbytes))):
            e = init.external_data.add()
            e.key, e.value = k, v
        mapped += 1
    onnx.save(m, str(OUT / "xraylens.onnx"))
    size = (OUT / "xraylens.onnx").stat().st_size
    print(f"mapped {mapped} initializers to model.safetensors; {len(kept)} inline: {kept[:6]}")
    print(f"graph file {size / 1024:.0f} KB")
    verify(model, cfg)


def verify(model, cfg):
    """ONNX (reading the original safetensors) must match PyTorch logits and Grad-CAM."""
    import shutil

    import onnxruntime as ort
    from PIL import Image

    work = OUT / "_verify"
    work.mkdir(exist_ok=True)
    shutil.copy(OUT / "xraylens.onnx", work / "xraylens.onnx")
    os.link(DEFAULT_DIR / "model.safetensors", work / "model.safetensors")  # ORT rejects symlinks
    sess = ort.InferenceSession(str(work / "xraylens.onnx"), providers=["CPUExecutionProvider"])
    worst_logit, worst_cam = 0.0, 0.0
    gallery = sorted((ROOT / "assets" / "gallery").glob("*.png"))
    gallery = [g for g in gallery if not g.stem.endswith("_lungs")]
    for p in gallery:
        _, xt = preprocess(Image.open(p), cfg)
        logits, cams = sess.run(None, {"input": xt.numpy()})
        with torch.no_grad():
            ref = model(xt).numpy()
        worst_logit = max(worst_logit, float(np.abs(logits - ref).max()))
        for c in (0, 1):
            g = gradcam(model, xt, c)
            cam = torch.nn.functional.interpolate(torch.from_numpy(cams[:, c:c + 1]), size=(320, 320),
                                                  mode="bilinear", align_corners=False)[0, 0].numpy()
            cam = cam / cam.max() if cam.max() > 1e-12 else np.zeros_like(cam)
```

```
            worst_cam = max(worst_cam, float(np.abs(cam - g).max()))
    shutil.rmtree(work)
    print(f"verify: max |Δlogit| = {worst_logit:.2e}, max |ΔCAM| = {worst_cam:.2e} over {len(gallery)} gallery files")
    assert worst_logit < 1e-3 and worst_cam < 1e-3


if __name__ == "__main__":
    main()

```

16 scripts/export_web.py

```python
"""Build the static data the web app needs (web/data, web/gallery).

* metrics.json, scores.json   evaluation results (from scripts/run_evaluation.py)
* gallery/                    teaching cases (from scripts/build_gallery.py), plus
                              PyTorch-precomputed outputs so pages render before the
                              in-browser model has downloaded: logits, raw 10x10
                              CAMs, stress-test scores and randomisation-test maps
* randomisation.json          the randomisation sanity check run across a sample of
                              evaluation images (it needs to re-initialise weights,
                              which the browser can't do with the fixed ONNX graph)

Run after export_onnx.py. Deterministic.
"""
from __future__ import annotations

import copy
import json
import shutil
import sys
from pathlib import Path

import numpy as np
import pandas as pd
import torch
import torch.nn.functional as F
from PIL import Image
from scipy.stats import spearmanr

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
from xraylens.model import load_model, predict_batch, preprocess  # noqa: E402
from xraylens.perturbations import PERTURBATIONS  # noqa: E402

WEB = ROOT / "web"


def raw_cams(model, x):
    """Logits and un-normalised CAMs (2, 10, 10), the same quantities the ONNX graph emits."""
    store = {}
    h = model.layer4.register_forward_hook(lambda m, i, o: store.__setitem__("a", o))
    with torch.no_grad():
        logits = model(x)
    h.remove()
    w = model.fc.weight
    diff = torch.stack([w[0] - w[1], w[1] - w[0]])
    cams = F.relu(torch.einsum("ck,bkhw->bchw", diff, store["a"]))[0]
    return logits[0].tolist(), cams.detach().numpy()


def upsample(cam10):
    t = F.interpolate(torch.from_numpy(cam10)[None, None], size=(320, 320), mode="bilinear", align_corners=False)
[0, 0].numpy()
    return t / t.max() if t.max() > 1e-12 else np.zeros_like(t)


def randomisation(model, x, target, seed=0):
    """Same cascade and statistic as explainability.randomization_check, but keeps raw 10x10 maps."""
    torch.manual_seed(seed)
    _, base = raw_cams(model, x)
    ref = upsample(base[target])
    probe = copy.deepcopy(model).eval()
    out = []
    for name, module in (("fc", probe.fc), ("layer4", probe.layer4), ("layer3", probe.layer3)):
        for m in module.modules():
            if hasattr(m, "reset_parameters"):
                m.reset_parameters()
        _, cams = raw_cams(probe, x)
```

```
        rho = spearmanr(ref.ravel()[::7], upsample(cams[target]).ravel()[::7]).statistic
        out.append({"randomised": name, "spearman": float(np.nan_to_num(rho)),
                    "cam": np.round(cams[target], 6).ravel().tolist()})
    return out


def main():
    model, cfg = load_model(device=torch.device("cpu"))
    data, gal_out = WEB / "data", WEB / "gallery"
    data.mkdir(parents=True, exist_ok=True)
    if gal_out.exists():
        shutil.rmtree(gal_out)
    gal_out.mkdir()

    shutil.copy(ROOT / "results" / "metrics.json", data / "metrics.json")
    if (ROOT / "results" / "web_parity.json").exists():
        shutil.copy(ROOT / "results" / "web_parity.json", data / "web_parity.json")
    s = pd.read_csv(ROOT / "results" / "scores.csv", keep_default_na=False, na_values=[""])
    cols = ["cohort", "source_id", "label", "finding", "score", "cam_lungs", "cam_lungs_area"]
    rows = s[cols].astype(object).where(s[cols].notna(), None).values.tolist()
    (data / "scores.json").write_text(json.dumps({"columns": cols, "rows": rows}, separators=(",", ":")))

    gallery = json.loads((ROOT / "assets" / "gallery" / "gallery.json").read_text())
    for item in gallery:
        for key in ("file", "thumb"):
            shutil.copy(ROOT / "assets" / "gallery" / item[key], gal_out / item[key])
        if item["mask"]:
            shutil.copy(ROOT / item["mask"], gal_out / Path(item["mask"]).name)
            item["mask"] = Path(item["mask"]).name
        im = Image.open(ROOT / "assets" / "gallery" / item["file"])
        im.load()
        _, x = preprocess(im, cfg)
        logits, cams = raw_cams(model, x)
        pred = int(torch.tensor(logits).softmax(0)[1] >= cfg["threshold"])
        stress_ims = [im] + [fn(im) for _, fn in PERTURBATIONS.values()]
        stress_scores = predict_batch(stress_ims, model, cfg).tolist()
        item["precomputed"] = {
            "logits": logits,
            "cams": [np.round(cams[c], 6).ravel().tolist() for c in (0, 1)],
            "stress": dict(zip(["original"] + list(PERTURBATIONS), stress_scores)),
            "randomisation": {cfg["classes"][c]: randomisation(model, x, c) for c in (0, 1)},
            "predicted": cfg["classes"][pred],
        }
        print(f"{item['id']:26s} score={torch.tensor(logits).softmax(0)[1]:.6f}")
    (gal_out / "gallery.json").write_text(json.dumps(gallery, separators=(",", ":")))

    # Population randomisation test: 60 RSNA + 60 Montgomery images, predicted class.
    man = pd.read_csv(ROOT / "data" / "eval" / "manifest.csv", keep_default_na=False)
    sample = pd.concat([man[man.cohort == c].sample(60, random_state=3) for c in ("rsna", "montgomery")])
    rows = []
    for r in sample.itertuples():
        im = Image.open(ROOT / "data" / "eval" / r.file)
        _, x = preprocess(im, cfg)
        logits, _ = raw_cams(model, x)
        target = int(logits[1] > logits[0])
        st = randomisation(model, x, target)
        rows.append({"cohort": r.cohort, **{f"rho_{d['randomised']}": d["spearman"] for d in st}})
    df = pd.DataFrame(rows)
    df["max_abs"] = df[["rho_fc", "rho_layer4", "rho_layer3"]].abs().max(axis=1)
    summary = {"n": len(df), "pass_threshold": 0.4,
               "pass_rate": float((df.max_abs < 0.4).mean()),
               "median_abs": {c: float(df[c].abs().median()) for c in ("rho_fc", "rho_layer4", "rho_layer3")},
               "rows": df.round(4).to_dict(orient="records")}
    (data / "randomisation.json").write_text(json.dumps(summary, separators=(",", ":")))
    print("randomisation pass rate", summary["pass_rate"], summary["median_abs"])


if __name__ == "__main__":
    main()

```

17 scripts/check_web_parity.py

```python
"""Check that the browser pipeline (web/js) reproduces the Python pipeline.

Serves web/ locally with cross-origin-isolation headers, opens a harness page in
headless Chrome, and compares, image by image:
  * letterboxed 320x320 model input        (SHA-1 of bytes, must be identical)
  * deterministic perturbations             (SHA-1, must be identical)
  * input-check warnings                    (must be identical)
  * PNEUMONIA score and Grad-CAM            (max abs difference)

Usage: python scripts/check_web_parity.py [--n 60]
Needs data/eval (scripts/fetch_eval_data.py) and playwright.
"""
from __future__ import annotations

import argparse
import functools
import hashlib
import http.server
import io
import json
import shutil
import sys
import threading
from pathlib import Path

import numpy as np
import pandas as pd
from PIL import Image

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
from xraylens.explainability import gradcam  # noqa: E402
from xraylens.inputs import assess  # noqa: E402
from xraylens.model import letterbox, load_model, predict, preprocess  # noqa: E402
from xraylens.perturbations import crop_center, hflip, invert, low_contrast, rotate10  # noqa: E402

WEB = ROOT / "web"
WORK = WEB / "_parity"
EXACT = {"hflip": hflip, "rotate10": rotate10, "crop": crop_center, "contrast": low_contrast, "invert": invert}


class Handler(http.server.SimpleHTTPRequestHandler):
    def end_headers(self):
        self.send_header("Cross-Origin-Opener-Policy", "same-origin")
        self.send_header("Cross-Origin-Embedder-Policy", "credentialless")
        super().end_headers()

    def log_message(self, *a):
        pass


def sha(im: Image.Image) -> str:
    return hashlib.sha1(im.tobytes()).hexdigest()


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--n", type=int, default=60)
    a = ap.parse_args()
    model, cfg = load_model()
    man = pd.read_csv(ROOT / "data" / "eval" / "manifest.csv", keep_default_na=False)
    sample = pd.concat([g.sample(a.n // 4, random_state=1) for _, g in man.groupby("cohort")])
    if WORK.exists():
        shutil.rmtree(WORK)
    WORK.mkdir()
    cases = []
    for i, r in enumerate(sample.itertuples()):
```

```
        im = Image.open(ROOT / "data" / "eval" / r.file)
        im.load()
        # Every third image as JPEG to exercise the JPEG decoder too.
        if i % 3 == 0:
            name = f"{i:03d}.jpg"
            im.convert("L").save(WORK / name, quality=90)
        else:
            name = f"{i:03d}.png"
            im.save(WORK / name)
        im = Image.open(WORK / name)
        im.load()
        boxed, x = preprocess(im, cfg)
        p = predict(im, model, cfg)
        cam = gradcam(model, x, 1)
        cases.append({
            "file": name, "cohort": r.cohort,
            "boxed": sha(letterbox(im, 320)), "score": p.pneumonia_score,
            "pert": {k: sha(fn(im)) for k, fn in EXACT.items()},
            "warnings": assess(im),
            "cam": [round(float(v), 6) for v in cam[::16, ::16].ravel()],  # 20x20 subsample
        })
    (WORK / "expected.json").write_text(json.dumps(cases))
    (WORK / "index.html").write_text(HARNESS)

    server = http.server.ThreadingHTTPServer(("127.0.0.1", 0), functools.partial(Handler, directory=str(WEB)))
    threading.Thread(target=server.serve_forever, daemon=True).start()
    url = f"http://127.0.0.1:{server.server_address[1]}/_parity/index.html"
    from playwright.sync_api import sync_playwright
    with sync_playwright() as pw:
        browser = pw.chromium.launch(channel="chrome", headless=True)
        page = browser.new_page()
        page.on("console", lambda m: print("  [browser]", m.text) if m.type == "error" else None)
        page.goto(url)
        page.wait_for_function("window.__done === true", timeout=600_000)
        res = page.evaluate("window.__results")
        info = page.evaluate("window.__info")
        browser.close()
    server.shutdown()
    shutil.rmtree(WORK)

    exp = {c["file"]: c for c in cases}
    boxed_ok = sum(r["boxed"] == exp[r["file"]]["boxed"] for r in res)
    pert_ok = {k: sum(r["pert"][k] == exp[r["file"]]["pert"][k] for r in res) for k in EXACT}
    warn_ok = sum(r["warnings"] == exp[r["file"]]["warnings"] for r in res)
    dscore = np.array([abs(r["score"] - exp[r["file"]]["score"]) for r in res])
    dcam = np.array([np.abs(np.array(r["cam"]) - np.array(exp[r["file"]]["cam"])).max() for r in res])
    labels_ok = sum((r["score"] >= .5) == (exp[r["file"]]["score"] >= .5) for r in res)
    n = len(res)
    report = {"images": n, "browser": info, "letterbox_identical": boxed_ok,
              "perturbations_identical": pert_ok, "warnings_identical": warn_ok,
              "labels_identical": labels_ok, "max_abs_score_diff": float(dscore.max()),
              "median_abs_score_diff": float(np.median(dscore)), "max_abs_cam_diff": float(dcam.max())}
    print(json.dumps(report, indent=1))
    for r in res:
        e = exp[r["file"]]
        if r["boxed"] != e["boxed"] or r["warnings"] != e["warnings"]:
            print("  mismatch", r["file"], e["cohort"], "boxed" if r["boxed"] != e["boxed"] else "",
r["warnings"], e["warnings"])
    (ROOT / "results" / "web_parity.json").write_text(json.dumps(report, indent=2))
    ok = boxed_ok == n and all(v == n for v in pert_ok.values()) and labels_ok == n and dscore.max() < 1e-4
    sys.exit(0 if ok else 1)


HARNESS = """<!doctype html><meta charset="utf-8"><script type="module">
import { decodeBlob, letterbox, assess, PERTURBATIONS, upsampleCam } from "../js/imaging.js";
import { loadModel, infer, backendInfo } from "../js/model.js";
const hex = async (u8) => [...new Uint8Array(await crypto.subtle.digest("SHA-1", u8))].map(b =>
b.toString(16).padStart(2, "0")).join("");
const cases = await (await fetch("expected.json")).json();
await loadModel();
```

```
window.__info = { ...backendInfo(), userAgent: navigator.userAgent };
const out = [];
for (const c of cases) {
  const img = await decodeBlob(await (await fetch(c.file)).blob());
  // decodeBlob returns RGB; PIL keeps greyscale JPEG/PNG as "L" then pads in RGB, which matches.
  const r = { file: c.file, boxed: await hex(letterbox(img, 320).data), pert: {} };
  for (const [k, , fn, exact] of PERTURBATIONS) if (exact) r.pert[k] = await hex(fn(img).data);
  try { r.warnings = assess(img); } catch (e) { r.warnings = ["ERROR " + e.message]; }
  const [res] = await infer([img]);
  r.score = res.pneumonia;
  const cam = upsampleCam(res.cams[1], res.camDims[0], res.camDims[1], 320, 320);
  r.cam = []; for (let y = 0; y < 320; y += 16) for (let x = 0; x < 320; x += 16) r.cam.push(cam[y * 320 + x]);
  out.push(r);
}
window.__results = out; window.__done = true;
</script>"""

if __name__ == "__main__":
    main()

```

18 scripts/serve_web.py

```
"""Serve web/ locally with the same cross-origin-isolation headers as vercel.json."""
import functools
import http.server
import sys
from pathlib import Path

WEB = Path(__file__).resolve().parent.parent / "web"


class Handler(http.server.SimpleHTTPRequestHandler):
    extensions_map = {**http.server.SimpleHTTPRequestHandler.extensions_map, ".mjs": "text/javascript",
                      ".js": "text/javascript", ".wasm": "application/wasm"}

    def end_headers(self):
        self.send_header("Cross-Origin-Opener-Policy", "same-origin")
        self.send_header("Cross-Origin-Embedder-Policy", "credentialless")
        super().end_headers()


if __name__ == "__main__":
    port = int(sys.argv[1]) if len(sys.argv) > 1 else 8600
    print(f"http://localhost:{port}")
    http.server.ThreadingHTTPServer(("127.0.0.1", port), functools.partial(Handler,
directory=str(WEB))).serve_forever()

```

19 web/js/model.js

```
// In-browser inference with ONNX Runtime Web.
//
// The graph (model/xraylens.onnx, ~66 KB) references every weight by byte range
// inside the model author's original model.safetensors, which we download straight
// from Hugging Face at a pinned commit. We never re-host the weights.
import * as ort from "../vendor/ort/ort.wasm.min.mjs";
import { letterbox, writeTensor } from "./imaging.js";

export const REPO = "Purino/resnet50-chest-xray-pneumonia";
export const REVISION = "da2a7724bfa53ca6693fd4e1783b211963ed2a55";
export const WEIGHTS_URL = `https://huggingface.co/${REPO}/resolve/${REVISION}/model.safetensors`;
export const WEIGHTS_BYTES = 94289936;
export const CONFIG = { classes: ["NORMAL", "PNEUMONIA"], imageSize: 320, threshold: 0.5,
                        mean: [0.485, 0.456, 0.406], std: [0.229, 0.224, 0.225] };
const CACHE = "xraylens-weights-v1";

ort.env.wasm.wasmPaths = new URL("../vendor/ort/", import.meta.url).href;
// Threads need cross-origin isolation (COOP/COEP headers, see vercel.json). Without it, one thread.
ort.env.wasm.numThreads = self.crossOriginIsolated ? Math.min(navigator.hardwareConcurrency || 4, 8) : 1;

async function fetchWeights(onProgress) {
  let cache = null;
  try { cache = await caches.open(CACHE); } catch { /* private mode: no Cache Storage */ }
  if (cache) {
    const hit = await cache.match(WEIGHTS_URL);
    if (hit) { onProgress?.(1, "cached"); return new Uint8Array(await hit.arrayBuffer()); }
  }
  const res = await fetch(WEIGHTS_URL, { mode: "cors" });
  if (!res.ok || !res.body) throw new Error(`Hugging Face returned ${res.status} for the model weights.`);
  const total = Number(res.headers.get("content-length")) || WEIGHTS_BYTES;
  const buf = new Uint8Array(total);
  const reader = res.body.getReader();
  let got = 0;
  for (;;) {
    const { done, value } = await reader.read();
    if (done) break;
    buf.set(value, got);
    got += value.length;
    onProgress?.(got / total, "download");
  }
  if (got !== WEIGHTS_BYTES) throw new Error(`Expected ${WEIGHTS_BYTES} bytes of weights, got ${got}.`);
  if (cache) {
    try { await cache.put(WEIGHTS_URL, new Response(buf, { headers: { "content-type": "application/octet-stream"
} })); }
    catch { /* quota exceeded: fine, we just re-download next time */ }
  }
  return buf;
}

let sessionPromise = null;

export function loadModel(onProgress) {
  if (!sessionPromise) {
    sessionPromise = (async () => {
      const [graph, weights] = await Promise.all([
        fetch(new URL("../model/xraylens.onnx", import.meta.url)).then((r) => r.arrayBuffer()),
        fetchWeights(onProgress),
      ]);
      onProgress?.(1, "compile");
      const session = await ort.InferenceSession.create(graph, {
        executionProviders: ["wasm"],
        externalData: [{ path: "model.safetensors", data: weights }],
        graphOptimizationLevel: "all",
      });
      onProgress?.(1, "ready");
      return session;
    })().catch((e) => { sessionPromise = null; throw e; });
```

```
  }
  return sessionPromise;
}

const softmax = (a, b) => { const m = Math.max(a, b), ea = Math.exp(a - m), eb = Math.exp(b - m); return [ea / (ea + eb), eb / (ea + eb)]; };

// Run a batch of decoded images. Returns per image: boxed input, scores, logits, raw 10x10 CAMs.
export async function infer(images) {
  const session = await loadModel();
  const S = CONFIG.imageSize, n = images.length, plane = 3 * S * S;
  const data = new Float32Array(n * plane);
  const boxed = images.map((im, i) => { const b = letterbox(im, S); writeTensor(b, CONFIG.mean, CONFIG.std, data, i); return b; });
  const out = await session.run({ input: new ort.Tensor("float32", data, [n, 3, S, S]) });
  const L = out.logits.data, C = out.cams.data, [, , ch, cw] = out.cams.dims;
  return boxed.map((b, i) => {
    const z = [L[2 * i], L[2 * i + 1]];
    const [pn, pp] = softmax(z[0], z[1]);
    const cams = [0, 1].map((c) => C.slice((i * 2 + c) * ch * cw, (i * 2 + c + 1) * ch * cw));
    return { boxed: b, logits: z, scores: { NORMAL: pn, PNEUMONIA: pp }, pneumonia: pp,
             label: pp >= CONFIG.threshold ? "PNEUMONIA" : "NORMAL", cams, camDims: [ch, cw] };
  });
}

export const backendInfo = () => ({ threads: ort.env.wasm.numThreads, isolated: self.crossOriginIsolated });

```

20 web/js/imaging.js

```
// Image handling for XRayLens in the browser.
//
// Everything that feeds the model is a line-by-line port of the Pillow code the
// model card's inference.py uses (Resample.c fixed-point kernels, ImageOps.pad,
// Geometry.c affine bilinear, ImagingBlend), so browser inputs are bit-identical
// to PyTorch's. tests/test_web_parity.py checks this in a real browser.

import { INFERNO } from "./inferno.js";

// An image is { w, h, c, data: Uint8Array } with c = 1 (grey) or 3 (RGB), row-major, interleaved.
export const makeImage = (w, h, c, data) => ({ w, h, c, data: data || new Uint8Array(w * h * c) });

// Python's round(): ties go to the even neighbour.
export function roundHalfEven(x) {
  const r = Math.round(x);
  return Math.abs(x % 1) === 0.5 && r % 2 !== 0 ? r - 1 : r;
}

// ── decoding ──────────────────────────────────────────────────────────────
export async function decodeBlob(blob) {
  // No colour management and no premultiplication: PIL applies neither.
  const bmp = await createImageBitmap(blob, { colorSpaceConversion: "none", premultiplyAlpha: "none",
                                               imageOrientation: "from-image" });
  const canvas = new OffscreenCanvas(bmp.width, bmp.height);
  const ctx = canvas.getContext("2d", { willReadFrequently: true });
  ctx.drawImage(bmp, 0, 0);
  const rgba = ctx.getImageData(0, 0, bmp.width, bmp.height).data;
  bmp.close();
  const out = makeImage(canvas.width, canvas.height, 3);
  for (let i = 0, j = 0; i < rgba.length; i += 4, j += 3) {
    out.data[j] = rgba[i]; out.data[j + 1] = rgba[i + 1]; out.data[j + 2] = rgba[i + 2];
  }
  return out;
}

export function toRGB(img) {
  if (img.c === 3) return img;
  const out = makeImage(img.w, img.h, 3);
  for (let i = 0; i < img.w * img.h; i++) out.data[3 * i] = out.data[3 * i + 1] = out.data[3 * i + 2] =
img.data[i];
  return out;
}

// PIL "L" conversion: ITU-R 601-2 luma in 16.16 fixed point.
export function toGray(img) {
  if (img.c === 1) return img;
  const out = makeImage(img.w, img.h, 1);
  const d = img.data;
  for (let i = 0; i < img.w * img.h; i++) {
    out.data[i] = (d[3 * i] * 19595 + d[3 * i + 1] * 38470 + d[3 * i + 2] * 7471 + 0x8000) >>> 16;
  }
  return out;
}

// ── Pillow Resample.c ─────────────────────────────────────────────────────
const PRECISION_BITS = 32 - 8 - 2;
const FILTERS = {
  bilinear: { support: 1.0, fn: (x) => { x = Math.abs(x); return x < 1 ? 1 - x : 0; } },
  bicubic: { support: 2.0, fn: (x) => {
    const a = -0.5; x = Math.abs(x);
    if (x < 1) return ((a + 2) * x - (a + 3)) * x * x + 1;
    if (x < 2) return (((x - 5) * x + 8) * x - 4) * a;
    return 0;
  } },
};

function coeffs(inSize, outSize, filter) {
```

```
  const scale = inSize / outSize;
  const filterscale = Math.max(scale, 1.0);
  const support = filter.support * filterscale;
  const ksize = Math.ceil(support) * 2 + 1;
  const kk = new Int32Array(outSize * ksize);
  const bounds = new Int32Array(outSize * 2);
  const ss = 1.0 / filterscale;
  const pre = new Float64Array(ksize);
  for (let xx = 0; xx < outSize; xx++) {
    const center = (xx + 0.5) * scale;
    let xmin = Math.trunc(center - support + 0.5); if (xmin < 0) xmin = 0;
    let xmax = Math.trunc(center + support + 0.5); if (xmax > inSize) xmax = inSize;
    xmax -= xmin;
    let ww = 0;
    for (let x = 0; x < xmax; x++) { const w = filter.fn((x + xmin - center + 0.5) * ss); pre[x] = w; ww += w; }
    for (let x = 0; x < xmax; x++) {
      const v = ww !== 0 ? pre[x] / ww : pre[x];
      kk[xx * ksize + x] = v < 0 ? Math.trunc(-0.5 + v * (1 << PRECISION_BITS)) : Math.trunc(0.5 + v * (1 << PRECISION_BITS));
    }
    bounds[2 * xx] = xmin; bounds[2 * xx + 1] = xmax;
  }
  return { kk, bounds, ksize };
}

const clip8 = (ss) => { const v = Math.floor(ss / (1 << PRECISION_BITS)); return v < 0 ? 0 : v > 255 ? 255 : v; };

function resampleH(img, outW, filter) {
  const { kk, bounds, ksize } = coeffs(img.w, outW, filter);
  const out = makeImage(outW, img.h, img.c);
  const c = img.c;
  for (let y = 0; y < img.h; y++) {
    const row = y * img.w * c;
    for (let xx = 0; xx < outW; xx++) {
      const xmin = bounds[2 * xx], xmax = bounds[2 * xx + 1], k = xx * ksize;
      for (let ch = 0; ch < c; ch++) {
        let ss = 1 << (PRECISION_BITS - 1);
        for (let x = 0; x < xmax; x++) ss += img.data[row + (x + xmin) * c + ch] * kk[k + x];
        out.data[(y * outW + xx) * c + ch] = clip8(ss);
      }
    }
  }
  return out;
}

function resampleV(img, outH, filter) {
  const { kk, bounds, ksize } = coeffs(img.h, outH, filter);
  const out = makeImage(img.w, outH, img.c);
  const stride = img.w * img.c;
  for (let yy = 0; yy < outH; yy++) {
    const ymin = bounds[2 * yy], ymax = bounds[2 * yy + 1], k = yy * ksize;
    for (let i = 0; i < stride; i++) {
      let ss = 1 << (PRECISION_BITS - 1);
      for (let y = 0; y < ymax; y++) ss += img.data[(y + ymin) * stride + i] * kk[k + y];
      out.data[yy * stride + i] = clip8(ss);
    }
  }
  return out;
}

export function resize(img, w, h, method = "bilinear") {
  const f = FILTERS[method];
  let out = img;
  if (w !== img.w) out = resampleH(out, w, f);
  if (h !== img.h) out = resampleV(out, h, f);
  return out === img ? makeImage(img.w, img.h, img.c, img.data.slice()) : out;
}

// ImageOps.pad(image, (size, size), BILINEAR, color=black, centering=(0.5, 0.5)).
```

```
export function letterbox(img, size) {
  const rgb = toRGB(img);
  let w = size, h = size;
  const ratio = rgb.w / rgb.h;
  if (ratio > 1) h = roundHalfEven(rgb.h / rgb.w * size);
  else if (ratio < 1) w = roundHalfEven(rgb.w / rgb.h * size);
  const resized = resize(rgb, w, h, "bilinear");
  const out = makeImage(size, size, 3);
  const x0 = w !== size ? roundHalfEven((size - w) * 0.5) : 0;
  const y0 = h !== size ? roundHalfEven((size - h) * 0.5) : 0;
  for (let y = 0; y < h; y++) out.data.set(resized.data.subarray(y * w * 3, (y + 1) * w * 3), ((y + y0) * size + x0) * 3);
  return out;
}

// torchvision ToTensor + Normalize, written into a (n, 3, H, W) batch at index b.
export function writeTensor(boxed, mean, std, target, b = 0) {
  const n = boxed.w * boxed.h, off = b * 3 * n;
  for (let i = 0; i < n; i++) {
    for (let ch = 0; ch < 3; ch++) target[off + ch * n + i] = (boxed.data[3 * i + ch] / 255 - mean[ch]) / std[ch];
  }
}

// ── explanation helpers (mirror xraylens/explainability.py) ──────────
// torch.nn.functional.interpolate(..., mode="bilinear", align_corners=False)
export function upsampleCam(cam, inH, inW, outH, outW) {
  const out = new Float32Array(outH * outW);
  const sy = inH / outH, sx = inW / outW;
  for (let y = 0; y < outH; y++) {
    let fy = (y + 0.5) * sy - 0.5; if (fy < 0) fy = 0;
    const y0 = Math.floor(fy), y1 = Math.min(y0 + 1, inH - 1), ly = fy - y0;
    for (let x = 0; x < outW; x++) {
      let fx = (x + 0.5) * sx - 0.5; if (fx < 0) fx = 0;
      const x0 = Math.floor(fx), x1 = Math.min(x0 + 1, inW - 1), lx = fx - x0;
      const top = cam[y0 * inW + x0] * (1 - lx) + cam[y0 * inW + x1] * lx;
      const bot = cam[y1 * inW + x0] * (1 - lx) + cam[y1 * inW + x1] * lx;
      out[y * outW + x] = top * (1 - ly) + bot * ly;
    }
  }
  let peak = 0;
  for (const v of out) if (v > peak) peak = v;
  if (peak > 1e-12) for (let i = 0; i < out.length; i++) out[i] /= peak; else out.fill(0);
  return out;
}

export function contentMask(boxed) {
  const g = toGray(boxed), { w, h } = g;
  const rows = new Array(h).fill(false), cols = new Array(w).fill(false);
  for (let y = 0; y < h; y++) for (let x = 0; x < w; x++) if (g.data[y * w + x] > 2) { rows[y] = true; cols[x] = true; }
  const mask = new Uint8Array(w * h);
  const r0 = rows.indexOf(true), c0 = cols.indexOf(true);
  if (r0 < 0 || c0 < 0) return { mask, box: null };
  const r1 = h - rows.slice().reverse().indexOf(true), c1 = w - cols.slice().reverse().indexOf(true);
  for (let y = r0; y < r1; y++) for (let x = c0; x < c1; x++) mask[y * w + x] = 1;
  return { mask, box: [r0, r1, c0, c1] };
}

export function attributionStats(cam, boxed, lung) {
  const n = cam.length;
  let total = 0; for (const v of cam) total += v; total = total || 1;
  const { mask, box } = contentMask(boxed);
  let pad = 0, padArea = 0;
  for (let i = 0; i < n; i++) if (!mask[i]) { pad += cam[i]; padArea++; }
  const s = { padding: pad / total, padding_area: padArea / n, border: 0, border_area: 0 };
  if (box) {
    const [r0, r1, c0, c1] = box, mh = Math.trunc((r1 - r0) * 0.12), mw = Math.trunc((c1 - c0) * 0.12), w = boxed.w;
    let b = 0, bArea = 0;
```

```
    for (let y = r0; y < r1; y++) for (let x = c0; x < c1; x++) {
      const inner = y >= r0 + mh && y < r1 - mh && x >= c0 + mw && x < c1 - mw;
      if (!inner) { b += cam[y * w + x]; bArea++; }
    }
    s.border = b / total; s.border_area = bArea / n;
  }
  if (lung) {
    let l = 0, la = 0;
    for (let i = 0; i < n; i++) if (lung[i]) { l += cam[i]; la++; }
    s.lungs = l / total; s.lungs_area = la / n;
  }
  return s;
}

// matplotlib lookup: index = min(floor(t * N), N - 1)
const inferno = (t) => INFERNO[Math.min(Math.floor(Math.min(Math.max(t, 0), 1) * 256), 255)];

// Same blending as explainability.overlay: weak activations stay transparent.
export function overlayToCanvas(canvas, boxed, cam, alpha, lung) {
  const { w, h } = boxed;
  canvas.width = w; canvas.height = h;
  const ctx = canvas.getContext("2d");
  const out = ctx.createImageData(w, h);
  let outline = null;
  if (lung) outline = lungOutline(lung, w, h);
  for (let i = 0; i < w * h; i++) {
    const a = cam ? alpha * Math.min(Math.max(cam[i] * 1.4, 0), 1) : 0;
    const heat = cam ? inferno(cam[i]) : [0, 0, 0];
    for (let ch = 0; ch < 3; ch++) out.data[4 * i + ch] = boxed.data[3 * i + ch] * (1 - a) + heat[ch] * a;
    out.data[4 * i + 3] = 255;
    if (outline && outline[i]) { out.data[4 * i] = 56; out.data[4 * i + 1] = 165; out.data[4 * i + 2] = 150; }
  }
  ctx.putImageData(out, 0, 0);
}

function lungOutline(mask, w, h) {
  // mask & ~erode(mask, 5x5), as in app_state.lung_outline
  const out = new Uint8Array(w * h);
  for (let y = 0; y < h; y++) for (let x = 0; x < w; x++) {
    if (!mask[y * w + x]) continue;
    let edge = false;
    for (let dy = -2; dy <= 2 && !edge; dy++) for (let dx = -2; dx <= 2; dx++) {
      const yy = y + dy, xx = x + dx;
      if (yy < 0 || yy >= h || xx < 0 || xx >= w || !mask[yy * w + xx]) { edge = true; break; }
    }
    out[y * w + x] = edge ? 1 : 0;
  }
  return out;
}

export function imageToCanvas(canvas, img) {
  const rgb = toRGB(img);
  canvas.width = rgb.w; canvas.height = rgb.h;
  const ctx = canvas.getContext("2d");
  const out = ctx.createImageData(rgb.w, rgb.h);
  for (let i = 0; i < rgb.w * rgb.h; i++) {
    out.data[4 * i] = rgb.data[3 * i]; out.data[4 * i + 1] = rgb.data[3 * i + 1]; out.data[4 * i + 2] =
rgb.data[3 * i + 2]; out.data[4 * i + 3] = 255;
  }
  ctx.putImageData(out, 0, 0);
}

// ── perturbations (mirror xraylens/perturbations.py) ──────────────────
const gray = (img) => toGray(img);

export function hflip(img) {
  const g = gray(img), out = makeImage(g.w, g.h, 1);
  for (let y = 0; y < g.h; y++) for (let x = 0; x < g.w; x++) out.data[y * g.w + x] = g.data[y * g.w + (g.w - 1 -
x)];
  return out;
```

```
}

export function invert(img) {
  const g = gray(img), out = makeImage(g.w, g.h, 1);
  for (let i = 0; i < g.data.length; i++) out.data[i] = 255 - g.data[i];
  return out;
}

// Image.rotate(10, BILINEAR, fillcolor=0): Pillow's affine matrix + Geometry.c bilinear_filter8.
export function rotate10(img) {
  const g = gray(img), { w, h } = g, out = makeImage(w, h, 1);
  const ang = -10 * Math.PI / 180;
  const r15 = (v) => Math.round(v * 1e15) / 1e15;
  const m = [r15(Math.cos(ang)), r15(Math.sin(ang)), 0, r15(-Math.sin(ang)), r15(Math.cos(ang)), 0];
  const cx = w / 2, cy = h / 2;
  m[2] = m[0] * -cx + m[1] * -cy + cx;
  m[5] = m[3] * -cx + m[4] * -cy + cy;
  const clipX = (x) => (x < 0 ? 0 : x >= w ? w - 1 : x), clipY = (y) => (y < 0 ? 0 : y >= h ? h - 1 : y);
  for (let y = 0; y < h; y++) {
    let xin = m[0] * 0.5 + m[1] * (y + 0.5) + m[2];
    let yin = m[3] * 0.5 + m[4] * (y + 0.5) + m[5];
    for (let x = 0; x < w; x++, xin += m[0], yin += m[3]) {
      if (xin < 0 || xin >= w || yin < 0 || yin >= h) { out.data[y * w + x] = 0; continue; }
      const xs = xin - 0.5, ys = yin - 0.5, x0 = Math.floor(xs), y0 = Math.floor(ys), dx = xs - x0, dy = ys - y0;
      const r = clipY(y0) * w, a0 = clipX(x0), a1 = clipX(x0 + 1);
      let v1 = g.data[r + a0] + (g.data[r + a1] - g.data[r + a0]) * dx;
      let v2 = v1;
      if (y0 + 1 >= 0 && y0 + 1 < h) { const r2 = (y0 + 1) * w; v2 = g.data[r2 + a0] + (g.data[r2 + a1] -
g.data[r2 + a0]) * dx; }
      out.data[y * w + x] = Math.trunc(v1 + (v2 - v1) * dy);
    }
  }
  return out;
}

// ImageEnhance.Contrast(0.55): blend toward the rounded mean grey with Pillow's truncation.
export function lowContrast(img) {
  const g = gray(img);
  let sum = 0; for (const v of g.data) sum += v;
  const mean = Math.trunc(sum / g.data.length + 0.5), out = makeImage(g.w, g.h, 1);
  // ImagingBlend takes alpha as a C float and evaluates in single precision.
  const f = Math.fround, alpha = f(0.55);
  for (let i = 0; i < g.data.length; i++) out.data[i] = Math.trunc(f(mean + f(alpha * (g.data[i] - mean))));
  return out;
}

export function cropCenter(img) {
  const g = gray(img);
  const x0 = Math.trunc(g.w * 0.15), y0 = Math.trunc(g.h * 0.15), x1 = Math.trunc(g.w * 0.85), y1 =
Math.trunc(g.h * 0.85);
  const out = makeImage(x1 - x0, y1 - y0, 1);
  for (let y = y0; y < y1; y++) out.data.set(g.data.subarray(y * g.w + x0, y * g.w + x1), (y - y0) * out.w);
  return out;
}

// Pillow approximates a Gaussian with three box blurs; we use a true Gaussian of the same radius.
export function blur(img) {
  const g = gray(img), sigma = Math.max(g.w, g.h) / 300;
  const r = Math.ceil(sigma * 3), k = [];
  let ks = 0;
  for (let i = -r; i <= r; i++) { const v = Math.exp(-(i * i) / (2 * sigma * sigma)); k.push(v); ks += v; }
  const pass = (src, w, h, horiz) => {
    const dst = new Float32Array(w * h);
    for (let y = 0; y < h; y++) for (let x = 0; x < w; x++) {
      let s = 0;
      for (let i = -r; i <= r; i++) {
        const xx = horiz ? Math.min(Math.max(x + i, 0), w - 1) : x, yy = horiz ? y : Math.min(Math.max(y + i, 0),
h - 1);
        s += src[yy * w + xx] * k[i + r];
      }
```

```
      dst[y * w + x] = s / ks;
    }
    return dst;
  };
  const t = pass(pass(g.data, g.w, g.h, true), g.w, g.h, false);
  const out = makeImage(g.w, g.h, 1);
  for (let i = 0; i < t.length; i++) out.data[i] = Math.min(255, Math.max(0, Math.round(t[i])));
  return out;
}

// Seeded Gaussian noise (σ = 18). The stream differs from NumPy's, so per-pixel values differ.
export function noise(img) {
  const g = gray(img), out = makeImage(g.w, g.h, 1);
  let s = 0x9e3779b9;
  const rnd = () => { s ^= s << 13; s ^= s >>> 17; s ^= s << 5; return ((s >>> 0) + 0.5) / 4294967296; };
  for (let i = 0; i < g.data.length; i++) {
    const n = Math.sqrt(-2 * Math.log(rnd())) * Math.cos(2 * Math.PI * rnd());
    out.data[i] = Math.min(255, Math.max(0, Math.trunc(g.data[i] + 18 * n)));
  }
  return out;
}

export function marker(img) {
  const g = gray(img), size = Math.max(12, Math.trunc(g.w / 14));
  const canvas = new OffscreenCanvas(g.w, g.h), ctx = canvas.getContext("2d");
  const id = ctx.createImageData(g.w, g.h);
  for (let i = 0; i < g.data.length; i++) { id.data[4 * i] = id.data[4 * i + 1] = id.data[4 * i + 2] = g.data[i];
id.data[4 * i + 3] = 255; }
  ctx.putImageData(id, 0, 0);
  ctx.textBaseline = "top";
  ctx.font = `${size}px sans-serif`;
  ctx.fillStyle = "rgb(255,255,255)"; ctx.fillText("L", Math.trunc(g.w * 0.78), Math.trunc(g.h * 0.04));
  ctx.fillStyle = "rgb(235,235,235)"; ctx.fillText("PORTABLE", Math.trunc(g.w * 0.04), Math.trunc(g.h * 0.04));
  const px = ctx.getImageData(0, 0, g.w, g.h).data, out = makeImage(g.w, g.h, 1);
  for (let i = 0; i < g.data.length; i++) out.data[i] = px[4 * i];
  return out;
}

export const PERTURBATIONS = [
  ["hflip", "Mirror left-right", hflip, true],
  ["rotate10", "Rotate 10°", rotate10, true],
  ["crop", "Tighter collimation (70% crop)", cropCenter, true],
  ["contrast", "Low contrast (×0.55)", lowContrast, true],
  ["blur", "Mild blur", blur, false],
  ["noise", "Sensor noise (σ = 18)", noise, false],
  ["marker", "Burned-in text markers", marker, false],
  ["invert", "Inverted greyscale", invert, true],
]; // 4th field: bit-exact with the Python version

// ── input checks (mirror xraylens/inputs.py) ──────────────────────────
export class InputError extends Error {}

export function assess(img) {
  const { w, h } = img;
  if (Math.min(w, h) < 64) throw new InputError(`The image is ${w}×${h} px. It needs to be at least 64 px on each
side.`);
  const small = resize(toRGB(img), 128, 128, "bicubic");
  let mean = 0; for (const v of small.data) mean += v; mean /= small.data.length;
  let vs = 0; for (const v of small.data) vs += (v - mean) ** 2;
  if (Math.sqrt(vs / small.data.length) < 2) throw new InputError("The image is blank (a single flat colour).");
  const warnings = [];
  let chroma = 0;
  for (let i = 0; i < 128 * 128; i++) {
    const r = small.data[3 * i], g = small.data[3 * i + 1], b = small.data[3 * i + 2];
    chroma += Math.max(r, g, b) - Math.min(r, g, b);
  }
  if (chroma / (128 * 128) > 12) warnings.push("This image contains colour. Radiographs are greyscale, so it may
not be an X-ray. The model will still produce a confident-looking score.");
  const ratio = Math.max(w, h) / Math.min(w, h);
  if (ratio > 1.8) warnings.push(`Unusual aspect ratio (${ratio.toFixed(1)}:1). The image is letterboxed with
```

```
black bars, and the model saw few images shaped like this.`);
  if (Math.min(w, h) < 256) warnings.push("Low resolution. The model works at 320 px, so fine detail is lost.");
  if (looksInverted(img)) warnings.push("The image looks inverted (bone dark, air bright). The model was trained on bone-is-white radiographs.");
  return warnings;
}

function looksInverted(img) {
  const g = resize(toGray(img), 128, 128, "bicubic").data;
  const at = (y, x) => g[y * 128 + x];
  const meanRect = (y0, y1, x0, x1) => { let s = 0, n = 0; for (let y = y0; y < y1; y++) for (let x = x0; x < x1; x++) { s += at(y, x); n++; } return s / n; };
  let bs = 0, bn = 0;
  for (let y = 0; y < 128; y++) for (let x = 0; x < 128; x++) if (y < 8 || y >= 120 || x < 8 || x >= 120) {
    // inputs.py concatenates the four 8-px strips, so corner pixels are counted twice
    const times = (y < 8 || y >= 120 ? 1 : 0) + (x < 8 || x >= 120 ? 1 : 0);
    bs += at(y, x) * times; bn += times;
  }
  const darkCentre = meanRect(40, 88, 40, 88) < bs / bn - 40;
  const lungs = (meanRect(32, 77, 26, 45) + meanRect(32, 77, 83, 102)) / 2;
  const darkMidline = meanRect(26, 100, 58, 70) < lungs - 5;
  return darkCentre || darkMidline;
}

// ── DICOM (uncompressed transfer syntaxes only) ──────────────────────
export function decodeDicom(buffer, dicomParser) {
  let ds;
  try { ds = dicomParser.parseDicom(new Uint8Array(buffer)); }
  catch { throw new InputError("This DICOM file could not be parsed."); }
  const ts = ds.string("x00020010") || "";
  if (!["1.2.840.10008.1.2", "1.2.840.10008.1.2.1", "1.2.840.10008.1.2.2"].includes(ts.trim()))
    throw new InputError("This DICOM uses a compressed transfer syntax, which the browser version can't decode. Export it as PNG.");
  const rows = ds.uint16("x00280010"), cols = ds.uint16("x00280011"), bits = ds.uint16("x00280100");
  const spp = ds.uint16("x00280002") || 1, frames = parseInt(ds.string("x00280008") || "1", 10);
  const signed = ds.uint16("x00280103") === 1, photo = (ds.string("x00280004") || "").trim();
  const el = ds.elements.x7fe00010;
  if (!el || !rows || !cols) throw new InputError("This DICOM has no readable pixel data.");
  if (spp !== 1) throw new InputError("Colour DICOM images are not supported.");
  const n = rows * cols, frame = Math.floor(frames / 2), start = el.dataOffset + frame * n * (bits / 8);
  const view = new DataView(ds.byteArray.buffer, ds.byteArray.byteOffset);
  const arr = new Float32Array(n);
  for (let i = 0; i < n; i++) {
    arr[i] = bits === 8 ? ds.byteArray[start + i]
      : signed ? view.getInt16(start + 2 * i, true) : view.getUint16(start + 2 * i, true);
  }
  let max = -Infinity; for (const v of arr) if (v > max) max = v;
  if (photo === "MONOCHROME1") for (let i = 0; i < n; i++) arr[i] = max - arr[i];
  const sorted = Float32Array.from(arr).sort();
  const pct = (p) => { const k = (sorted.length - 1) * p, f = Math.floor(k), c = Math.min(f + 1, sorted.length - 1); return sorted[f] + (sorted[c] - sorted[f]) * (k - f); };
  const lo = pct(0.005), hi = pct(0.995), span = Math.max(hi - lo, 1e-6);
  const out = makeImage(cols, rows, 1);
  for (let i = 0; i < n; i++) out.data[i] = Math.trunc(Math.min(Math.max((arr[i] - lo) / span, 0), 1) * 255);
  const meta = {};
  for (const [tag, name] of [["x00080060", "Modality"], ["x00185101", "ViewPosition"], ["x00180015", "BodyPartExamined"]]) {
    const v = ds.string(tag); if (v) meta[name] = v.trim();
  }
  return { image: out, meta };
}

```

21 web/js/examine.js

```
// Examine page: steps 01–04, running the classifier in the browser.
import { $, esc, pct, note, stats, getJSON, getScores, chart, COLORS } from "./common.js";
import { decodeBlob, letterbox, toGray, assess, InputError, upsampleCam, attributionStats, overlayToCanvas,
         imageToCanvas, PERTURBATIONS, decodeDicom } from "./imaging.js";
import { loadModel, infer, CONFIG, WEIGHTS_BYTES, backendInfo } from "./model.js";

const [gallery, metrics, scores, randPop] = await Promise.all([
  getJSON("gallery/gallery.json"), getJSON("data/metrics.json"), getScores(), getJSON("data/randomisation.json"),
]);
const rsna = scores.filter((r) => r.cohort === "rsna");
const state = { token: 0 };
let modelReady = false;

// ── live model status ──────────────────────────────────────────────
const statusEl = $("#model-status");
function setStatus(html, frac) {
  statusEl.innerHTML = `${html}<span class="bar"><i style="width:${Math.round((frac ?? 0) * 100)}%"></i></span>`;
}
function startModel() {
  return loadModel((f, phase) => {
    if (phase === "download") setStatus(`Live model · downloading weights from Hugging Face <b>${pct(f, 0)}</b>
of ${(WEIGHTS_BYTES / 1e6).toFixed(0)} MB`, f);
    else if (phase === "cached") setStatus("Live model · weights cached on this device", 1);
    else if (phase === "compile") setStatus("Live model · <b>compiling</b> ResNet-50…", 1);
  }).then(() => {
    const b = backendInfo();
    modelReady = true;
    setStatus(`Live model · <b>ready</b> · ONNX Runtime Web, ${b.threads} thread${b.threads > 1 ? "s" : ""}, on
this device`, 1);
    if (state.kind === "gallery") liveCheck(state.token);
  }).catch((e) => {
    setStatus(`Live model · <b>failed to load</b>: ${esc(e.message)}. Gallery results still work.`, 0);
    throw e;
  });
}
const saveData = navigator.connection && (navigator.connection.saveData ||
/2g/.test(navigator.connection.effectiveType || ""));
let modelPromise = null;
const ensureModel = () => (modelPromise ||= startModel());
if (!saveData) ensureModel();
else setStatus('Live model · not loaded (data saver on) · <a href="#" id="load-model">load 94 MB</a>', 0);
document.addEventListener("click", (e) => { if (e.target.id === "load-model") { e.preventDefault();
ensureModel(); } });

// ── 01 contact sheet and upload ────────────────────────────────────
function renderSheet() {
  $("#sheet").innerHTML = gallery.map((g) => `
    <div class="card${state.id === g.id ? " active" : ""}">
      <img src="gallery/${g.thumb}" alt="${esc(g.title)}" loading="lazy" width="240" height="300">
      <div class="cap"><b>${esc(g.title)}</b>${esc(g.subtitle)}</div>
      <button class="btn${state.id === g.id ? " primary" : ""}" data-id="${g.id}">${state.id === g.id ? "●
Loaded" : "Load"}</button>
    </div>`).join("");
}
$("#sheet").addEventListener("click", (e) => { const id = e.target.closest("button")?.dataset.id; if (id)
selectGallery(id); });

const drop = $("#drop");
$("#file").addEventListener("change", (e) => e.target.files[0] && handleUpload(e.target.files[0]));
drop.addEventListener("dragover", (e) => { e.preventDefault(); drop.classList.add("over"); });
drop.addEventListener("dragleave", () => drop.classList.remove("over"));
drop.addEventListener("drop", (e) => { e.preventDefault(); drop.classList.remove("over"); if
(e.dataTransfer.files[0]) handleUpload(e.dataTransfer.files[0]); });

const softmax = ([a, b]) => { const m = Math.max(a, b), ea = Math.exp(a - m), eb = Math.exp(b - m); return [ea /
(ea + eb), eb / (ea + eb)]; };

```

```
async function maskFromUrl(url) {
  const m = letterbox(await decodeBlob(await (await fetch(url)).blob()), CONFIG.imageSize);
  const g = toGray(m).data, out = new Uint8Array(g.length);
  for (let i = 0; i < g.length; i++) out[i] = g[i] > 127 ? 1 : 0;
  return out;
}

async function selectGallery(id) {
  const token = ++state.token;
  const item = gallery.find((g) => g.id === id);
  Object.assign(state, { kind: "gallery", id, item, meta: {}, stressLive: null });
  renderSheet();
  $("#error").innerHTML = "";
  const img = await decodeBlob(await (await fetch(`gallery/${item.file}`)).blob());
  const lung = item.mask ? await maskFromUrl(`gallery/${item.mask}`) : null;
  if (token !== state.token) return;
  const pre = item.precomputed;
  const [pn, pp] = softmax(pre.logits);
  Object.assign(state, { img, lung, warnings: assess(img), boxed: letterbox(img, CONFIG.imageSize),
    result: { logits: pre.logits, scores: { NORMAL: pn, PNEUMONIA: pp }, pneumonia: pp, label: pp >=
CONFIG.threshold ? "PNEUMONIA" : "NORMAL",
              cams: pre.cams.map((c) => Float32Array.from(c)), camDims: [10, 10], source: "pytorch" } });
  render();
  if (modelReady) liveCheck(token);
}

// Re-run a gallery case through the in-browser model and compare with PyTorch.
async function liveCheck(token) {
  if (state.kind !== "gallery" || !state.img) return;
  const [r] = await infer([state.img]);
  if (token !== state.token) return;
  state.live = { pneumonia: r.pneumonia, delta: Math.abs(r.pneumonia - state.result.pneumonia) };
  renderLive();
}

async function handleUpload(file) {
  const token = ++state.token;
  Object.assign(state, { kind: "upload", id: null, item: null, lung: null, live: null, stressLive: null });
  renderSheet();
  $("#error").innerHTML = "";
  try {
    if (file.size > 20 * 1024 * 1024) throw new InputError(`The file is ${(file.size / 1e6).toFixed(1)} MB. The
limit is 20 MB.`);
    if (!file.size) throw new InputError("The file is empty.");
    const ext = (file.name.split(".").pop() || "").toLowerCase();
    const buf = await file.arrayBuffer();
    const isDicom = ["dcm", "dicom"].includes(ext) || (buf.byteLength > 132 && new TextDecoder().decode(new
Uint8Array(buf, 128, 4)) === "DICM");
    let img, meta = {};
    if (isDicom) {
      await loadScript("vendor/dicom-parser/dicomParser.min.js");
      ({ image: img, meta } = decodeDicom(buf, window.dicomParser));
    } else {
      if (ext && !["png", "jpg", "jpeg", "bmp", "webp"].includes(ext)) throw new InputError(`'.${ext}' files are
not supported. Use PNG, JPEG or DICOM.`);
      try { img = await decodeBlob(new Blob([buf], { type: file.type })); }
      catch { throw new InputError("This file is not a readable image, or it is truncated."); }
      if (img.w * img.h > 50_000_000) throw new InputError("The image has too many pixels to process safely.");
    }
    const warnings = assess(img);
    if (token !== state.token) return;
    Object.assign(state, { img, meta, warnings, source: file.name, boxed: letterbox(img, CONFIG.imageSize),
result: null });
    renderPending();
    await ensureModel();
    const [r] = await infer([img]);
    if (token !== state.token) return;
    state.result = { ...r, source: "browser" };
    render();
    runLiveStress(token);
```

```
    } catch (e) {
      if (token !== state.token) return;
      $("#error").innerHTML = note(e instanceof InputError ? `<b>Can't analyse this file.</b> ${esc(e.message)}`
        : `<b>Something went wrong.</b> ${esc(e.message)}`, "warn");
    }
  }

  function loadScript(src) {
    if (document.querySelector(`script[src="${src}"]`)) return Promise.resolve();
    return new Promise((ok, fail) => { const s = document.createElement("script"); s.src = src; s.onload = ok;
s.onerror = fail; document.head.append(s); });
  }

  // ── 02 output ──────────────────────────────────────────────────────────
  function renderPending() {
    imageToCanvas($("#input-canvas"), state.boxed);
    $("#readout").innerHTML = `<div class="readout"><div class="lab">Model output</div><div class="call"><span
class="spin"></span></div>
      <p class="fig">Waiting for the in-browser model. First visit downloads 94 MB of weights from Hugging Face;
later visits use the cached copy.</p></div>`;
    for (const id of ["#context", "#story", "#attr", "#attr-note", "#stress-head", "#stress-note", "#thumbs",
"#stress-chart", "#cam-cap", "#rand-body"]) $(id).innerHTML = "";
  }

  function render() {
    const { result: r, boxed, img } = state;
    imageToCanvas($("#input-canvas"), boxed);
    $("#input-cap").innerHTML = `<b>MODEL INPUT</b> · ${img.w}×${img.h} source → letterboxed to 320×320, black bars
added · ${esc(state.item ? state.item.title : state.source)}`;
    const p = r.pneumonia, sat = p < 1e-3 || p > 1 - 1e-3;
    $("#readout").innerHTML = `
      <div class="readout">
        <div class="lab">Model output · class with score ≥ threshold</div>
        <div class="call ${r.label.toLowerCase()}">${r.label === "PNEUMONIA" ? "Pneumonia" : "Normal"}</div>
        <div class="track" role="img" aria-label="Pneumonia score ${p.toFixed(4)} against threshold 0.5">
          <div class="bar"></div><div class="thr" style="left:50%"><span>threshold 0.5</span></div><div class="pin"
style="left:${(p * 100).toFixed(2)}%"></div>
        </div>
        <div class="axis"><span>0 · NORMAL</span><span>PNEUMONIA · 1</span></div>
        <dl class="kv">
          <dt>PNEUMONIA score</dt><dd>${p.toFixed(6)}</dd>
          <dt>NORMAL score</dt><dd>${r.scores.NORMAL.toFixed(6)}</dd>
          <dt>Logit margin (z₁ - z₀)</dt><dd>${(r.logits[1] - r.logits[0] ≥ 0 ? "+" : "") + (r.logits[1] -
r.logits[0]).toFixed(2)}</dd>
          <dt>Saturated (&lt;0.001 or &gt;0.999)</dt><dd>${sat ? "yes" : "no"}</dd>
          <dt>Computed by</dt><dd id="live-line">${r.source === "browser" ? "your browser (ONNX Runtime Web)" :
"PyTorch (precomputed) · live check pending"}</dd>
        </dl>
      </div>`;
    const shareN = rsna.filter((x) => x.finding === "Normal" && x.score ≥ p).length / 200;
    const shareO = rsna.filter((x) => x.finding === "Lung Opacity" && x.score ≥ p).length / 200;
    $("#context").innerHTML = note(`<b>What the score is, and what it isn't.</b> ${p.toFixed(4)} is the network's
output for this image, not the probability that a patient has pneumonia. For context, from our own external test:
<b>${pct(shareN, 0)}</b> of 200 adult radiographs read as <i>normal</i> (RSNA) scored at least this high, as did
<b>${pct(shareO, 0)}</b> of 200 with lung opacity.`);
    let story = "";
    if (state.item) {
      story += `<div style="margin-top:.6rem">${state.item.tags.map(([t, c]) => `<span class="tag"
style="color:${c}">${esc(t)}</span>`).join("")}</div>`;
      story += note(state.item.story, state.item.seen_in_training ? "warn" : "cool");
    }
    for (const w of state.warnings) story += note(`<b>Input check.</b> ${esc(w)}`, "warn");
    if (state.meta && Object.keys(state.meta).length) story += `<div class="fig">DICOM ·
${Object.entries(state.meta).map(([k, v]) => `${k} ${esc(v)}`).join(" · ")}</div>`;
    $("#story").innerHTML = story;
    renderLive();
    renderExplain();
    renderStress();
  }

```

```
function renderLive() {
  const el = $("#live-line");
  if (!el || state.kind !== "gallery") return;
  if (state.live) el.innerHTML = `PyTorch ${state.result.pneumonia.toFixed(6)} · your browser ${state.live.pneumonia.toFixed(6)} · |Δ| ${state.live.delta.toExponential(1)}`;
  else el.textContent = modelReady ? "PyTorch (precomputed) · checking live…" : "PyTorch (precomputed) · live check when the model loads";
}

// ── 03 explanation ──────────────────────────────────────────────────
let cls = "pred";
$("#alpha").addEventListener("input", (e) => { $("#alpha-val").textContent = Number(e.target.value).toFixed(2); renderExplain(); });
$("#cls").addEventListener("change", (e) => { cls = e.target.value; renderExplain(); });

function renderExplain() {
  const r = state.result;
  if (!r) return;
  $("#cls").innerHTML = [["pred", `Predicted: ${r.label}`], ["NORMAL", "NORMAL"], ["PNEUMONIA", "PNEUMONIA"]]
    .map(([v, t]) => `<label><input type="radio" name="cls" value="${v}"${cls === v ? " checked" : ""}> ${t}</label>`).join("");
  const name = cls === "pred" ? r.label : cls, idx = name === "NORMAL" ? 0 : 1;
  const cam = upsampleCam(r.cams[idx], r.camDims[0], r.camDims[1], 320, 320);
  const alpha = Number($("#alpha").value);
  imageToCanvas($("#cam-a"), state.boxed);
  overlayToCanvas($("#cam-b"), state.boxed, cam, alpha, state.lung);
  $("#cam-cap").innerHTML = `<b>B</b> · Grad-CAM for ${name}, layer4 (10×10 upsampled)${state.lung ? " · teal outline = radiologist lung mask" : ""}`;
  const s = attributionStats(cam, state.boxed, state.lung);
  const cell = (label, mass, area, flag) => [label, pct(mass, 0), `of heat · ${pct(area, 0)} of pixels · ×${(mass / area).toFixed(2)}`, flag && mass / area > 1.3];
  const rows = [cell("Black padding", s.padding, s.padding_area, true), cell("Outer 12% frame", s.border, s.border_area, true)];
  if (s.lungs !== undefined) rows.push(cell("Inside lungs", s.lungs, s.lungs_area, false));
  $("#attr").innerHTML = stats(rows, "one");
  $("#attr-note").innerHTML = s.padding + s.border > 0.5 ? note("<b>Most of this map sits outside the anatomy</b>, on the black bars or the image frame. Either there is little evidence for this class, or the model is using acquisition artefacts. Neither supports a clinical reading.", "warn") : "";
  renderRandomisation(name);
}

function renderRandomisation(name) {
  const pop = `Across ${randPop.n} evaluation images (60 RSNA, 60 Montgomery), <b>${pct(randPop.pass_rate, 0)}</b> pass this test (|ρ| &lt; ${randPop.pass_threshold} at every stage). Median |ρ|: fc ${randPop.median_abs.rho_fc.toFixed(2)}, layer4 ${randPop.median_abs.rho_layer4.toFixed(2)}, layer3 ${randPop.median_abs.rho_layer3.toFixed(2)}.`;
  const intro = `<p>Adebayo et al. (2018) showed that some saliency maps look the same even when a network's weights are randomised. Those maps act as edge detectors, not explanations. This test destroys the learned weights from the top down (fc → layer4 → layer3) and recomputes Grad-CAM. A faithful map should <b>lose its resemblance</b> (rank correlation near 0).</p>`;
  if (state.kind !== "gallery") {
    $("#rand-body").innerHTML = intro + note(`Re-initialising weights needs the full PyTorch model, so this check is precomputed for gallery cases rather than run on uploads. ${pop}`);
    return;
  }
  const stages = state.item.precomputed.randomisation[name];
  const base = upsampleCam(state.result.cams[name === "NORMAL" ? 0 : 1], 10, 10, 320, 320);
  const worst = Math.max(...stages.map((s) => Math.abs(s.spearman)));
  $("#rand-body").innerHTML = intro + `<div class="cols" style="grid-template-columns:repeat(4,minmax(0,1fr));gap:10px">
      <div><canvas class="frame" id="rand-0"></canvas><div class="fig">Trained weights</div></div>
      ${stages.map((s, i) => `<div><canvas class="frame" id="rand-${i + 1}"></canvas><div class="fig">+ random ${s.randomised} · ρ = ${s.spearman ≥ 0 ? "+" : ""}${s.spearman.toFixed(2)}</div></div>`).join("")}
    </div>` +
    (worst < 0.4 ? note(`<b>Passes.</b> Every randomised map is weakly correlated with the original (|ρ| ≤ ${worst.toFixed(2)}), so this heatmap depends on the learned weights. That is necessary but not sufficient: it can still faithfully show a shortcut.`, "cool")
                 : note(`<b>Caution.</b> A randomised model reproduces this map (|ρ| up to ${worst.toFixed(2)}). Some of what you see is image structure, not model reasoning.`, "warn")) +
    `<p class="fig">Computed in PyTorch (seed 0) and shipped with the page. ${pop}</p>`;
```

```
  overlayToCanvas($("#rand-0"), state.boxed, base, 0.6, null);
  stages.forEach((s, i) => overlayToCanvas($(`#rand-${i + 1}`), state.boxed,
upsampleCam(Float32Array.from(s.cam), 10, 10, 320, 320), 0.6, null));
}

// ── 04 stress test ──────────────────────────────────────────────────────
async function runLiveStress(token) {
  $("#stress-head").innerHTML = `<p class="fig"><span class="spin"></span> Scoring 9 images in your browser…
</p>`;
  await ensureModel();
  const ims = [state.img, ...PERTURBATIONS.map(([, , fn]) => fn(state.img))];
  const res = await infer(ims);
  if (token !== state.token) return;
  state.stressLive = Object.fromEntries([["original", res[0].pneumonia], ...PERTURBATIONS.map(([k], i) => [k,
res[i + 1].pneumonia])]);
  renderStress();
}

function renderStress() {
  const live = state.stressLive, pre = state.item?.precomputed.stress;
  const sc = live || pre;
  const thumbs = [["original", "Original", (x) => x], ...PERTURBATIONS.map(([k, l, fn]) => [k, l, fn])];
  if (!sc) { $("#stress-head").innerHTML = ""; return; }
  const base = sc.original, thr = CONFIG.threshold;
  const src = live ? "Scored live in your browser. Blur, noise and marker are browser re-implementations of the
Python edits; the other five are pixel-identical."
                   : "Scored with PyTorch using the exact edits from the evaluation.";
  $("#stress-head").innerHTML = `<p class="fig">${src}${!live && state.kind === "gallery" ? ' <button class="btn
inline" id="rerun" style="margin-left:.6rem">Re-run live in browser</button>' : ""}</p>`;
  $("#rerun")?.addEventListener("click", (e) => { e.target.disabled = true; runLiveStress(state.token); });
  const rows = PERTURBATIONS.map(([k, label]) => ({ edit: label, score: sc[k], baseline: base,
    label: sc[k] >= thr ? "PNEUMONIA" : "NORMAL", flipped: (sc[k] >= thr) !== (base >= thr) }));
  const order = rows.map((r) => r.edit);
  chart($("#stress-chart"), {
    height: 290, data: { values: rows },
    layer: [
      { data: { values: [{ t: thr }] }, mark: { type: "rule", strokeDash: [4, 4], color: COLORS.BONE, opacity:
0.5 }, encoding: { x: { field: "t", type: "quantitative" } } },
      { mark: { type: "rule", color: COLORS.MUTED, strokeWidth: 2 }, encoding: { y: { field: "edit", type:
"nominal", sort: order, title: null, axis: { labelLimit: 260 } }, x: { field: "baseline", type: "quantitative" }, x2: {
field: "score" } } },
      { mark: { type: "tick", color: COLORS.BONE, thickness: 2, size: 16 }, encoding: { y: { field: "edit", type:
"nominal", sort: order }, x: { field: "baseline", type: "quantitative" } } },
      { mark: { type: "circle", size: 140, stroke: COLORS.INK, strokeWidth: 2, opacity: 1 },
        encoding: { y: { field: "edit", type: "nominal", sort: order },
          x: { field: "score", type: "quantitative", scale: { domain: [0, 1] }, title: "PNEUMONIA score  (| =
original image)" },
          color: { field: "label", type: "nominal", scale: { domain: ["NORMAL", "PNEUMONIA"], range:
[COLORS.NORMAL, COLORS.PNEUMONIA] }, legend: { title: "Label after edit" } },
          tooltip: [{ field: "edit" }, { field: "score", format: ".5f" }, { field: "baseline", format: ".5f",
title: "original" }, { field: "flipped" }] } },
    ],
  });
  $("#thumbs").innerHTML = thumbs.map(([k, l], i) => `<div><canvas id="th-${i}"></canvas><div class="fig${k !==
"original" && (sc[k] >= thr) !== (base >= thr) ? " flip" : ""}"><b>${esc(l)}</b><br>${sc[k].toFixed(4)}${k !==
"original" && (sc[k] >= thr) !== (base >= thr) ? " ⟲ flipped" : ""}</div></div>`).join("");
  thumbs.forEach(([, , fn], i) => imageToCanvas($(`#th-${i}`), fn(state.img)));
  const flips = rows.filter((r) => r.flipped).length, mk = metrics.robustness.marker, inv =
metrics.robustness.invert;
  $("#stress-note").innerHTML = note(`<b>${flips ? `${flips} of 8 edits flip this image's label.` : "No edit
flips this image's label."}</b> Across ${mk.n} external adult radiographs, burned-in text markers flipped
${pct(mk.flip_rate)} of labels, ${mk.to_pneumonia} toward PNEUMONIA and ${mk.to_normal} toward NORMAL. Inverting
greyscale flipped ${pct(inv.flip_rate)}. Edits that only add artefacts push the model one way, toward PNEUMONIA.`);
}

renderSheet();
selectGallery(gallery[0].id);

```

22 web/js/reliability.js

```
// Reliability dashboard. Every metric is recomputed from data/scores.json in the browser.
import { $, esc, pct, ci, note, stats, getJSON, getScores, binaryMetrics, wilson, chart, COLORS } from "./common.js";

const [M, S, R, P] = await Promise.all([getJSON("data/metrics.json"), getScores(), getJSON("data/randomisation.json"),
  getJSON("data/web_parity.json").catch(() => null)]);
const NAMES = { kermany_test: "Kermany test", rsna: "RSNA", montgomery: "Montgomery", shenzhen: "Shenzhen" };
const k = M.cohorts.kermany_test, r = M.cohorts.rsna_normal_vs_opacity;

$("#kick").textContent = `Reliability report · ${S.length.toLocaleString()} images · 4 cohorts · every number reproducible`;
$("#hero").innerHTML = `
  <div><div class="kicker" style="color:var(--muted)">Kermany “test” folder · n = ${k.n}</div>
    <div class="n"><span class="strike">${k.auc.toFixed(3)}</span> <small>AUC</small></div>
    <div class="cap">Sensitivity ${pct(k.sensitivity)} · specificity ${pct(k.specificity)}. Not a valid test: the published weights were refit on these exact images.</div></div>
  <div><div class="kicker">RSNA adults · external · n = ${r.n}</div>
    <div class="n" style="color:var(--pneumonia)">${pct(r.specificity)}<small> specificity</small></div>
    <div class="cap">${ci(r.specificity_ci)}. Normal adult radiographs that the model correctly calls NORMAL at the shipped 0.5 threshold. AUC ${r.auc.toFixed(3)} (${r.auc_ci[0].toFixed(3)}–${r.auc_ci[1].toFixed(3)}).</div></div>`;
$("#sat").innerHTML = stats([
  ["Kermany saturated", pct(M.saturation.kermany_test, 0), "score < 0.001 or > 0.999"],
  ["RSNA saturated", pct(M.saturation.rsna, 0), "same definition"],
  ["RSNA calibration error", M.calibration.rsna_normal_vs_opacity.ece.toFixed(2), "ECE, 10 bins"],
]);

// ── 02 threshold explorer ──────────────────────────────────────────
const ops = M.operating_points || {};
const choices = [["0.5 (shipped)", 0.5], ["0.9", 0.9], ["0.99", 0.99], ["0.999", 0.999], ["0.9999", 0.9999],
  ...Object.entries(ops).map(([n, o]) => [`spec ≥ ${n.split("_").pop()}% on RSNA (${o.threshold.toFixed(7)})`, o.threshold])];
const thrEl = $("#thr");
thrEl.max = choices.length - 1;
const rsMain = S.filter((x) => x.cohort === "rsna" && (x.finding === "Normal" || x.finding === "Lung Opacity"));
const y = rsMain.map((x) => x.label), sc = rsMain.map((x) => x.score);

const groups = [...new Set(S.filter((x) => !["Pneumonia", "Lung Opacity"].includes(x.finding)).map((x) => `${x.cohort}|${x.finding}`))];
const groupLabel = (g) => { const [c, f] = g.split("|"); return `${NAMES[c]} · ${f === "No Lung Opacity / Not Normal" ? "other abnormality" : f}`; };

const rocView = chart($("#roc"), {
  height: 280, title: `ROC, RSNA · AUC ${r.auc.toFixed(3)}`,
  layer: [
    { data: { values: [{ x: 0, y: 0 }, { x: 1, y: 1 }] }, mark: { type: "line", strokeDash: [4, 4], color: COLORS.MUTED, strokeWidth: 1 },
      encoding: { x: { field: "x", type: "quantitative", title: "1 - specificity" }, y: { field: "y", type: "quantitative", title: "sensitivity" } } },
    { data: { values: M.roc.rsna_normal_vs_opacity.fpr.map((f, i) => ({ fpr: f, tpr: M.roc.rsna_normal_vs_opacity.tpr[i] })) },
      mark: { type: "line", color: COLORS.BONE, strokeWidth: 2, interpolate: "step-after" },
      encoding: { x: { field: "fpr", type: "quantitative" }, y: { field: "tpr", type: "quantitative" } } },
    { data: { name: "op" }, mark: { type: "point", size: 180, filled: true, color: COLORS.AMBER, stroke: COLORS.INK, strokeWidth: 2 },
      encoding: { x: { field: "fpr", type: "quantitative" }, y: { field: "tpr", type: "quantitative" },
        tooltip: [{ field: "label" }, { field: "fpr", format: ".3f" }, { field: "tpr", format: ".3f" }] } },
  ],
});

$("#roc-note").innerHTML = note(`<b>Good ranking, bad threshold.</b> An AUC of ${r.auc.toFixed(3)} means the model usually ranks an opacity image above a normal one. The shipped 0.5 cut-off just sits in the wrong place for adults. ` +
  Object.values(ops).map((o) => `For ${(o.specificity * 100).toFixed(1)}% specificity you need a threshold of ${o.threshold.toFixed(7)}, which keeps ${(o.sensitivity * 100).toFixed(1)}% sensitivity. `).join("") +
  "Choosing that threshold on the same 400 images we report on is optimistic. A real deployment would fix it on separate data.");
```

```
  52
  53  const logit = (s) => { const c = Math.min(Math.max(s, 1e-7), 1 - 1e-7); return Math.log(c / (1 - c)); };
  54  // Seeded jitter so dots don't move between renders.
  55  let seed = 7; const rnd = () => ((seed = (seed * 16807) % 2147483647) / 2147483647);
  56  const stripRows = S.map((x) => ({ group: `${NAMES[x.cohort]} · ${x.finding === "No Lung Opacity / Not Normal" ?
"Other abnormality" : x.finding}`,
  57    expected: ["Pneumonia", "Lung Opacity"].includes(x.finding) ? "Pneumonia / opacity" : x.finding === "Normal" ?
"Normal" : "Out of scope",
  58    logit: logit(x.score), score: x.score, id: x.source_id, j: rnd() }));
  59  const groupOrder = [...new Set(stripRows.map((d) => d.group))].sort((a, b) => {
  60    const e = (g) => ({ Normal: 0, "Out of scope": 1, "Pneumonia / opacity": 2 }[stripRows.find((d) => d.group ===
g).expected]);
  61    return e(a) - e(b) || a.localeCompare(b);
  62  });
  63  const stripView = chart($("#strip"), {
  64    height: 480, title: "Every image's score, by cohort (amber = threshold)",
  65    layer: [
  66      { data: { values: stripRows }, mark: { type: "circle", size: 14, opacity: 0.6 },
  67        encoding: { x: { field: "logit", type: "quantitative", scale: { domain: [-17, 17] }, title: "model logit of
PNEUMONIA score (0 = score 0.5; ±16 ≈ fully saturated)" },
  68          y: { field: "group", type: "nominal", sort: groupOrder, title: null, axis: { labelLimit: 240 } },
yOffset: { field: "j", type: "quantitative" },
  69          color: { field: "expected", type: "nominal", scale: { domain: ["Normal", "Pneumonia / opacity", "Out of
scope"], range: [COLORS.NORMAL, COLORS.PNEUMONIA, COLORS.OTHER] }, legend: { title: null } },
  70          tooltip: [{ field: "group" }, { field: "score", format: ".6f" }, { field: "id", title: "source id" }] }
},
  71      { data: { name: "thr" }, mark: { type: "rule", color: COLORS.AMBER, strokeDash: [4, 3] }, encoding: { x: {
field: "t", type: "quantitative" } } },
  72    ],
  73  });
  74  $("#strip-note").innerHTML = note("<b>Read the strip chart left to right.</b> Normal adult radiographs (teal)
stretch across the whole range and many sit far right, fully saturated as PNEUMONIA. No threshold separates them
cleanly from lung opacity, and the 0.5 threshold falls inside the normal cluster.");
  75
  76  let cmView = null, flagView = null;
  77  async function updateThreshold() {
  78    const [label, t] = choices[Number(thrEl.value)];
  79    $("#thr-val").textContent = label;
  80    const m = binaryMetrics(y, sc, t);
  81    $("#thr-stats").innerHTML = stats([
  82      ["Sensitivity", pct(m.sensitivity), ci(m.sensitivity_ci)], ["Specificity", pct(m.specificity),
ci(m.specificity_ci)],
  83      ["Precision", pct(m.precision), ci(m.precision_ci)], ["F1", m.f1.toFixed(3), "harmonic mean of precision and
sensitivity"],
  84    ]);
  85    $("#cm-cap").innerHTML = `<b>RSNA</b> · 200 normal vs 200 lung-opacity adults · threshold ${t}`;
  86    const cm = [
  87      { true: "Lung opacity", pred: "PNEUMONIA", n: m.tp, kind: "correct" }, { true: "Lung opacity", pred:
"NORMAL", n: m.fn, kind: "missed" },
  88      { true: "Normal", pred: "PNEUMONIA", n: m.fp, kind: "false alarm" }, { true: "Normal", pred: "NORMAL", n:
m.tn, kind: "correct" },
  89    ];
  90    const flags = groups.map((g) => {
  91      const [c, f] = g.split("|"), s = S.filter((x) => x.cohort === c && x.finding === f).map((x) => x.score);
  92      const kk = s.filter((v) => v >= t).length, [lo, hi] = wilson(kk, s.length);
  93      return { group: groupLabel(g), rate: kk / s.length, lo, hi, n: s.length, kind: f === "Normal" ? "Normal" :
"Out of scope" };
  94    }).sort((a, b) => b.rate - a.rate);
  95    if (!cmView) {
  96      cmView = await chart($("#cm"), {
  97        height: 240, data: { name: "cm" },
  98        encoding: { x: { field: "pred", type: "nominal", sort: ["NORMAL", "PNEUMONIA"], title: "Model says", axis:
{ orient: "top", labelAngle: 0 } },
  99                    y: { field: "true", type: "nominal", sort: ["Normal", "Lung opacity"], title: "RSNA label" } },
 100        layer: [
 101          { mark: { type: "rect", stroke: COLORS.INK, strokeWidth: 3 }, encoding: { color: { field: "n", type:
"quantitative", scale: { range: ["#141c21", "#5c6b70"], domain: [0, 200] }, legend: null }, tooltip: [{ field: "true"
}, { field: "pred" }, { field: "n" }, { field: "kind" }] } },
 102          { mark: { type: "text", fontSize: 26, font: "Instrument Serif", color: COLORS.BONE }, encoding: { text: {
field: "n" } } },
```

```
          { mark: { type: "text", dy: 22, fontSize: 10, color: COLORS.MUTED }, encoding: { text: { field: "kind" } } },
        ],
      });
      flagView = await chart($("#flags"), {
        height: { step: 30 }, title: "False-alarm rate where no pneumonia is labelled", data: { name: "flags" },
        encoding: { y: { field: "group", type: "nominal", sort: { field: "rate", order: "descending" }, title: null, axis: { labelLimit: 260 } } },
        layer: [
          { mark: { type: "bar", height: 14, cornerRadiusEnd: 4 }, encoding: {
            x: { field: "rate", type: "quantitative", scale: { domain: [0, 1] }, axis: { format: "%" }, title: "share called PNEUMONIA (no pneumonia label)" },
            color: { field: "kind", type: "nominal", scale: { domain: ["Normal", "Out of scope"], range: [COLORS.NORMAL, COLORS.OTHER] }, legend: { title: null } },
            tooltip: [{ field: "group" }, { field: "rate", format: ".1%" }, { field: "lo", format: ".1%", title: "95% CI low" }, { field: "hi", format: ".1%", title: "95% CI high" }, { field: "n" }] } },
          { mark: { type: "rule", color: COLORS.BONE, opacity: 0.6 }, encoding: { x: { field: "lo", type: "quantitative" }, x2: { field: "hi" } } },
        ],
      });
    }
    cmView.data("cm", cm).run();
    flagView.data("flags", flags).run();
    (await rocView).data("op", [{ fpr: 1 - m.specificity, tpr: m.sensitivity, label: `threshold ${t}` }]).run();
    (await stripView).data("thr", [{ t: logit(t) }]).run();
  }
  thrEl.addEventListener("input", updateThreshold);
  updateThreshold();

  // ── 03 calibration ──────────────────────────────────────────────────
  const cal = M.calibration.rsna_normal_vs_opacity, top = cal.bins[cal.bins.length - 1];
  chart($("#cal"), {
    height: 290,
    layer: [
      { data: { values: [{ x: 0, y: 0 }, { x: 1, y: 1 }] }, mark: { type: "line", strokeDash: [4, 4], color: COLORS.MUTED },
        encoding: { x: { field: "x", type: "quantitative", title: "mean model score in bin" }, y: { field: "y", type: "quantitative", title: "observed share with lung opacity" } } },
      { data: { values: cal.bins }, mark: { type: "circle", color: COLORS.AMBER, opacity: 1, stroke: COLORS.INK, strokeWidth: 2 },
        encoding: { x: { field: "mean_score", type: "quantitative" }, y: { field: "observed_rate", type: "quantitative" },
          size: { field: "n", type: "quantitative", scale: { range: [60, 900] }, legend: null },
          tooltip: [{ field: "lo", format: ".1f", title: "bin from" }, { field: "hi", format: ".1f", title: "bin to" }, { field: "n", title: "images" }, { field: "mean_score", format: ".3f" }, { field: "observed_rate", format: ".3f" }] } },
    ],
  });
  $("#cal-stats").innerHTML = stats([["ECE", cal.ece.toFixed(3), "0 = perfectly calibrated"],
    ["Top bin", `${(top.observed_rate * 100).toFixed(0)}%`, `observed, at mean score ${top.mean_score.toFixed(3)} (n=${top.n})`]]);

  // ── 04 heatmaps ─────────────────────────────────────────────────────
  const att = M.attribution.montgomery;
  $("#lung-stats").innerHTML = stats([
    ["Heat inside lungs", pct(att.lungs_median, 0), `median · IQR ${pct(att.lungs_iqr[0], 0)}–${pct(att.lungs_iqr[1], 0)}`],
    ["Lungs' share of pixels", pct(att.lungs_area_median, 0), "median"],
    ["Enrichment", `×${att.lungs_enrichment_median.toFixed(2)}`, `${pct(att.lungs_enrichment_gt1, 0)} of images above ×1`],
  ], "one");
  chart($("#lung-hist"), {
    height: 220, title: "Lung enrichment per Montgomery image (×1 = no more than a uniform map)",
    layer: [
      { data: { values: S.filter((x) => x.cohort === "montgomery").map((x) => ({ e: x.cam_lungs / x.cam_lungs_area })) },
        mark: { type: "bar", color: COLORS.AMBER, cornerRadiusEnd: 4, binSpacing: 2 },
        encoding: { x: { field: "e", type: "quantitative", bin: { step: 0.25 }, title: "heat share ÷ area share" }, y: { aggregate: "count", type: "quantitative", title: "images" }, tooltip: [{ aggregate: "count", title: "images" }] } },
```

```
      { data: { values: [{ x: 1 }] }, mark: { type: "rule", color: COLORS.BONE, strokeDash: [4, 4] }, encoding: { x: { field: "x", type: "quantitative" } } },
    ],
  });
  $("#rand-stats").innerHTML = stats([
    ["Randomisation test", pct(R.pass_rate, 0), `of ${R.n} images pass (|ρ| < ${R.pass_threshold} at every stage)`],
    ["Median |ρ| after random fc", R.median_abs.rho_fc.toFixed(2), "rank correlation with the trained map"],
    ["… + random layer4, layer3", `${R.median_abs.rho_layer4.toFixed(2)} · ${R.median_abs.rho_layer3.toFixed(2)}`, "cumulative re-initialisation"],
  ], "one");
  chart($("#rand-chart"), {
    height: 220, title: "Largest |ρ| per image across the three randomisation stages",
    layer: [
      { data: { values: R.rows }, mark: { type: "bar", color: COLORS.OTHER, cornerRadiusEnd: 4, binSpacing: 2 },
        encoding: { x: { field: "max_abs", type: "quantitative", bin: { step: 0.05 }, title: "max |Spearman ρ| vs trained map (lower is better)" },
          y: { aggregate: "count", type: "quantitative", title: "images" }, tooltip: [{ aggregate: "count", title: "images" }] } },
      { data: { values: [{ x: R.pass_threshold }] }, mark: { type: "rule", color: COLORS.BONE, strokeDash: [4, 4] }, encoding: { x: { field: "x", type: "quantitative" } } },
    ],
  });
  $("#att-note").innerHTML = note(`<b>What this does and doesn't show.</b> Enrichment above ×1 means the explanation favours lung tissue beyond chance, and a passing randomisation test means the map depends on the learned weights. The ${pct(1 - R.pass_rate, 0)} of images that fail keep some resemblance after the weights are destroyed, so part of their map reflects image structure. Neither check proves the model reasons clinically. Grad-CAM is a coarse 10×10 summary of one layer.`);

  // ── 05 robustness ──────────────────────────────────────────────────────
  const rob = Object.values(M.robustness).sort((a, b) => b.flip_rate - a.flip_rate);
  $("#rob-sub").textContent = `Each of ${M.robustness.marker.n} external adult radiographs was re-scored after eight edits that do not change the anatomy.`;
  chart($("#rob"), {
    height: { step: 30 },
    data: { values: rob.flatMap((v) => [{ edit: v.label, direction: "to PNEUMONIA", images: v.to_pneumonia, flip: v.flip_rate },
                                        { edit: v.label, direction: "to NORMAL", images: v.to_normal, flip: v.flip_rate }]) },
    mark: { type: "bar", height: 16, cornerRadiusEnd: 3, stroke: COLORS.INK, strokeWidth: 2 },
    encoding: { y: { field: "edit", type: "nominal", sort: rob.map((v) => v.label), title: null, axis: { labelLimit: 260 } },
      x: { field: "images", type: "quantitative", stack: "zero", title: "labels flipped" },
      color: { field: "direction", type: "nominal", scale: { domain: ["to NORMAL", "to PNEUMONIA"], range: [COLORS.NORMAL, COLORS.PNEUMONIA] }, legend: { title: "Flipped" } },
      order: { field: "direction", sort: "descending" },
      tooltip: [{ field: "edit" }, { field: "direction" }, { field: "images" }, { field: "flip", format: ".1%", title: "total flip rate" }] },
  });
  const mk = M.robustness.marker;
  $("#rob-note").innerHTML = note(`<b>The asymmetry is the finding.</b> Adding a text marker, noise or inversion moves labels almost only toward PNEUMONIA (markers: ${mk.to_pneumonia} toward PNEUMONIA, ${mk.to_normal} toward NORMAL). The model treats "looks unusual" as evidence of disease. One plausible cause, which we have not verified, is that pneumonia images in the training set differ systematically in acquisition (portable films, more annotations, lower contrast). Either way it is a shortcut, not radiology.`, "warn");

  // ── 06 browser parity ──────────────────────────────────────────────────
  if (P) {
    const exact = Object.entries(P.perturbations_identical);
    $("#parity").innerHTML = stats([
      ["Model input identical", `${P.letterbox_identical}/${P.images}`, "letterboxed 320×320 pixels, byte for byte"],
      ["Labels identical", `${P.labels_identical}/${P.images}`, "NORMAL / PNEUMONIA at 0.5"],
      ["Max score difference", P.max_abs_score_diff.toExponential(1), `median ${P.median_abs_score_diff.toExponential(1)}`],
      ["Max Grad-CAM difference", P.max_abs_cam_diff.toExponential(1), "on a 0–1 normalised map"],
    ]) + note(`Checked on ${P.images} evaluation images (a third re-encoded as JPEG) in headless Chrome by <code>scripts/check_web_parity.py</code>. Preprocessing is a line-by-line port of Pillow's resampling. ${exact.map(([k2, v]) => `${k2} ${v}/${P.images}`).join(", ")} perturbed images were byte-identical too. Input-check warnings matched on ${P.warnings_identical}/${P.images}. Remaining score differences are floating-point noise from a
```

```
different maths library.`, "cool");
 200  }
 201
 202  // ── tables ──────────────────────────────────────────────────────────
 203  const tbl = (head, rows) => `<table><tr>${head.map((h) => `<th>${h}</th>`).join("")}</tr>${rows.map((rw) =>
`<tr>${rw.map((c) => `<td>${c}</td>`).join("")}</tr>`).join("")}</table>`;
 204  $("#tables").innerHTML =
 205    tbl(["Cohort", "n", "Valid test", "Sensitivity", "Specificity", "Precision", "F1", "AUC"],
 206        Object.entries(M.cohorts).map(([n, v]) => [esc(n), v.n, v.valid_test ? "yes" : "<b>no</b>",
pct(v.sensitivity), pct(v.specificity), pct(v.precision), v.f1.toFixed(3), v.auc.toFixed(3)])) +
 207    `<br>` + tbl(["Group (no pneumonia label)", "n", "Called PNEUMONIA", "Rate", "95% CI"],
 208        Object.entries(M.flag_rates).map(([g, v]) => [esc(g), v.n, v.flagged, pct(v.rate), ci(v.rate_ci)])) +
 209    `<br>` + tbl(["Edit", "n", "→ PNEUMONIA", "→ NORMAL", "Flip rate"], rob.map((v) => [esc(v.label), v.n,
v.to_pneumonia, v.to_normal, pct(v.flip_rate)])) +
 210    `<p><a href="data/scores.json" download>Download per-image scores (JSON)</a></p>`;
 211  $("#foot").innerHTML = `Model revision ${M.model.revision.slice(0, 12)} · evaluated with torch
${M.environment.torch} · reproduce with <code>python scripts/fetch_eval_data.py && python
scripts/run_evaluation.py</code>`;
 212
```

23 web/js/common.js

```
// Small shared helpers: escaping, formatting, notes, stats, metrics (mirror xraylens/evaluation.py).

export const $ = (sel, root = document) => root.querySelector(sel);
export const esc = (s) => String(s).replace(/[&<>"']/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[c]));
export const pct = (x, d = 1) => (Number.isFinite(x) ? `${(x * 100).toFixed(d)}%` : "—");
export const ci = ([lo, hi], d = 1) => (Number.isFinite(lo) ? `95% CI ${(lo * 100).toFixed(d)}–${(hi * 100).toFixed(d)}%` : "");
export const note = (html, kind = "") => `<div class="note ${kind}"><p>${html}</p></div>`;
export const stats = (items, cls = "") =>
  `<div class="stats ${cls}">${items.map(([k, v, c, hot]) => `<div><div class="k">${k}</div><div class="v${hot ? " hot" : ""}">${v}</div><div class="ci">${c || ""}</div></div>`).join("")}</div>`;

export async function getJSON(path) {
  const r = await fetch(path);
  if (!r.ok) throw new Error(`${path}: ${r.status}`);
  return r.json();
}

export async function getScores() {
  const { columns, rows } = await getJSON("data/scores.json");
  return rows.map((r) => Object.fromEntries(columns.map((c, i) => [c, r[i]])));
}

export function wilson(k, n, z = 1.96) {
  if (!n) return [NaN, NaN];
  const p = k / n, d = 1 + (z * z) / n, c = (p + (z * z) / (2 * n)) / d;
  const h = (z * Math.sqrt((p * (1 - p)) / n + (z * z) / (4 * n * n))) / d;
  return [Math.max(0, c - h), Math.min(1, c + h)];
}

export function binaryMetrics(y, s, t) {
  let tp = 0, fn = 0, fp = 0, tn = 0;
  y.forEach((yi, i) => { const p = s[i] >= t; if (yi && p) tp++; else if (yi) fn++; else if (p) fp++; else tn++; });
  const sens = tp / (tp + fn), spec = tn / (tn + fp), prec = tp + fp ? tp / (tp + fp) : NaN;
  return { tp, fn, fp, tn, sensitivity: sens, specificity: spec, precision: prec,
           f1: (2 * prec * sens) / (prec + sens), sensitivity_ci: wilson(tp, tp + fn),
           specificity_ci: wilson(tn, tn + fp), precision_ci: wilson(tp, tp + fp) };
}

export const VEGA_OPTS = { actions: false, renderer: "svg", config: {
  background: null, font: "JetBrains Mono", view: { stroke: null },
  axis: { labelColor: "#93a1a6", titleColor: "#93a1a6", gridColor: "rgba(236,230,214,.08)", domainColor: "rgba(236,230,214,.3)",
          tickColor: "rgba(236,230,214,.3)", labelFont: "JetBrains Mono", titleFont: "JetBrains Mono", titleFontWeight: 400,
          labelFontSize: 10, titleFontSize: 10 },
  legend: { labelColor: "#ece6d6", titleColor: "#93a1a6", labelFont: "JetBrains Mono", titleFont: "JetBrains Mono",
            titleFontWeight: 400, orient: "top" },
  title: { color: "#ece6d6", font: "Newsreader", fontWeight: 400, fontSize: 15, anchor: "start" },
  text: { color: "#ece6d6", font: "JetBrains Mono" },
} };

export const COLORS = { NORMAL: "#38a596", PNEUMONIA: "#e05c35", OTHER: "#8b7ff0", AMBER: "#ffb547", BONE: "#ece6d6", MUTED: "#93a1a6", INK: "#0b0f12" };

export async function chart(el, spec) {
  // vega-embed is loaded as a classic script (pinned, see the page <head>).
  // Wait for web fonts so Vega measures axis labels in JetBrains Mono, not the fallback.
  await document.fonts.ready;
  const full = { $schema: "https://vega.github.io/schema/vega-lite/v5.json", width: "container", autosize: { type: "fit-x", contains: "padding" }, ...spec };
  const res = await window.vegaEmbed(el, full, VEGA_OPTS);
  return res.view;
```

```
}

```

24 web/index.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>XRayLens AI</title>
<meta name="description" content="An explainable-AI laboratory for chest radiographs: a real ResNet-50 pneumonia classifier running in your browser, with Grad-CAM, stress tests and an honest evaluation.">
<link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 32 32'%3E%3Crect width='32' height='32' fill='%230b0f12'/%3E%3Ccircle cx='16' cy='16' r='9' fill='none' stroke='%23ffb547' stroke-width='3'/%3E%3Cpath d='M16 7a9 9 0 0 1 0 18z' fill='%23ffb547'/%3E%3C/svg%3E">
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Instrument+Serif:ital@0;1&family=Newsreader:ital,opsz,wght@0,6..72,300..700;1,6..72,300..700&family=JetBrains+Mono:wght@400;600&display=swap" rel="stylesheet">
<link rel="stylesheet" href="css/site.css">
<script src="https://cdn.jsdelivr.net/npm/vega@5.30.0/build/vega.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/vega-lite@5.21.0/build/vega-lite.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/vega-embed@6.26.0/build/vega-embed.min.js"></script>
</head>
<body>
<nav class="nav" aria-label="Main">
  <a class="brand" href="./">XRayLens</a>
  <a href="./" aria-current="page">Examine</a>
  <a href="reliability.html">Reliability</a>
  <a href="provenance.html">Provenance</a>
  <div class="status" id="model-status" aria-live="polite">Live model · <b>not loaded</b><span class="bar"><i></i></span></div>
</nav>
<div class="runhead" aria-hidden="true">XRAYLENS · RESEARCH PROTOTYPE · NOT A DIAGNOSTIC DEVICE</div>

<main class="wrap">
  <header class="mast">
    <div class="kicker">An explainable-AI laboratory for chest radiographs</div>
    <h1 class="title">XRay<em>Lens</em></h1>
    <p class="dek">A pneumonia classifier gives you an answer. This lab shows how it got there, how far you can trust it, and where it breaks.</p>
    <div class="stamp">Research &amp;amp; education only<br>Not for diagnosis</div>
  </header>

  <section aria-labelledby="s1">
    <div class="sec"><div class="num">01</div><h2 id="s1">Choose a radiograph</h2>
      <p class="sub">Start with the curated contact sheet. Each case was picked from the evaluation to teach one thing. You can also upload a public, de-identified research image. It is analysed on your device and never uploaded anywhere.</p></div>
    <div class="cols c-8-3">
      <div class="sheet" id="sheet"></div>
      <div>
        <span class="label">Upload an image</span>
        <label class="drop" id="drop">
          <input type="file" id="file" accept=".png,.jpg,.jpeg,.bmp,.webp,.dcm,.dicom,image/png,image/jpeg,image/bmp,image/webp,application/dicom">
          <span class="big">↥ Choose or drop a file</span>
          <small>PNG, JPEG, BMP, WebP or uncompressed DICOM · up to 20 MB · processed in your browser</small>
        </label>
        <p class="fig">Use only public, de-identified research images. Do not upload real patient records. DICOM tags other than modality and view are ignored.</p>
      </div>
    </div>
  </section>

  <div id="error"></div>

  <section aria-labelledby="s2">
    <div class="sec"><div class="num">02</div><h2 id="s2">Read the model’s output</h2>
      <p class="sub">The model returns two softmax scores that sum to 1. A fixed threshold of 0.5 turns them into
```

```
a label.</p></div>
      <div class="cols c-5-6">
        <div><canvas class="frame" id="input-canvas" width="320" height="320" role="img" aria-label="Model input, letterboxed to 320 by 320 pixels"></canvas><div class="fig" id="input-cap"></div></div>
        <div><div id="readout"></div><div id="context"></div></div>
      </div>
      <div id="story"></div>
    </section>

    <section aria-labelledby="s3">
      <div class="sec"><div class="num">03</div><h2 id="s3">See where the model looked</h2>
        <p class="sub">Grad-CAM weights the last convolutional block’s feature maps by how strongly they push the chosen class’s logit up. Bright regions influenced the output. That does not make them correct.</p></div>
      <div class="cols c-3-2" style="margin-bottom:1rem">
        <div><span class="label">Explain which class</span><div class="radio" id="cls"></div></div>
        <div><span class="label">Heatmap opacity · <span class="range-val" id="alpha-val">0.50</span></span><input type="range" id="alpha" min="0" max="0.9" step="0.05" value="0.5" aria-label="Heatmap opacity"></div>
      </div>
      <div class="cols c-4-4-3">
        <div><canvas class="frame" id="cam-a"></canvas><div class="fig"><b>A</b> · input as seen by the model</div></div>
        <div><canvas class="frame" id="cam-b" role="img" aria-label="Grad-CAM heatmap overlay"></canvas><div class="fig" id="cam-cap"></div></div>
        <div><div id="attr"></div><p class="fig">×1.00 is what a uniform, uninformative map would score. Heat on padding or the frame (text markers, tubes, collimator edges) is a shortcut warning.</p></div>
      </div>
      <div id="attr-note"></div>
      <details id="rand">
        <summary>Sanity check: does this map depend on what the model learned?</summary>
        <div class="inner" id="rand-body"></div>
      </details>
    </section>

    <section aria-labelledby="s4">
      <div class="sec"><div class="num">04</div><h2 id="s4">Stress-test this image</h2>
        <p class="sub">Eight edits that do not change the anatomy a radiologist would read. A trustworthy model’s score should hardly move.</p></div>
      <div id="stress-head"></div>
      <div class="chart" id="stress-chart"></div>
      <div class="thumbs" id="thumbs"></div>
      <div id="stress-note"></div>
    </section>

    <p style="margin-top:2rem"><a href="reliability.html">Next: how reliable is this model across 1,462 images? →</a></p>
    <footer class="foot">XRayLens AI · Model: <a href="https://huggingface.co/Purino/resnet50-chest-xray-pneumonia">Purino/resnet50-chest-xray-pneumonia</a> (research only), weights fetched from Hugging Face and run locally with ONNX Runtime Web · Gallery images: Kermany et al. 2018 (CC BY 4.0) and the NLM Montgomery/Shenzhen sets · Outputs are not medical advice.</footer>
  </main>
  <script type="module" src="js/examine.js"></script>
  </body>
  </html>
```

25 web/vercel.json

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "cleanUrls": true,
  "headers": [
    {
      "source": "/(.*)",
      "headers": [
        { "key": "Cross-Origin-Opener-Policy", "value": "same-origin" },
        { "key": "Cross-Origin-Embedder-Policy", "value": "credentialless" },
        { "key": "X-Content-Type-Options", "value": "nosniff" },
        { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" },
        { "key": "Permissions-Policy", "value": "camera=(), microphone=(), geolocation=()" }
      ]
    },
    {
      "source": "/vendor/(.*)",
      "headers": [{ "key": "Cache-Control", "value": "public, max-age=31536000, immutable" }]
    }
  ]
}

```

26 tests/conftest.py

```python
import sys
from pathlib import Path

import numpy as np
import pytest
from PIL import Image

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))


@pytest.fixture
def cxr_like():
    """Synthetic 'radiograph': dark lungs either side of a bright mediastinum, wider than tall."""
    h, w = 300, 400
    y, x = np.mgrid[0:h, 0:w]
    arr = np.full((h, w), 40, np.float32)
    body = ((x - w / 2) / (w * .45)) ** 2 + ((y - h * .55) / (h * .5)) ** 2 < 1
    arr[body] = 170
    for cx in (w * .32, w * .68):
        lung = ((x - cx) / (w * .13)) ** 2 + ((y - h * .5) / (h * .32)) ** 2 < 1
        arr[lung] = 70
    arr[:, int(w * .46):int(w * .54)][body[:, int(w * .46):int(w * .54)]] = 220
    arr += np.random.default_rng(0).normal(0, 6, arr.shape)
    return Image.fromarray(arr.clip(0, 255).astype(np.uint8), "L")

```

27 tests/test_preprocessing.py

```python
"""Preprocessing must match the model card's inference.py exactly."""
import io

import numpy as np
import pytest
import torch
from PIL import Image, ImageOps
from torchvision import transforms as T

from xraylens.inputs import InputError, assess, load_upload
from xraylens.model import letterbox, preprocess

CFG = {"image_size": 320, "mean": [0.485, 0.456, 0.406], "std": [0.229, 0.224, 0.225]}


def reference(image):
    """Verbatim from Purino/resnet50-chest-xray-pneumonia inference.py."""
    size = CFG["image_size"]
    image = ImageOps.exif_transpose(image).convert("RGB")
    image = ImageOps.pad(image, (size, size), method=Image.Resampling.BILINEAR,
                         color=(0, 0, 0), centering=(0.5, 0.5))
    return T.Compose([T.ToTensor(), T.Normalize(CFG["mean"], CFG["std"])])(image)


def test_matches_model_card_reference(cxr_like):
    boxed, x = preprocess(cxr_like, CFG)
    assert x.shape == (1, 3, 320, 320)
    assert boxed.size == (320, 320)
    torch.testing.assert_close(x[0], reference(cxr_like))


def test_letterbox_pads_with_black_and_keeps_aspect(cxr_like):
    boxed = np.asarray(letterbox(cxr_like, 320))
    # 400x300 → 320x240 content, 40 px bars top and bottom
    assert boxed[:38].max() == 0 and boxed[-38:].max() == 0
    assert boxed[160].max() > 0


def test_grayscale_rgba_and_16bit_all_become_rgb(cxr_like):
    for im in (cxr_like, cxr_like.convert("RGBA"), cxr_like.convert("I;16")):
        assert preprocess(im, CFG)[1].shape == (1, 3, 320, 320)


def _png_bytes(im, fmt="PNG"):
    buf = io.BytesIO()
    im.save(buf, format=fmt)
    return buf.getvalue()


def test_upload_roundtrip(cxr_like):
    loaded = load_upload(_png_bytes(cxr_like), "chest.png")
    assert loaded.image.size == cxr_like.size
    assert loaded.warnings == []


@pytest.mark.parametrize("data,name,msg", [
    (b"", "x.png", "empty"),
    (b"not an image at all", "x.png", "not a readable image"),
    (b"%PDF-1.4", "report.pdf", "not supported"),
])
def test_bad_uploads_raise_friendly_errors(data, name, msg):
    with pytest.raises(InputError, match=msg):
        load_upload(data, name)


def test_truncated_image_rejected(cxr_like):
    data = _png_bytes(cxr_like)
```

```
    with pytest.raises(InputError):
        load_upload(data[: len(data) // 2], "cut.png")


def test_tiny_and_blank_images_rejected():
    with pytest.raises(InputError, match="at least"):
        assess(Image.new("L", (40, 40), 128))
    with pytest.raises(InputError, match="blank"):
        assess(Image.new("L", (400, 400), 128))


def test_warnings_for_colour_inverted_and_odd_aspect(cxr_like):
    colour = Image.merge("RGB", [cxr_like, cxr_like.point(lambda v: 255 - v), cxr_like])
    assert any("colour" in w for w in assess(colour))
    assert any("inverted" in w for w in assess(ImageOps.invert(cxr_like)))
    wide = cxr_like.resize((900, 300))
    assert any("aspect" in w for w in assess(wide))


def test_dicom_upload(cxr_like, tmp_path):
    pydicom = pytest.importorskip("pydicom")
    from pydicom.dataset import FileDataset, FileMetaDataset
    from pydicom.uid import ExplicitVRLittleEndian, generate_uid
    meta = FileMetaDataset()
    meta.MediaStorageSOPClassUID = "1.2.840.10008.5.1.4.1.1.1"
    meta.MediaStorageSOPInstanceUID = generate_uid()
    meta.TransferSyntaxUID = ExplicitVRLittleEndian
    ds = FileDataset(str(tmp_path / "x.dcm"), {}, file_meta=meta, preamble=b"\0" * 128)
    arr = np.asarray(cxr_like, dtype=np.uint16) * 16
    ds.Rows, ds.Columns = arr.shape
    ds.SamplesPerPixel, ds.PhotometricInterpretation = 1, "MONOCHROME1"
    ds.BitsAllocated, ds.BitsStored, ds.HighBit, ds.PixelRepresentation = 16, 12, 11, 0
    ds.Modality, ds.ViewPosition, ds.PatientName = "CR", "PA", "Should^NotAppear"
    ds.PixelData = (4095 - arr).astype(np.uint16).tobytes()  # stored inverted (MONOCHROME1)
    ds.save_as(tmp_path / "x.dcm", enforce_file_format=True)
    loaded = load_upload((tmp_path / "x.dcm").read_bytes(), "x.dcm")
    assert loaded.meta == {"Modality": "CR", "ViewPosition": "PA"}
    assert "PatientName" not in loaded.meta
    # MONOCHROME1 must be flipped back so bone is white: centre brighter than corner
    out = np.asarray(loaded.image)
    assert out[150, 200] > out[5, 5]

```

28 tests/test_inference.py

```python
"""Model, Grad-CAM and metric behaviour. Model tests skip if the weights aren't downloaded."""
import math

import numpy as np
import pytest
import torch
from PIL import Image

from xraylens import evaluation as ev
from xraylens.explainability import attribution_stats, gradcam, overlay
from xraylens.model import DEFAULT_DIR, preprocess
from xraylens.perturbations import PERTURBATIONS

needs_weights = pytest.mark.skipif(not (DEFAULT_DIR / "model.safetensors").exists(),
                                   reason="run the app once (or model.download_model()) to fetch weights")


@pytest.fixture(scope="module")
def model_cfg():
    from xraylens.model import load_model
    return load_model(device=torch.device("cpu"))


@needs_weights
def test_scores_sum_to_one_and_label_follows_threshold(model_cfg, cxr_like):
    from xraylens.model import predict
    model, cfg = model_cfg
    p = predict(cxr_like, model, cfg)
    assert set(p.scores) == {"NORMAL", "PNEUMONIA"}
    assert math.isclose(sum(p.scores.values()), 1.0, abs_tol=1e-5)
    assert p.label == ("PNEUMONIA" if p.pneumonia_score >= cfg["threshold"] else "NORMAL")


@needs_weights
def test_inference_is_deterministic(model_cfg, cxr_like):
    from xraylens.model import predict_batch
    model, cfg = model_cfg
    a = predict_batch([cxr_like, cxr_like], model, cfg)
    assert np.allclose(a[0], a[1], atol=1e-6)


@needs_weights
def test_gradcam_shape_range_and_class_dependence(model_cfg, cxr_like):
    model, cfg = model_cfg
    boxed, x = preprocess(cxr_like, cfg)
    c0, c1 = gradcam(model, x, 0), gradcam(model, x, 1)
    for c in (c0, c1):
        assert c.shape == (320, 320)
        assert c.min() >= 0 and c.max() <= 1 + 1e-6
    assert not np.allclose(c0, c1)
    assert overlay(boxed, c1).size == (320, 320)


@needs_weights
def test_gradcam_leaves_model_unchanged(model_cfg, cxr_like):
    from xraylens.model import predict_batch
    model, cfg = model_cfg
    before = predict_batch([cxr_like], model, cfg)
    gradcam(model, preprocess(cxr_like, cfg)[1], 1)
    assert np.allclose(before, predict_batch([cxr_like], model, cfg))
    assert all(len(m._forward_hooks) == 0 for m in model.modules())


def test_attribution_stats_on_known_maps():
    img = Image.new("L", (320, 320), 0)
    img.paste(Image.new("L", (320, 240), 120), (0, 40))  # letterbox bars top and bottom
    cam = np.zeros((320, 320), np.float32)
```

```
    cam[:40] = 1.0                                      # all heat on the top bar
    s = attribution_stats(cam, img)
    assert s["padding"] == pytest.approx(1.0)
    assert s["padding_area"] == pytest.approx(80 / 320)
    lung = np.zeros((320, 320), bool)
    lung[120:200, 60:260] = True
    cam = lung.astype(np.float32)
    s = attribution_stats(cam, img, lung)
    assert s["lungs"] == pytest.approx(1.0) and s["padding"] == 0


def test_perturbations_preserve_type(cxr_like):
    for key, (_, fn) in PERTURBATIONS.items():
        out = fn(cxr_like)
        assert isinstance(out, Image.Image), key
        assert min(out.size) ⩾ 64


def test_metrics_against_hand_computed():
    y = np.array([1, 1, 1, 1, 0, 0, 0, 0])
    s = np.array([.9, .8, .7, .2, .6, .3, .1, .05])
    m = ev.binary_metrics(y, s, 0.5)
    assert m["confusion"] == {"tp": 3, "fn": 1, "fp": 1, "tn": 3}
    assert m["sensitivity"] == pytest.approx(.75) and m["specificity"] == pytest.approx(.75)
    assert m["precision"] == pytest.approx(.75) and m["f1"] == pytest.approx(.75)
    assert m["auc"] == pytest.approx(14 / 16)  # 0.2 positive beats only 2 of 4 negatives
    lo, hi = m["sensitivity_ci"]
    assert lo < .75 < hi


def test_wilson_matches_known_value():
    lo, hi = ev.wilson(77, 200)  # 38.5%
    assert lo == pytest.approx(.3197, abs=1e-3) and hi == pytest.approx(.4547, abs=1e-3)


def test_calibration_perfect_model_has_zero_ece():
    rng = np.random.default_rng(0)
    s = rng.uniform(size=20000)
    y = (rng.uniform(size=20000) < s).astype(int)
    assert ev.calibration(y, s)["ece"] < 0.02
    assert ev.calibration(np.zeros(100, int), np.ones(100))["ece"] == pytest.approx(1.0)

```

29 requirements.txt

```
# Tested on Python 3.12, macOS arm64 (MPS) and CPU.
streamlit==1.66.0
torch==2.14.1
torchvision==0.29.1
safetensors==0.8.0
huggingface_hub==2.2.0
pillow==12.3.0
numpy==2.5.4
pandas==3.0.6
scikit-learn==1.9.1
scipy==1.18.1
matplotlib==3.11.2
altair==6.3.0
pydicom==3.0.2

```

30 .streamlit/config.toml

```
[theme]
base = "dark"
primaryColor = "#ffb547"
backgroundColor = "#0b0f12"
secondaryBackgroundColor = "#141c21"
textColor = "#ece6d6"
font = "serif"

[server]
maxUploadSize = 20
runOnSave = false

[browser]
gatherUsageStats = false

[client]
toolbarMode = "minimal"

```